

ỦY BAN NHÂN DÂN TP HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO CUỐI KỲ

Học phần: Seminar Chuyên đề

**Đề tài: Xây dựng quy trình Vibe Engineering phát triển
phần mềm quản lý chế độ ăn uống**

Giảng viên hướng dẫn: TS. Đỗ Như Tài

Lớp: DCT122C3

Nhóm sinh viên thực hiện:

STT	Họ và Tên	MSSV
1	Nguyễn Hữu Anh Khoa	3122411098
2	Trang Gia Huy	3122411068
3	Nguyễn Lê Quỳnh Hương	3122411078
4	Võ Thành Danh	3114111012

TP. HỒ CHÍ MINH, THÁNG 5 NĂM 2026

Mục lục

Lời mở đầu	ix
Chương 1: Giới thiệu đề tài	1
1. 1. Lý do chọn đề tài	1
1. 2. Mục tiêu nghiên cứu	2
1.3. Đối tượng và phạm vi của đề tài.....	2
1.4. Phương pháp nghiên cứu	3
1.5. Công nghệ sử dụng	4
1.6. Cấu trúc báo cáo	5
Chương 2: Cơ sở lý thuyết và công nghệ sử dụng	6
2.1. Cơ sở lý thuyết về dinh dưỡng	6
2.1.1. Phương pháp tính toán.....	6
2.1.2. Phân loại tình trạng cơ thể.....	6
2.1.3. Vai trò của chỉ số BMI trong quản lý dinh dưỡng.....	7
2.2. Cơ sở lý thuyết về nhu cầu năng lượng hàng ngày.....	7
2.2.1. Vai trò của chỉ số BMI trong quản lý dinh dưỡng.....	7
2.2.2. Tổng mức tiêu thụ năng lượng hàng ngày (Total Daily Energy Expenditure - TDEE)	8
2.3. Nguyên lý cân bằng năng lượng trong quản trị thể trọng.....	8
2.4. Định nghĩa về Vibe Coding	9
2.5. Định nghĩa về Vibe Engineering	10
2.6. Định nghĩa về Prompt Engineering và Context Engineering	11
2.7. Kiến trúc điều phối và thực thi đa tầng	12
2.8. AI Coding trong quy trình phát triển phần mềm	14
2.8.1. Giai đoạn Tư duy và Thiết kế (Design Loop)	14
2.8.2. Vòng lặp thực thi trọng tâm (Inner Loop)	14

2.8.2. Vòng lặp phát hành và Duy trì (Outer Loop & Submit Loop)	15
2.9. Khảo sát các dự án liên quan	15
2.10. Công nghệ sử dụng	16
2.10.1. Opencode	16
2.10.2. Openrouter	17
2.10.3. Antigravity	17
Chương 3: Phương pháp thực hiện.....	19
3.1 GIAI ĐOẠN 1: PHÂN TÍCH YÊU CẦU VỚI AI.....	19
3.1.1. Thực hiện phân tích yêu cầu với AI	19
3.1.2 Quy trình Brainstorming Động và Vai trò BA của AI Agent.....	20
3.1.3. Khởi tạo PRD.md, SRS.md và User Stories.....	21
3.1.3 Use Case, Business Process.....	24
3.2 GIAI ĐOẠN 2: THIẾT KẾ KIẾN TRÚC HỆ THỐNG VÀ HẠ TẦNG KỸ THUẬT (ARCHITECTURE & BLUEPRINTING).....	29
3.2.1 Thiết kế Kiến trúc	29
3.2.2 Thiết lập Bộ Quy tắc Lập trình Toàn cục Khắt khe (rules.md)	30
3.2.3. Phần con người thực hiện	33
3.3 GIAI ĐOẠN 3: PHÂN RÃ CÔNG VIỆC THÀNH CẤU TRÚC THỰC THI NGUYÊN TỬ.....	38
3.3.1 Nguyên lý Giới hạn Ngữ cảnh.....	38
3.3.2 Thiết kế các Agents cần thiết cho dự án	40
3.4 GIAI ĐOẠN 4: QUẢN TRỊ NGŨ CẢNH VÀ TỐI ƯU HÓA NGŨ CẢNH.....	43
3.4.1 Kiến trúc File Cấu hình Toàn cục Dự án.....	43
3.4.2 Kỹ thuật Đóng gói Ngữ cảnh.....	46
3.5 GIAI ĐOẠN 5: TRIỂN KHAI LẬP TRÌNH BẰNG PHƯƠNG PHÁP VIBE CODING VÀ ĐA TÁC TỬ	47
3.5.1. Thực hiện sinh các giao diện từ context	47

3.5.1 Vòng lặp Thực thi Trọng tâm (Inner Loop) và Triết lý Phát triển Hướng Kiểm thử (TDD)	52
3.5.2 Cơ chế Điều phối, Phân vai và Kiểm tra Chéo Hệ thống Đa Tác tử (Project Lead, Coder, Reviewer).....	55
3.5.3 Tiêu chuẩn Hợp nhất Mã nguồn và Kiểm soát Hành vi của Coder Agent	57
3.6 GIAI ĐOẠN 6: KIỂM THỬ TỰ ĐỘNG VÀ CƠ CHẾ TỰ PHỤC HỒI HỆ THỐNG	58
3.6.1 Chiến lược Kiểm thử và sinh Test Plan.....	58
3.6.2 Chu trình Tự động Quét Log, Bóc tách Stack Trace và Khởi tạo Bản vá	60
3.6.3 Giám sát Trạng thái và Tiêu chuẩn Tỷ lệ Bao phủ Kiểm thử.....	61
3.7 GIAI ĐOẠN 7: TỔNG KẾT, ĐÓNG GÓI.....	63
3.7.1 Đóng gói mã nguồn và Triển khai (Packaging & Deployment).....	63
3.7.2 Hoàn thiện Tài liệu Kỹ thuật (Documentation)	63
Chương 4: Kết quả đạt được và đánh giá	64
4.1 Giới thiệu chương.....	64
4.1 Kết quả Hiện thực hóa Sản phẩm và Giao diện Thực tế (Frontend Layer).....	65
4.1.1 Ứng dụng di động theo dõi dinh dưỡng cá nhân (React Native Client)	66
Giao diện được thiết kế ra cho chức năng tính toán BMI.....	66
4.1.2 Bảng điều khiển quản trị và kiểm duyệt danh mục thực phẩm (React.js Web Admin) ..	70
4.2 Hiện thực hóa Lỗi Nghiệp vụ Backend và Cơ sở Dữ liệu (Backend & Database Layer) ..	73
4.3 Kết quả Thực nghiệm Kiểm thử và Đảm bảo Chất lượng (QA & Test Plan Metrics)	79
4.4 Đánh giá Hiệu quả Phương pháp luận Vibe Coding và Kiến trúc Đa Tác tử.....	80
4.4.1 Đánh giá năng suất lập trình và tốc độ phát hành phiên bản (Velocity).....	80
4.4.2 Đánh giá chu trình tự phục hồi (Self-healing Loop) và kiểm soát lỗi của AI Coder	81
4.4.3 Kết luận	83
4.5 Hạn chế của Hệ thống.....	83
Chương 5: Kết luận và hướng phát triển	85
5.1 Các kết quả đạt được	85

5.2 Hướng phát triển trong tương lai (Future Directions)	85
Tài liệu tham khảo	87

BẢNG PHÂN CÔNG NHIỆM VỤ

Họ và Tên	MSSV	Công việc thực hiện
Nguyễn Hữu Anh Khoa	3122411098	Nhóm trưởng, thực hiện nghiên cứu về vibe engineering
Trang Gia Huy	3122411068	Viết tài liệu, phụ trách thực hiện phần backend của dự án
Nguyễn Lê Quỳnh Hương	3122411078	Làm slide thuyết trình, phụ trách thực hiện phần frontend của dự án
Võ Thành Danh	3114111012	Viết tài liệu, phụ trách thực hiện phần backend của dự án

DANH MỤC HÌNH ẢNH

Hình 2.1. Quy trình vibe coding.....	10
Hình 2.2. Hình ảnh mô tả về kiến trúc thực thi đa tác tử).....	13
Hình 2.3. AI Coding trong quy trình phát triển phần mềm.....	14
Hình 2.4. Hình ảnh tổng quan về ứng dụng VNeNUTRITION.....	16
Hình 2.5. Giao diện của Opencode.....	17
Hình 2.6. Giao diện của Openrouter.....	17
Hình 2.7. Giao diện của Openrouter.....	18
Hình 3.1. Hình ảnh mô tả phương pháp Vibe Engineering của nhóm.....	19
Hình 3.2. Hình ảnh minh họa quá trình phân tích yêu cầu với AI.....	20
Hình 3.3. Prompts sử dụng trong đề yêu cầu AI thực hiện phân tích.....	21
Hình 3.4. Prompts được sử dụng để tạo ra SRS.md.....	22
Hình 3.6. Sơ đồ Use Case.....	24
Hình 3.7. Sơ đồ Business Process của chức năng tính BMI.....	26
Hình 3.8. Sơ đồ Business Process của chức năng lên kế hoạch dinh dưỡng.....	28
Hình 3.9. Prompts được sử dụng để tạo ra các files thiết kế và mô tả hệ thống.....	29
Hình 3.11. Sơ đồ lớp của hệ thống.....	33
Hình 3.12. Sơ đồ ERD.....	35
Hình 3.12. Prompts thực hiện phân rã.....	39
Hình 3.14. Hình ảnh những agents đã được tôi xây dựng trong hệ thống.....	41
Hình 3.15. JSON mô tả metadata.....	42
Hình 3.16. Cấu trúc context của dự án.....	44
Hình 3.18. Prompts sử dụng để tạo context cho giao diện Sau khi đã có được phần context ta tiếp tục sử dụng những context này để có thể chính thức sinh ra các thiết kế giao diện hoàn chỉnh (không phải code).....	48

Hình 3.21. Ảnh giao diện sinh ra cho chức năng lộ trình dinh dưỡng - 1.....	51
Hình 3.22. Ảnh giao diện sinh ra cho chức năng lộ trình dinh dưỡng - 2.....	52
Hình 3.23. Vòng lặp inner loop	54
Hình 3.24. Vòng lặp inner loop	56
Hình 3.25. Prompts sinh test plan.....	58
Hình 3.26. Luồng kiểm thử	59
Hình 4.1. Giao diện được thiết kế của chức năng tính BMI.....	66
Hình 4.2. Giao diện được code ra của chức năng tính BMI.....	67
Hình 4.3. Giao diện được thiết kế của chức năng lên lộ trình dinh dưỡng	68
Hình 4.4.. Giao diện được hiển thị từ code của chức năng lên lộ trình dinh dưỡng	69
Hình 4.5. Giao diện được thiết kế cho chức năng quản lý user	70
Hình 4.6. Giao diện được code cho chức năng quản lý user.....	71
Hình 4.7.. Giao diện được thiết kế cho chức năng quản lý dữ liệu thực phẩm.....	72
Hình 4.8.. Giao diện được code cho chức năng quản lý dữ liệu thực phẩm	73
Hình 4.9.. Mô tả kiến trúc của dự án được AI xây dựng.....	74
Hình 4.10. Code được sinh ra cho việc đăng nhập đăng ký	76
Hình 4.11.. Code được sinh ra cho việc tính toán BMI theo công thức.....	77
Hình 4.12.. Code được sinh ra cho thuật tính toán Lập Kế hoạch Calo Thích ứng Động	78
Hình 4.13.. Hình so sánh tốc độ phát triển giữa quy trình thông thường và quy trình vibe engineering	80
Hình 4.14. Quy trình tự sửa code của AI.....	82

DANH MỤC BẢNG

Bảng 2.1. Bảng phân loại tình trạng cơ thể	7
Bảng 3.1. Mô tả dữ liệu của bảng USERS	36
Bảng 3.2. Mô tả dữ liệu của bảng FOODS.....	37
Bảng 3.3. Mô tả dữ liệu của bảng LOGS	37
Bảng 3.4. Mô tả dữ liệu của bảng LOGS_FOOD	38
Bảng 3.5. Mô tả dữ liệu của bảng BMI_STATUS_ENUM	38
Bảng 3.6. Mô tả dữ liệu của bảng BMI_STATUS_ENUM	38

Lời mở đầu

Trong bối cảnh sức khỏe trở thành ưu tiên hàng đầu, việc kiểm soát chế độ ăn uống khoa học là thách thức lớn đối với nhiều người. Dự án App Nutrition được phát triển như một giải pháp toàn diện, hỗ trợ người dùng quản lý dinh dưỡng cá nhân hóa dựa trên chỉ số BMI và mục tiêu cân nặng cụ thể. Ứng dụng tích hợp khả năng tính toán calo thông minh qua hình ảnh thực phẩm, giúp tối ưu hóa việc theo dõi năng lượng nạp vào hàng ngày. Bằng cách kết nối dữ liệu giữa thực đơn thực tế và biến động thể trọng trên Dashboard, hệ thống giúp người dùng chủ động điều chỉnh lối sống, duy trì kỷ luật dinh dưỡng để đạt được cơ thể khỏe mạnh một cách bền vững.

Chương 1: Giới thiệu đề tài

1. 1. Lý do chọn đề tài

Trong những năm gần đây, sự phát triển mạnh mẽ của Gen AI đang tạo ra nhiều thay đổi trong quy trình phát triển phần mềm. Các công cụ AI hiện nay không chỉ hỗ trợ sinh mã nguồn mà còn có khả năng phân tích yêu cầu, đề xuất kiến trúc hệ thống, tạo giao diện và hỗ trợ kiểm thử. Tuy nhiên, việc sử dụng AI theo hướng Vibe Coding tự phát thường dẫn đến nhiều vấn đề như mã nguồn thiếu nhất quán, khó kiểm soát kiến trúc, phát sinh lỗi logic và khó bảo trì trong quá trình mở rộng hệ thống.

Từ thực tế đó, đề tài “Xây dựng quy trình Vibe Engineering phát triển phần mềm App Nutrition” được thực hiện nhằm nghiên cứu cách ứng dụng AI vào quá trình phát triển phần mềm một cách có kiểm soát. Thay vì chỉ sử dụng AI để sinh mã đơn thuần, đề tài tập trung xây dựng một quy trình phát triển bao gồm phân tích yêu cầu, thiết kế hệ thống, sinh mã, kiểm thử và đánh giá chất lượng đầu ra với sự hỗ trợ của AI.

Để kiểm chứng tính khả thi của quy trình, nhóm lựa chọn bài toán xây dựng ứng dụng quản lý chế độ ăn uống làm hệ thống thực nghiệm. Đây là một bài toán có tính thực tiễn cao do nhu cầu theo dõi sức khỏe và kiểm soát dinh dưỡng ngày càng phổ biến. Đồng thời, bài toán này cũng có độ phức tạp phù hợp để đánh giá khả năng hỗ trợ của AI trong quá trình phát triển phần mềm, bao gồm các chức năng như tính toán BMI, quản lý thực đơn, quản lý calo nạp vào, nhận diện thực phẩm qua hình ảnh và trực quan hóa dữ liệu sức khỏe bằng Dashboard.

Bên cạnh việc xây dựng ứng dụng, đề tài còn tập trung đánh giá vai trò của AI trong việc hỗ trợ kỹ sư phần mềm nâng cao tốc độ phát triển, giảm khối lượng công việc lặp lại và tối ưu quy trình xây dựng sản phẩm. Qua đó, đề tài hướng tới việc nghiên cứu một mô hình phát triển phần mềm có sự kết hợp giữa năng lực của con người và khả năng hỗ trợ của AI trong bối cảnh công nghệ hiện đại.

1. 2. Mục tiêu nghiên cứu

Mục tiêu chính của đề tài là nghiên cứu và xây dựng quy trình phát triển phần mềm theo hướng Vibe Engineering với sự hỗ trợ của Gen AI. Đề tài tập trung đánh giá khả năng ứng dụng AI trong các giai đoạn phân tích yêu cầu, thiết kế hệ thống, sinh mã nguồn, kiểm thử và tối ưu quy trình phát triển phần mềm.

Để phục vụ quá trình nghiên cứu, đề tài triển khai một hệ thống quản lý chế độ ăn uống làm bài toán thực nghiệm. Hệ thống được xây dựng nhằm hỗ trợ người dùng theo dõi các chỉ số sức khỏe cơ bản như BMI, lượng calo nạp vào và kế hoạch dinh dưỡng cá nhân hóa dựa trên thông tin nhân cân nặng, độ tuổi, giới tính, chiều cao của từng người dùng.

Bên cạnh đó, đề tài hướng tới việc tích hợp các công nghệ AI Agents vào hệ thống nhằm hỗ trợ nhận diện thực phẩm qua hình ảnh và phân tích thành phần dinh dưỡng tự động. Thông qua quá trình triển khai, nghiên cứu đánh giá khả năng hỗ trợ của AI Agents trong việc giảm khối lượng công việc thủ công, tăng tốc độ phát triển hệ thống và nâng cao hiệu quả xây dựng phần mềm.

Ngoài việc xây dựng ứng dụng, đề tài còn tập trung đánh giá tính khả thi của mô hình kết hợp giữa kỹ sư phần mềm và AI trong quy trình phát triển hiện đại. Kết quả nghiên cứu hướng tới việc đề xuất một quy trình phát triển phần mềm có kiểm soát, tận dụng được khả năng hỗ trợ của AI nhưng vẫn đảm bảo chất lượng hệ thống, tính ổn định và khả năng bảo trì trong thực tế.

1.3. Đối tượng và phạm vi của đề tài

Đối tượng nghiên cứu của đề tài là quy trình phát triển phần mềm có sự hỗ trợ của trí tuệ nhân tạo theo hướng Vibe Engineering, đồng thời áp dụng quy trình này vào việc xây dựng hệ thống quản lý chế độ ăn uống cá nhân.

- Về nội dung nghiên cứu, đề tài tập trung vào các chức năng chính gồm:
 - Tính toán chỉ số BMI
 - Xây dựng lộ trình dinh dưỡng theo mục tiêu tăng hoặc giảm cân

- Trực quan hóa dữ liệu sức khỏe bằng Dashboard

Hỗ trợ nhận diện thực phẩm thông qua hình ảnh để phân tích thông tin dinh dưỡng

Bên cạnh việc xây dựng hệ thống, đề tài tập trung nghiên cứu khả năng ứng dụng AI trong các giai đoạn phát triển phần mềm như phân tích yêu cầu, hỗ trợ sinh mã nguồn, thiết kế giao diện và tối ưu quy trình triển khai hệ thống.

Về phạm vi triển khai, hệ thống hướng đến nhóm người dùng cá nhân có nhu cầu quản lý chế độ dinh dưỡng và theo dõi sức khỏe cơ bản. Đề tài không đi sâu vào các chức năng y khoa chuyên sâu như chẩn đoán bệnh, tư vấn điều trị hoặc huấn luyện mô hình AI ở mức nghiên cứu học thuật.

1.4. Phương pháp nghiên cứu

Để thực hiện đề tài, nhóm sử dụng kết hợp nhiều phương pháp nghiên cứu khác nhau nhằm xây dựng và đánh giá quy trình phát triển phần mềm theo hướng Vibe Engineering.

- Về phương pháp nghiên cứu tài liệu, đề tài tiến hành tìm hiểu các kiến thức liên quan đến dinh dưỡng, chỉ số BMI, TDEE, các phương pháp tính toán nhu cầu năng lượng, đồng thời nghiên cứu các mô hình phát triển phần mềm có sự hỗ trợ của trí tuệ nhân tạo. Ngoài ra, nhóm cũng tham khảo các công nghệ liên quan đến phát triển ứng dụng di động, kiến trúc Backend và các công cụ AI hỗ trợ sinh mã nguồn.
- Về phương pháp thiết kế và phát triển hệ thống, đề tài xây dựng hệ thống quản lý chế độ ăn uống. Phần Backend được phát triển nhằm xử lý logic nghiệp vụ, quản lý dữ liệu người dùng và cung cấp API cho ứng dụng. Phần giao diện người dùng được xây dựng trên nền tảng di động nhằm hỗ trợ nhập liệu, theo dõi dữ liệu sức khỏe và trực quan hóa thông tin bằng Dashboard, đổi món thông minh giúp hạn chế được việc người dùng không có đủ nguyên liệu để chế biến món ăn theo lộ trình được đặt ra hoặc muốn đổi món khác tránh nhàm chán

giúp tăng người dùng dễ dàng đạt được mục tiêu mong muốn. Bên cạnh đó, hệ thống tích hợp AI hỗ trợ nhận diện thực phẩm thông qua hình ảnh để phân tích thông tin dinh dưỡng.

Trong quá trình phát triển, đề tài áp dụng phương pháp Vibe Engineering bằng cách sử dụng AI hỗ trợ ở nhiều giai đoạn như phân tích yêu cầu, sinh mã nguồn, thiết kế giao diện và tối ưu quy trình phát triển phần mềm. Các kết quả sinh ra từ AI được kiểm tra, hiệu chỉnh và đánh giá lại nhằm đảm bảo tính chính xác và khả năng bảo trì của hệ thống.

1.5. Công nghệ sử dụng

Để hiện thực hóa giải pháp dinh dưỡng thông minh, dự án được xây dựng dựa trên sự giao thoa giữa các framework hiện đại và công nghệ phát triển theo triết lý Vibe Engineering – phương thức lấy trí tuệ nhân tạo làm nhân điều phối bối cảnh nhằm rút ngắn khoảng cách từ ý tưởng đến sản phẩm thực tế (Production-ready).

Công nghệ điều phối Vibe Engineering & Hệ sinh thái Tác tử (Multi-Agent): Đóng vai trò là phương thức vận hành chính, Vibe Engineering sử dụng Kiến trúc Prompt đa lớp (Multi-layer Prompt Architecture) để quản trị bối cảnh từ tầng chiến lược đến tầng thực thi mã nguồn. Quy trình này được vận hành bởi kiến trúc Vibe Multi-Agent, nơi các tác tử AI chuyên biệt phối hợp song song:

- Architect Agent: Quản lý bối cảnh tổng thể, đảm bảo sự đồng bộ giữa logic tính toán BMI/TDEE trên Backend và giao diện Dashboard trên Frontend.
- Feature Agents: Thực thi các module riêng lẻ như nhận diện thực phẩm, gợi ý thực đơn và quản lý nhật ký ăn uống.
- Quality & Guardrails Agent: Đóng vai trò giám sát, sử dụng các Rào chắn kỹ thuật (Guardrails) và bộ kiểm thử TDD để đảm bảo mọi mã nguồn và nội dung dinh dưỡng luôn đạt độ chính xác cao, an toàn y khoa và không vi phạm các tiêu chuẩn kỹ thuật.

Framework phát triển (Frontend & Backend):

- Spring Boot : Được sử dụng để xây dựng hệ thống Backend, cung cấp môi trường ổn định để các tác tử AI triển khai các dịch vụ tính toán và quản lý nghiệp vụ dinh dưỡng phức tạp.
- React và Reat-Native: Bộ đôi framework chủ đạo cho phía Frontend, giúp chuyển hóa các cảm quan thiết kế thành giao diện đa nền tảng, hỗ trợ tốt các tính năng tương tác như Dashboard trực quan.

Hệ quản trị cơ sở dữ liệu (Database):

PostgreSQL: Cung cấp khả năng lưu trữ linh hoạt cho các tập dữ liệu dinh dưỡng có cấu trúc rõ ràng. Cơ sở dữ liệu này cho phép các AI Agents dễ dàng truy xuất và mở rộng thuộc tính thực phẩm mà không cần can thiệp sâu vào sơ đồ dữ liệu truyền thống, tối ưu hóa việc quản lý lịch sử ăn uống và biến động cân nặng.

Tối ưu hóa vòng lặp:

Trong hệ sinh thái này, con người đóng vai trò là kiến trúc sư trưởng. Thông qua các báo cáo từ Dashboard, người điều phối sẽ đánh giá sự tương quan giữa dữ liệu thực tế và mục tiêu sức khỏe, từ đó tinh chỉnh bối cảnh và các lớp Prompt để hệ thống Multi-Agent tiếp tục tối ưu hóa hệ thống một cách liên tục.

1.6. Cấu trúc báo cáo

Đề tài được tổ chức thành 5 chương chính như sau:

Chương 1. Giới thiệu đề tài: Giới thiệu tổng quan về đề tài gồm lý do chọn đề tài, mục tiêu của đề tài, phương pháp thực hiện, giới thiệu về phần mềm mục tiêu.

Chương 2. Cơ sở lý thuyết.

Chương 3. Phương pháp thực hiện đề tài

Chương 4. Kết quả đạt được và những nhược điểm

Chương 5. Kết luận

Chương 2: Cơ sở lý thuyết và công nghệ sử dụng

2.1. Cơ sở lý thuyết về dinh dưỡng

Chỉ số khối cơ thể (Body Mass Index - BMI) là một chỉ số nhân trắc học được sử dụng phổ biến trên toàn thế giới để đánh giá tình trạng thể trọng của một người trưởng thành dựa trên mối tương quan giữa chiều cao và cân nặng. Chỉ số này được nhà khoa học người Bỉ Adolphe Quetelet đưa ra vào năm 1832 và hiện nay được Tổ chức Y tế Thế giới (WHO) công nhận là công cụ sàng lọc tiêu chuẩn để xác định các trạng thái thiếu cân, thừa cân hoặc béo phì.

2.1.1. Phương pháp tính toán

Chỉ số BMI được tính toán dựa trên trọng lượng cơ thể và bình phương chiều cao của một cá nhân.

Công thức chuẩn để xác định BMI như sau:

$$BMI = \frac{W}{H^2}$$

Trong đó:

- W (Weight): Trọng lượng cơ thể, tính bằng kilôgam (kg).
- H (Height): Chiều cao cơ thể, tính bằng mét (m).

Chỉ số này cung cấp một con số định lượng cho phép so sánh trạng thái cơ thể của các cá nhân có chiều cao khác nhau một cách khách quan.

2.1.2. Phân loại tình trạng cơ thể

Dựa trên giá trị BMI thu được, tình trạng sức khỏe của người dùng sẽ được phân loại vào các nhóm khác nhau. Tuy nhiên, do đặc điểm cấu trúc cơ thể và phân bố mỡ nội tạng, các ngưỡng phân loại có sự điều chỉnh giữa tiêu chuẩn quốc tế và tiêu chuẩn dành riêng cho khu vực Châu Á (IDI & WPRO):

Phân loại	Chỉ số BMI (Tiêu chuẩn WHO)	Chỉ số BMI (Tiêu chuẩn Châu Á - IDI & WPRO)
Cân nặng thấp (Gầy)	< 18.5	< 18.5
Bình thường	18.5 – 24.9	18.5 – 22.9

Tiền béo phì / Thừa cân	25.0 – 29.9	23.0 – 24.9
Béo phì độ I	30.0 – 34.9	25.0 – 29.9
Béo phì độ II	35.0 – 39.9	≥ 30.0
Béo phì độ III	≥ 40.0	≥ 40.0

Bảng 2.1. Bảng phân loại tình trạng cơ thể

2.1.3. Vai trò của chỉ số BMI trong quản lý dinh dưỡng

Trong lĩnh vực dinh dưỡng, BMI đóng vai trò là kim chỉ nam bước đầu để xác định mục tiêu năng lượng:

- Xác định mục tiêu sức khỏe: Chỉ số này giúp nhận diện sớm tình trạng béo phì hoặc suy dinh dưỡng, từ đó đưa ra hướng đi cụ thể là cần giảm cân, tăng cân hay duy trì cân nặng hiện tại.
- Đánh giá rủi ro bệnh lý: Chỉ số BMI cao thường có mối liên hệ mật thiết với các bệnh lý mãn tính không lây như cao huyết áp, tiểu đường tuýp 2, và các bệnh về tim mạch. Ngược lại, BMI quá thấp có thể là dấu hiệu của tình trạng thiếu hụt dưỡng chất hoặc suy giảm hệ miễn dịch.
- Cơ sở cho lộ trình năng lượng: Kết quả phân loại BMI là căn cứ quan trọng để tính toán Tổng mức tiêu thụ năng lượng hàng ngày (TDEE), giúp người dùng thiết lập chế độ ăn thặng dư hoặc thâm hụt Calo một cách hợp lý.

2.2. Cơ sở lý thuyết về nhu cầu năng lượng hàng ngày

Nền tảng của quản lý thể trọng trong khoa học dinh dưỡng hiện đại dựa trên nguyên lý cân bằng năng lượng (Energy Balance). Để xây dựng lộ trình cá nhân hóa, hệ thống tập trung phân tích hai chỉ số định lượng then chốt là BMR và TDEE.

2.2.1. Vai trò của chỉ số BMI trong quản lý dinh dưỡng

BMR được định nghĩa là mức tiêu hao năng lượng tối thiểu cần thiết để duy trì các chức năng sinh lý sống còn của cơ thể trong trạng thái nghỉ ngơi hoàn toàn, không tiêu hóa và ở môi trường nhiệt độ trung hòa. Các chức năng này bao gồm hoạt động của hệ tuần hoàn, hệ hô hấp, duy trì thân nhiệt và hoạt động của các tế bào thần kinh.

Trong nghiên cứu này, hệ thống áp dụng công thức Mifflin-St Jeor, một phương pháp được kiểm chứng có độ chính xác cao nhất đối với các quần thể dân cư hiện đại để tính toán giá trị BMR:

- Đối với nam giới: $BMR = (10 \times W) + (6.25 \times H) - (5 \times A) + 5$
- Đối với nữ giới: $BMR = (10 \times W) + (6.25 \times H) - (5 \times A) - 161$

(Trong đó: W là cân nặng tính bằng kg; H là chiều cao tính bằng cm; A là tuổi tính bằng năm).

2.2.2. Tổng mức tiêu thụ năng lượng hàng ngày (Total Daily Energy Expenditure - TDEE)

Chỉ số TDEE đại diện cho tổng lượng năng lượng thực tế mà một cá nhân tiêu thụ trong chu kỳ 24 giờ, bao gồm năng lượng nghỉ ngơi và năng lượng tiêu hao thông qua các hoạt động thể chất. Đây là thông số quyết định để hệ thống xác định trạng thái thặng dư hoặc thâm hụt năng lượng.

Giá trị TDEE được xác lập dựa trên tích số của BMR và hệ số hoạt động thể chất (Physical Activity Level - PAL):

Hệ số PAL được phân loại định lượng dựa trên cường độ vận động đặc trưng của người dùng:

$$TDEE = BMR \times PAL$$

Hệ số PAL được phân loại định lượng dựa trên cường độ vận động đặc trưng của người dùng:

- Low (Rất ít hoặc không hoạt động thể chất): 1.2
- Average (Hoạt thể chất từ 1 đến 3 ngày trong tuần): 1.55
- High (Hoạt động thể chất từ 4 ngày hoặc nhiều hơn): 1.725

2.3. Nguyên lý cân bằng năng lượng trong quản trị thể trọng

Hệ thống điều phối dinh dưỡng vận hành dựa trên cơ chế đối chiếu giữa năng lượng nạp vào (Calories In) và năng lượng tiêu thụ (Calories Out), được chia thành ba kịch bản quản lý:

- Trạng thái Cân bằng (Maintenance): Khi mức năng lượng nạp vào tương đương với giá trị TDEE. Ở trạng thái này, trọng lượng cơ thể được duy trì ở mức ổn định.
- Trạng thái Thâm hụt (Caloric Deficit): Xảy ra khi năng lượng nạp vào thấp hơn TDEE. Cơ thể kích hoạt cơ chế huy động năng lượng từ các mô dự trữ (mô mỡ). Để đảm bảo an toàn sinh học, mức thâm hụt khuyến nghị thường không vượt quá

20% - 25% tổng mức TDEE nhằm tránh các phản ứng tiêu cực của hệ chuyển hóa.

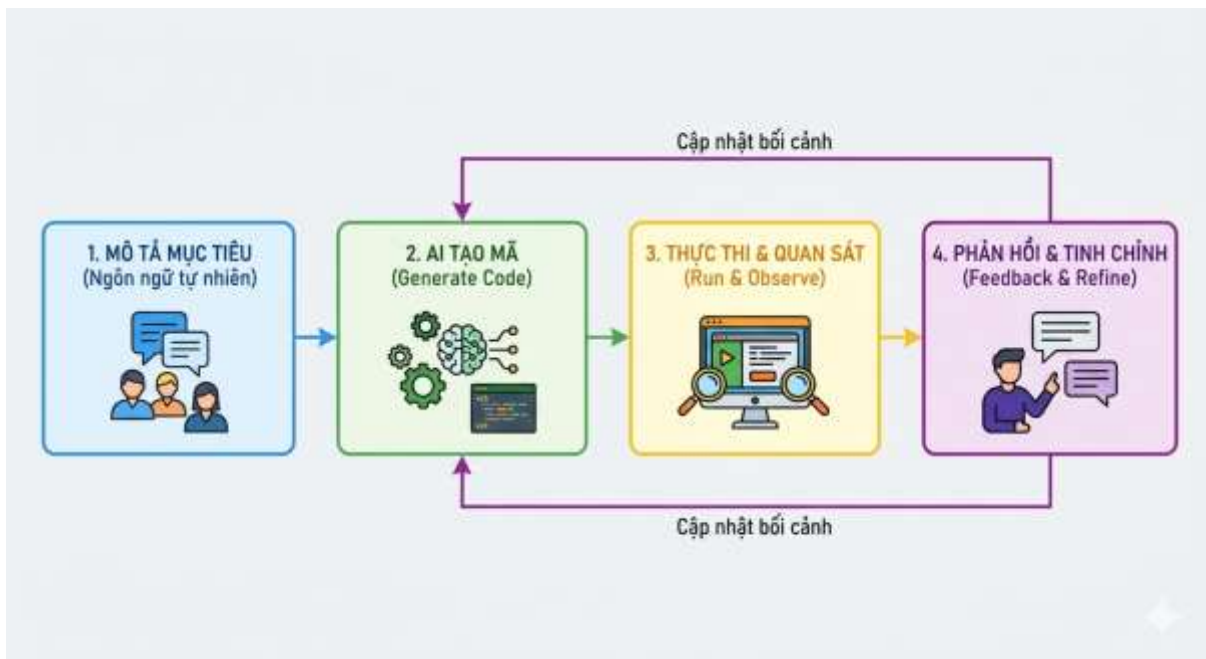
- Trạng thái Thặng dư (Caloric Surplus): Xảy ra khi năng lượng nạp vào cao hơn TDEE. Năng lượng dư thừa được ưu tiên cho quá trình tổng hợp mô (đặc biệt là mô cơ khi có kích thích từ vận động kháng lực) hoặc tích lũy năng lượng dự trữ.

Cấu trúc lý thuyết này tạo thành bộ quy tắc logic để hệ thống tính toán, phân tích hình ảnh thực phẩm và đưa ra các khuyến nghị dinh dưỡng chính xác trên giao diện người dùng.

2.4. Định nghĩa về Vibe Coding

Vibe Coding là một phương thức phát triển phần mềm mà ở đó vai trò chính của con người chuyển dịch từ việc viết mã nguồn thủ công sang việc hướng dẫn một trợ lý AI (LLM) để tạo ra, tinh chỉnh và sửa lỗi ứng dụng thông qua quá trình đối thoại bằng ngôn ngữ tự nhiên. Thuật ngữ này được Andrej Karpathy đưa ra vào tháng 2 năm 2025 để mô tả một trạng thái lập trình mà người dùng hoàn toàn tin tưởng vào các bước tiến nhảy vọt của trí tuệ nhân tạo và thậm chí "quên đi sự tồn tại của mã nguồn".

Bản chất của Vibe Coding nằm ở mức độ trừu tượng hóa cao: thay vì tập trung vào chi tiết triển khai, cú pháp hay dấu câu, người dùng chỉ cần mô tả mục tiêu (intent) bằng ngôn ngữ tự nhiên (như tiếng Anh hoặc tiếng Việt), AI sau đó diễn giải yêu cầu này để sản xuất mã nguồn tương ứng. Trong quy trình này, con người đóng vai trò là người gợi ý (prompter), người hướng dẫn và người kiểm thử, thực hiện một vòng lặp chặt chẽ.



Hình 2.1. Quy trình vibe coding

Sự trỗi dậy của Vibe Coding đánh dấu kỷ nguyên "ngôn ngữ lập trình mới nóng nhất hiện nay là ngôn ngữ tự nhiên", cho phép cả những người không chuyên có thể xây dựng các công cụ cá nhân hóa phức tạp.

2.5. Định nghĩa về Vibe Engineering

Vibe Engineering (hay Kỹ thuật hóa cảm hứng) là một bước tiến lớn từ Vibe Coding, bắt nguồn từ nhu cầu chuyển dịch từ các thử nghiệm bản mẫu nhanh sang việc xây dựng các hệ thống sản xuất thực tế. Trong Vibe Engineering, nhà phát triển sử dụng AI như một cộng sự đắc lực (collaborator) chứ không phải là sự thay thế cho năng lực phán đoán kỹ thuật. Vai trò của kỹ sư tiến hóa thành một kiến trúc sư cấp cao và người đánh giá, chịu trách nhiệm thiết kế hệ thống, đảm bảo chất lượng mã, kiểm thử và quản trị rủi ro.

Trái ngược với Vibe Coding vốn thường dẫn đến tình trạng "context amnesia" (quên bối cảnh - mã hoạt động nhưng không thể giải trình hoặc kiểm toán), Vibe Engineering thiết lập các quy trình kỷ luật thông qua ba trụ cột cốt lõi:

- Đánh giá mã nguồn nâng cao (Advanced Code Review): Kỹ sư phải hiểu logic đằng sau mỗi dòng mã do AI tạo ra. Quy tắc vàng là "không commit mã nếu không thể giải thích chức năng của nó cho người khác".

- Kiểm thử cộng tác (Collaborative Testing): Sử dụng AI để viết các đặc tả kiểm thử và áp dụng Phát triển dựa trên Kiểm thử (TDD) như một công cụ bắt buộc để AI tạo mã đáp ứng đúng các ràng buộc kỹ thuật.
- Đánh giá rủi ro nghiêm ngặt (Rigorous Risk Assessment): Trước khi triển khai, mọi thành phần AI tạo ra phải trải qua quy trình kiểm tra bảo mật và hiệu năng nhằm tránh các lỗ hổng hoặc lỗi logic tiềm ẩn.

Về bản chất, Vibe Engineering là việc áp dụng các nguyên tắc kỹ thuật nghiêm ngặt vào quá trình tương tác với AI, thay thế sự tin tưởng mù quáng bằng việc tìm hiểu bối cảnh chính xác.

2.6. Định nghĩa về Prompt Engineering và Context Engineering

Prompt Engineering là kỹ thuật thiết kế, tối ưu hóa và tinh chỉnh các đầu vào văn bản (prompts) để hướng dẫn các mô hình ngôn ngữ lớn (LLM) thực hiện các tác vụ cụ thể và sản xuất các đầu ra chính xác, đáng tin cậy. Về bản chất, đây là nghệ thuật giao tiếp với máy tính thông qua ngôn ngữ tự nhiên, đóng vai trò như một "lớp trừu tượng lập trình" mới (Software 3.0). Kỹ thuật này không chỉ đơn thuần là việc tìm kiếm từ ngữ phù hợp mà còn là việc cấu trúc hóa chỉ thị sao cho mô hình có thể diễn giải đúng ý định người dùng.

Các kỹ thuật phổ biến trong Prompt Engineering bao gồm: Phân bổ vai trò (Role prompting), Lập luận từng bước (Chain-of-Thought), và Cung cấp ví dụ mẫu (Few-shot learning). Mặc dù quan trọng, Prompt Engineering thường chỉ tập trung vào các tương tác đơn lẻ hoặc ngắn hạn (single-turn) và phụ thuộc nhiều vào khả năng ngôn ngữ của người vận hành.

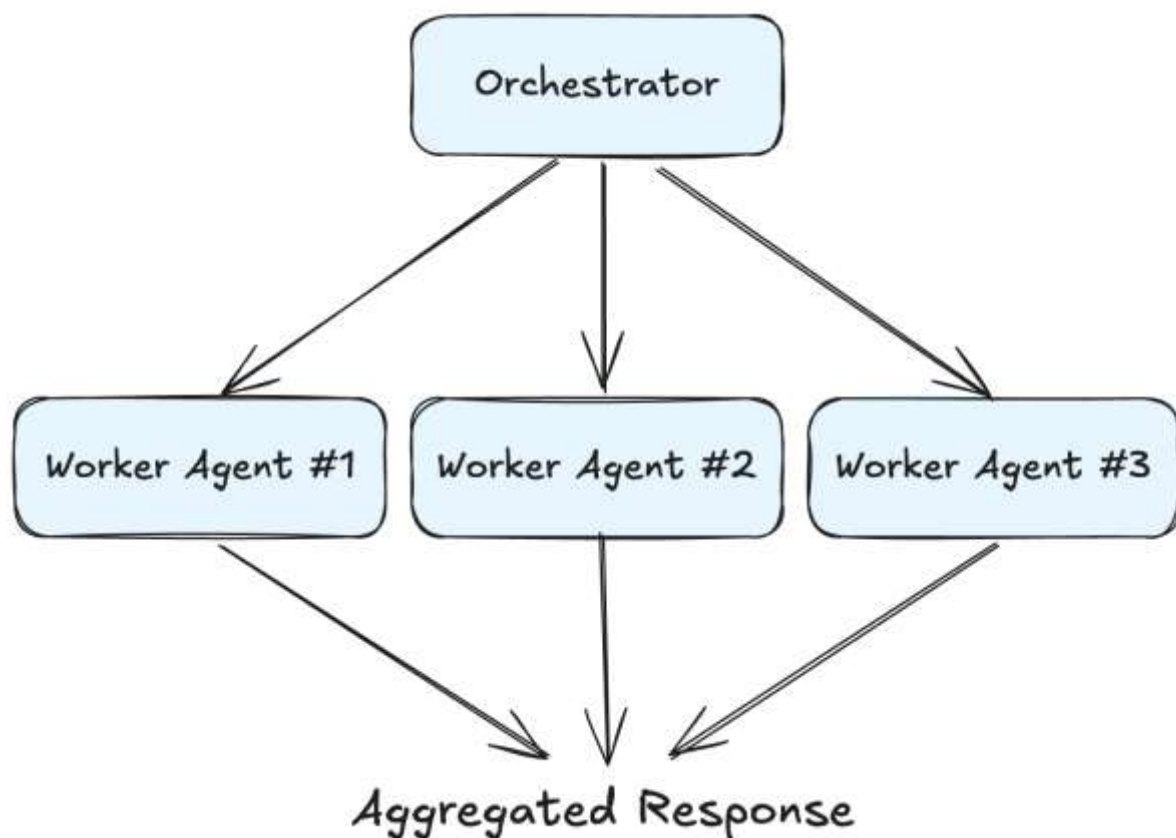
Context Engineering là một kỹ thuật cao hơn, mang tính hệ thống hơn, tập trung vào việc thiết kế, cấu trúc và quản trị toàn bộ môi trường thông tin mà trí tuệ nhân tạo truy cập trong suốt quá trình suy luận. Trong khi Prompt Engineering hỏi "Tôi nên diễn đạt yêu cầu này như thế nào?", Context Engineering giải quyết bài toán "Mô hình cần biết những thông tin gì ngay bây giờ để đưa ra phản hồi đúng nhất?". Xuất hiện như một sự tiến hóa tất yếu từ giữa năm 2025, kỹ thuật này xử lý các thách thức của hệ thống thực tế mà

phương pháp gợi ý đơn thuần không thể đáp ứng, như duy trì bộ nhớ dài hạn, quản trị dữ liệu quy mô lớn và đảm bảo tính tuân thủ.

Context Engineering coi ngữ cảnh là một tài nguyên hữu hạn và quý giá, bao gồm 5 lớp cốt lõi: Lệnh hệ thống (Persona & Constraints), Công cụ (Function calling/MCP), Bộ nhớ (Memory), Trạng thái (Task progress). Việc làm chủ Context Engineering giúp hệ thống tránh khỏi hiện tượng "Context Poisoning" (nhiễm độc thông tin) hoặc "Context Confusion" (nhầm lẫn công cụ), từ đó biến trí tuệ nhân tạo từ một chatbot đơn thuần thành một cộng sự đắc lực có khả năng thấu hiểu sâu sắc bối cảnh doanh nghiệp và dự án.

2.7. Kiến trúc điều phối và thực thi đa tầng

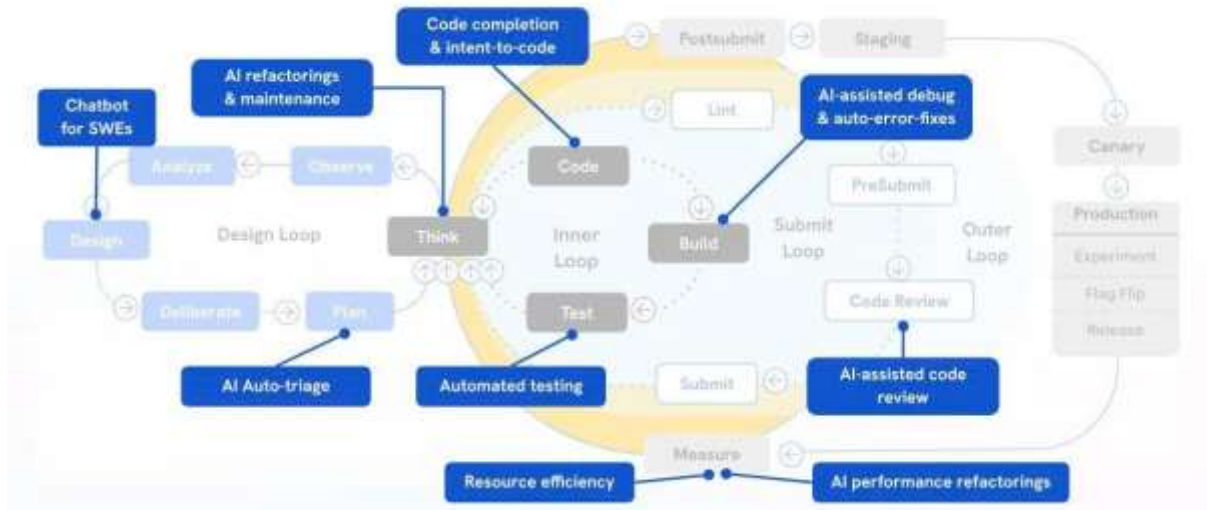
Kỷ nguyên Vibe Engineering năm 2026 đánh dấu cuộc đại tu trong cách con người kiến tạo phần mềm: chuyển từ việc gõ mã nguồn thủ công sang điều phối "cảm quan" kỹ thuật hệ thống. Trọng tâm của phương thức này là Context Engineering (Làm chủ bối cảnh), nơi kỹ sư đóng vai trò kiến trúc sư trưởng, chuyển hóa ý niệm thành các đặc tả chiến lược để dẫn dắt trí tuệ nhân tạo. Với sự hỗ trợ từ các IDE thế hệ mới có khả năng thấu hiểu toàn bộ biểu đồ dự án, mọi thay đổi quy mô lớn giờ đây có thể thực hiện qua ngôn ngữ tự nhiên, vận hành trong vòng lặp phản hồi tức thì để tối ưu hóa hiệu suất và giải phóng con người khỏi các tác vụ lặp lại.



Hình 2.2. Hình ảnh mô tả về kiến trúc thực thi đa tác tử

Trong dự án App Nutrition, mô hình này được cụ thể hóa bằng một "nhạc trưởng" điều phối song song các đơn vị chuyên môn về dịch vụ nền, giao diện đa nền tảng và thuật toán AI. Thay vì làm việc cô lập, các bộ phận này tương tác qua hệ thống "hộp thư bồi cảnh", giúp tự động cập nhật và phản biện kiến trúc để duy trì sự liền mạch cho toàn bộ mã nguồn. Để đảm bảo tính tin cậy, một "hiến pháp" kỹ thuật nghiêm ngặt cùng quy trình kiểm thử tự động (TDD) được thiết lập, buộc mọi logic từ tính toán BMI đến gợi ý thực đơn phải đạt độ chính xác tuyệt đối, đồng thời ngăn chặn triệt để rủi ro "quên bồi cảnh" của AI.

2.8. AI Coding trong quy trình phát triển phần mềm



Hình 2.3. AI Coding trong quy trình phát triển phần mềm

Sự tích hợp của AI không chỉ là việc thay đổi công cụ, mà là tái cấu trúc toàn bộ vòng đời phát triển phần mềm (SDLC). Theo sơ đồ phân tích, tầm ảnh hưởng của AI trải dài qua ba vòng lặp cốt lõi: Design Loop, Inner Loop, và Outer Loop.

2.8.1. Giai đoạn Tư duy và Thiết kế (Design Loop)

AI đóng vai trò là người cộng tác phân tích nghiệp vụ là lên ý tưởng ngay từ những bước đầu tiên:

- Chatbot for SWEs & AI Auto-triage: Ở các giai đoạn Design, Analyze và Plan, AI hỗ trợ kỹ sư phần mềm tóm tắt yêu cầu, phân loại vấn đề tự động và đề xuất kiến trúc hệ thống. Điều này giúp giảm thiểu sai sót từ bước lập kế hoạch và tối ưu hóa tài liệu thiết kế.
- Chuyển đổi từ Think sang Code: AI giúp rút ngắn khoảng cách giữa ý tưởng và thực thi. Thay vì phải viết mọi dòng code từ đầu, các tác tử AI có thể chuyển đổi các đặc tả bối cảnh (Context Specs) thành khung mã nguồn sơ khởi.

2.8.2. Vòng lặp thực thi trọng tâm (Inner Loop)

Đây là nơi AI thể hiện sức mạnh rõ rệt nhất thông qua việc trực tiếp tham gia vào quá trình viết mã và kiểm thử:

- Code completion & Intent-to-code: AI dự đoán ý định của lập trình viên để hoàn thiện mã nguồn thời gian thực. Điều này không chỉ tăng tốc độ gõ phím mà còn gợi ý các hàm, thư viện tối ưu dựa trên ngữ cảnh dự án.

- Automated Testing & AI-assisted Debug: Trong giai đoạn Test và Build, AI tự động hóa việc tạo các bộ kiểm thử (Unit tests) và phát hiện các lỗi tiềm ẩn (Error-fixes). Khả năng tự động vá lỗi (auto-error-fixes) giúp giảm thiểu thời gian chờ đợi giữa việc phát hiện lỗi và xử lý lỗi.

2.8.2. Vòng lặp phát hành và Duy trì (Outer Loop & Submit Loop)

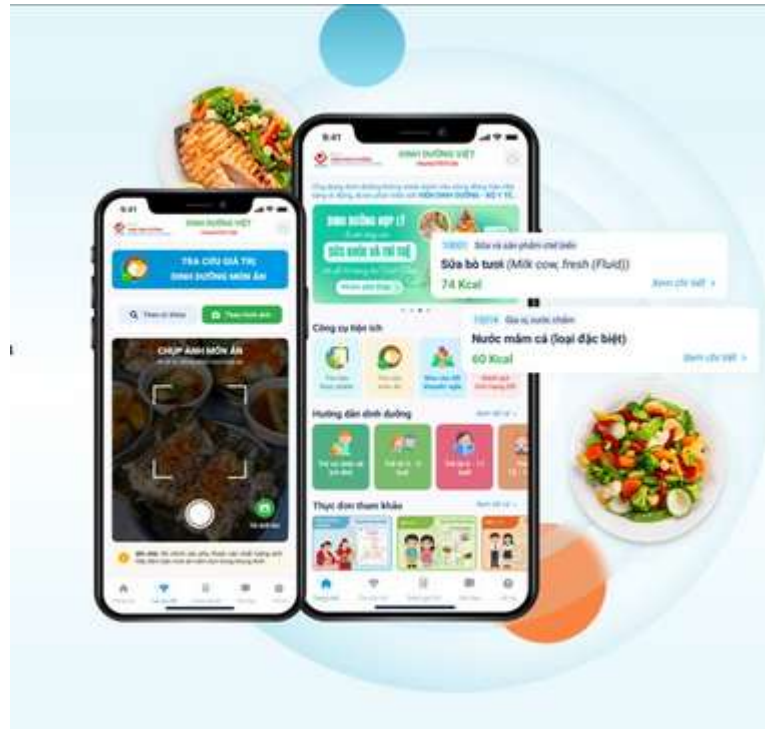
AI đảm bảo phần mềm khi ra mắt đạt được tiêu chuẩn chất lượng và hiệu năng cao nhất:

- AI-assisted Code Review: Thay vì chỉ dựa vào sự kiểm tra thủ công, AI tham gia soát xét mã nguồn, phát hiện các lỗ hổng bảo mật hoặc các đoạn code không tối ưu (Bad smells) trước khi Submit.
- AI Performance Refactorings: Sau khi phần mềm được triển khai, AI tiếp tục quan sát (Observe) và đo lường (Measure) hiệu năng thực tế. Từ đó, nó đề xuất các phương án tái cấu trúc (Refactoring) để tối ưu hóa tài nguyên (Resource efficiency), giúp hệ thống vận hành bền vững hơn.

2.9. Khảo sát các dự án liên quan

VNeNUTRITION là một ứng dụng hỗ trợ quản lý chế độ ăn uống và theo dõi dinh dưỡng, được phát triển với mục tiêu hướng đến người dùng Việt Nam. Ứng dụng tập trung vào việc xây dựng một hệ thống đơn giản, trực quan và phù hợp với thói quen ăn uống hằng ngày. Mục tiêu của hệ thống không chỉ dừng lại ở việc tính toán lượng calories tiêu thụ mà còn hỗ trợ người dùng theo dõi sức khỏe, kiểm soát cân nặng và duy trì chế độ ăn uống lành mạnh trong thời gian dài.

Giao diện của VNeNUTRITION được thiết kế theo hướng tối giản và trực quan nhằm tạo cảm giác dễ sử dụng đối với người dùng phổ thông. Các chức năng chính như theo dõi calories, nhập nguyên liệu món ăn, thống kê cân nặng và dashboard phân tích được bố trí rõ ràng, giúp người dùng có thể thao tác nhanh chóng mà không cần nhiều thời gian làm quen.



Hình 2.4. Hình ảnh tổng quan về ứng dụng VNeNUTRITION

2.10. Công nghệ sử dụng

2.10.1. Opencode

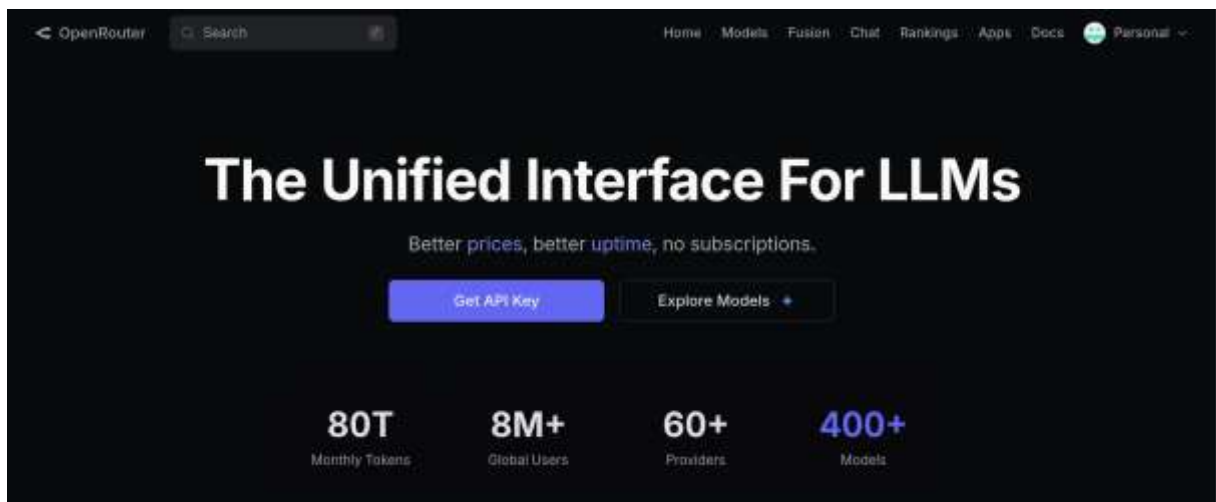
Opencode là một công cụ hỗ trợ lập trình bằng AI theo hướng mã nguồn mở, được thiết kế để hoạt động trực tiếp trong môi trường terminal và IDE của lập trình viên. Công nghệ này cho phép người dùng tương tác với nhiều mô hình AI khác nhau nhằm hỗ trợ sinh mã nguồn, phân tích lỗi, chỉnh sửa dự án và tự động hóa các tác vụ phát triển phần mềm. Điểm nổi bật của Opencode là khả năng tích hợp linh hoạt với các nhà cung cấp mô hình như OpenRouter hoặc các mô hình cục bộ thông qua API. Trong dự án App Nutrition, Opencode được sử dụng nhằm hỗ trợ quá trình phát triển backend, frontend và quản lý mã nguồn, giúp tăng tốc độ lập trình, giảm thời gian xử lý lỗi và nâng cao hiệu quả xây dựng hệ thống.



Hình 2.5. Giao diện của Opencode

2.10.2. Openrouter

OpenRouter là một nền tảng trung gian cung cấp khả năng truy cập đến nhiều mô hình trí tuệ nhân tạo thông qua một API thống nhất. Thay vì phải cấu hình riêng cho từng nhà cung cấp AI như OpenAI, Anthropic hoặc Google, lập trình viên chỉ cần sử dụng một API duy nhất để chuyển đổi linh hoạt giữa các mô hình khác nhau. OpenRouter hỗ trợ tối ưu chi phí, cân bằng hiệu suất và giúp lựa chọn mô hình phù hợp với từng nhu cầu xử lý cụ thể. Trong quá trình phát triển dự án VNeNUTRITION, OpenRouter được sử dụng để kết nối các mô hình AI phục vụ hỗ trợ lập trình và xử lý dữ liệu. Việc sử dụng nền tảng này giúp hệ thống dễ mở rộng hơn, đồng thời đơn giản hóa việc quản lý và tích hợp các dịch vụ AI.



Hình 2.6. Giao diện của Openrouter

2.10.3. Antigravity

Antigravity là một nền tảng hỗ trợ xây dựng và quản lý các tác vụ phát triển phần mềm theo hướng tự động hóa bằng AI. Công cụ này cung cấp khả năng tổ chức workflow, hỗ trợ quản lý tác vụ và tối ưu quy trình phát triển ứng dụng thông qua các agent AI. Antigravity được thiết

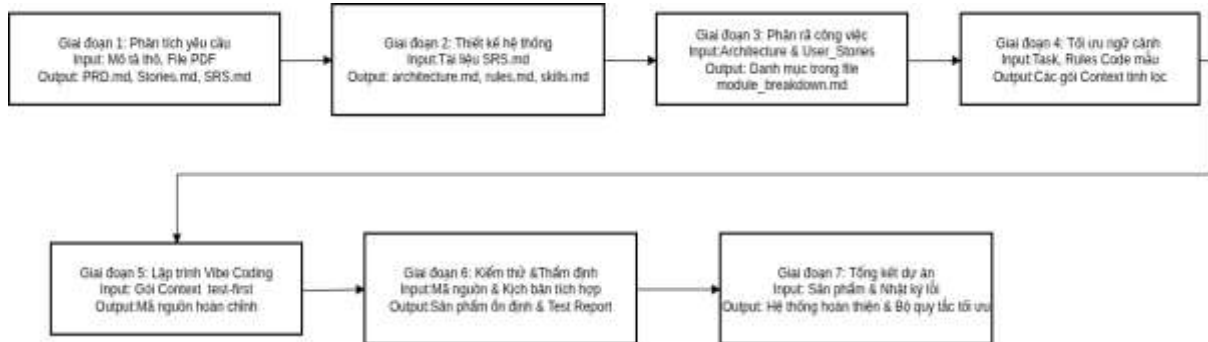
kể nhằm giúp lập trình viên giảm bớt các thao tác thủ công trong quá trình xây dựng hệ thống, đồng thời hỗ trợ phối hợp giữa nhiều công cụ phát triển khác nhau. Trong dự án VNeNUTRITION, Antigravity được sử dụng như một công cụ hỗ trợ quản lý quy trình phát triển và tối ưu hóa việc làm việc với các mô hình AI. Việc áp dụng công nghệ này góp phần nâng cao hiệu quả triển khai dự án, giảm thời gian xử lý công việc và cải thiện năng suất phát triển phần mềm.



Hình 2.7. Giao diện của Openrouter

Chương 3: Phương pháp thực hiện

Chương này trình bày các phương pháp và quy trình được sử dụng để xây dựng hệ thống App Nutrition. Nội dung chương tập trung vào việc phân tích kiến trúc hệ thống, thiết kế cơ sở dữ liệu, lựa chọn công nghệ phù hợp và mô tả quá trình triển khai các chức năng chính của ứng dụng.

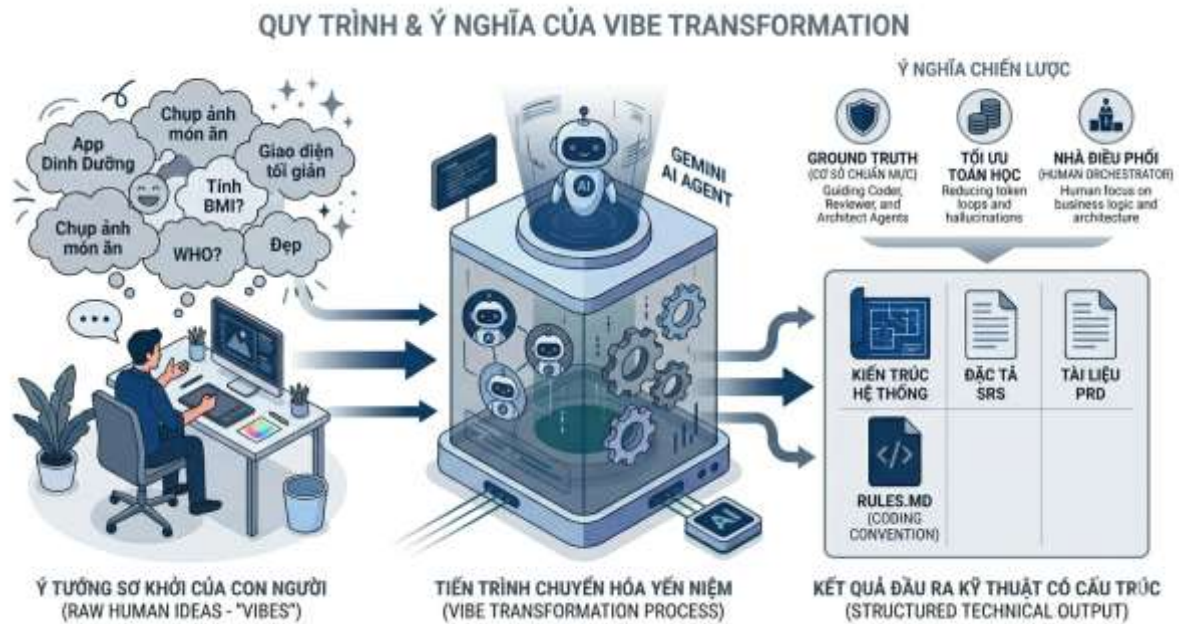


Hình 3.1. Hình ảnh mô tả phương pháp Vibe Engineering của nhóm

3.1 GIAI ĐOẠN 1: PHÂN TÍCH YÊU CẦU VỚI AI

3.1.1. Thực hiện phân tích yêu cầu với AI

Trong phương pháp luận Vibe Engineering, "Vibe Transformation" (Chuyển hóa Ý niệm) là tiến trình hệ thống hóa nhằm dịch thuật các ý tưởng mang tính định tính, trực giác ban đầu của người kỹ sư thành một hệ thống tài liệu kỹ thuật có cấu trúc logic chặt chẽ. Mục tiêu tối thượng của tiến trình này là thiết lập một "mỏ neo ngữ cảnh" vững chắc cho hệ thống, nhằm triệt tiêu hiện tượng lệch hướng ngữ cảnh (Context Drift) và hạn chế tối đa sự ảo tưởng tri thức (Hallucination) của các mô hình ngôn ngữ lớn (LLM) trong các giai đoạn phát triển kế tiếp. Việc chuẩn hóa ý niệm ngay từ khởi đầu không chỉ đơn thuần là hoạt động viết tài liệu truyền thống, mà là quá trình định hình biên giới tri thức và thiết lập các rào cản kỹ thuật để kiểm soát hành vi của hệ thống đa tác tử.



Hình 3.2. Hình ảnh minh họa quá trình phân tích yêu cầu với AI

3.1.2 Quy trình Brainstorming Động và Vai trò BA của AI Agent

Trong khuôn khổ mô hình Vibe Engineering, quy trình xác định và đặc tả yêu cầu nghiệp vụ được vận hành thông qua một phiên thảo luận động giữa Người điều phối (Orchestrator) và Tác tử AI đóng vai trò Chuyên viên Phân tích Nghiệp vụ (BA Agent). Khi tiếp nhận các yêu cầu cấu hình đầu vào, BA Agent không xử lý thông tin một cách thụ động; thay vào đó, hệ thống kích hoạt cơ chế phản biện ngược chủ động (Active Counter-Argument Mechanism).

Thay vì ghi nhận trực tiếp các yêu cầu ban đầu, tác tử tiến hành bóc tách chuyên sâu các kịch bản sử dụng (Use Cases), nhận diện các lỗ hổng logic tiềm ẩn và thiết lập chuỗi chất vấn phân tích về các trường hợp biên (Edge Cases). Minh chứng cụ thể trong phân hệ quản lý thể trọng, BA Agent sẽ tự động đặt ra các bài toán chất vấn về tiêu chuẩn phân loại chỉ số khối cơ thể (áp dụng chuẩn WHO hay IDI & WPRO), kịch bản xử lý hệ thống khi xảy ra tình trạng khuyết thiếu dữ liệu (Missing Data Execution), cũng như phương thức đồng bộ hóa và chuẩn hóa các đơn vị đo lường khác nhau. Quy trình này đảm bảo tính toàn vẹn của logic nghiệp vụ trước khi chuyển giao sang giai đoạn thiết kế kiến trúc và phát triển mã nguồn.

Mô hình được chúng tôi sử dụng là Gemini Pro 3.1 được cung cấp miễn phí trong IDE AI Coding Antigravity của Goolge. Sở dĩ chúng tôi chọn mô hình này là vì AI Agent được cho miễn phí rất nhiều quota (tức lượt sử dụng) nên rất phù hợp với việc phải làm việc với nhiều đoạn chat lâu dài như thế này.

Hãy đóng vai trò Business Analyst cấp cao, đọc tệp "Yêu cầu về hệ thống NutritionApp.pdf" và thực hiện phản biện ngược (Reverse Interrogation) theo phương pháp Vibe Engineering. Đầu tiên, hãy phân tích chỉ ra các lỗ hổng logic, điểm mập mờ và trường hợp biên chưa được xử lý. Đặt ra chuỗi câu hỏi chuyên sâu để tôi làm rõ ngữ cảnh, tránh hiện tượng lệch hướng. Hãy đưa ra danh sách câu hỏi này trước. Sau khi tôi phản hồi, bạn sẽ chuẩn hóa danh mục User Stories kèm Tiêu chí chấp nhận và xuất tài liệu SRS.md/PRD.md ở bước sau

Hình 3.3. Prompts sử dụng trong đề yêu cầu AI thực hiện phân tích

Kết quả mong muốn nhận được sau quá trình này sẽ là một files Yêu cầu về App Nutrition hoàn chỉnh.

3.1.3. Khởi tạo PRD.md, SRS.md và User Stories

Trong mô hình Vibe Coding, hệ thống tài liệu PRD.md, SRS.md và các câu chuyện người dùng (User Stories) đóng vai trò là nguồn dữ liệu đầu vào tối cao để cấu trúc hóa bộ nhớ đệm ngữ cảnh (Context Window) của các LLM. Nhằm tối ưu hóa khả năng đọc hiểu của AI Agent, tài liệu được tiêu chuẩn hóa nghiêm ngặt qua các phân mục: Phạm vi Dự án (xác định In-Scope/Out-of-Scope) và Yêu cầu Chức năng (phân rã hệ thống thành các mô-đun nguyên tử như BMI, AI Vision). Các yêu cầu này được cụ thể hóa thành hệ thống User Stories chi tiết, giúp định hình hành vi người dùng và thiết lập tiêu chí nghiệm thu rõ ràng cho các Coder Agent triển khai mã nguồn.

Ở bước này công cụ thực hiện vẫn là mô hình Gemini Pro 3.1 từ Google.

Bạn sẽ đóng vai trò là một BA chuyên nghiệp

Dựa vào những gì được mô tả về yêu cầu của hệ thống App Nutrition được để ở file "Yêu cầu về App Nutrition.pdf" hãy viết ra cho tôi một file PRD.md với các mục là: – Overview: Mô tả ngắn về dự án.

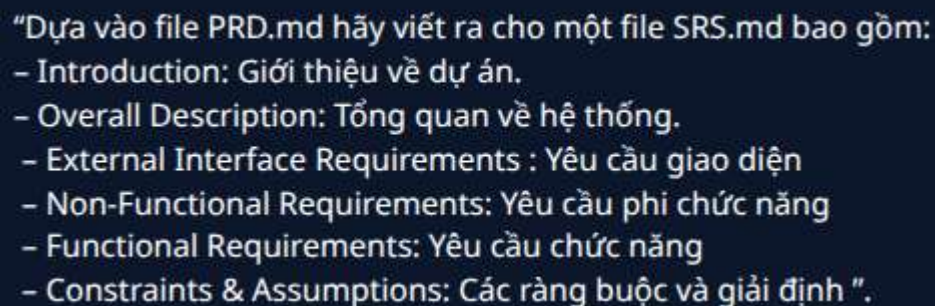
- Core Features: Những chức năng cốt lõi.
- Business Context: Mô tả nghiệp vụ
- Success Metrics: Điều kiện chấp nhận
- System Users: Người dùng hệ thống
- Development Roadmap: Chiến lược phát triển

(Hình 3.3. Prompts được sử dụng để tạo ra PRD.md)

Sau khi đã hoàn thành file PRD.md chúng tôi sẽ đánh giá nó dựa trên tiêu chí:

- Tính đầy đủ (Completeness): Đảm bảo có đủ 6 mục cốt lõi theo yêu cầu của prompt và chuẩn định dạng Markdown (.md).
- Độ rõ ràng của tính năng (Clarity): Các chức năng cốt lõi (tính BMI, tracking) phải chi tiết, dễ hiểu đối với lập trình viên.
- Chính xác nghiệp vụ (Business Alignment): Phản ánh đúng nghiệp vụ quản lý dinh dưỡng (công thức tính TDEE, macro, luồng dữ liệu).
- Tiêu chí đo lường được (Measurability): Các chỉ số thành công và điều kiện chấp nhận (Acceptance Criteria) phải rõ ràng, kiểm thử được.
- Phân tách đối tượng (User Personas): Xác định rõ các nhóm người dùng hệ thống và phân quyền tương ứng.
- Lộ trình khả thi (Roadmap): Chia giai đoạn phát triển hợp lý (từ MVP đến hoàn thiện) theo mô hình Agile.

Sau khi đã có được PRD.md ta tiến hành tiếp đến SRS.md



"Dựa vào file PRD.md hãy viết ra cho một file SRS.md bao gồm:

- Introduction: Giới thiệu về dự án.
- Overall Description: Tổng quan về hệ thống.
- External Interface Requirements : Yêu cầu giao diện
- Non-Functional Requirements: Yêu cầu phi chức năng
- Functional Requirements: Yêu cầu chức năng
- Constraints & Assumptions: Các ràng buộc và giả định "

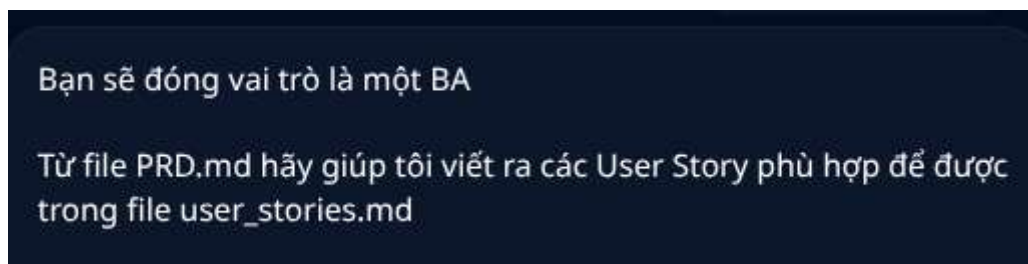
Hình 3.4. Prompts được sử dụng để tạo ra SRS.md

Sau khi đã hoàn thành file SRS.md chúng tôi sẽ đánh giá nó dựa trên tiêu chí:

- Tính đầy đủ (Completeness): Đảm bảo cấu trúc chuẩn hóa với đủ 6 mục cốt lõi theo yêu cầu của prompt dưới định dạng Markdown (.md).
- Độ chi tiết chức năng (Functional Specifications): Các yêu cầu chức năng (như đăng nhập, quét món ăn bằng AI, lưu lịch sử) phải được mô tả cụ thể dưới dạng User Story hoặc Use Case chi tiết.
- Chuẩn hóa giao diện (External Interfaces): Định nghĩa rõ ràng các yêu cầu về giao diện người dùng (UI), giao diện phần cứng (Camera để quét món ăn), và các API tích hợp bên thứ ba.

- Chỉ số phi chức năng rõ ràng (Non-Functional Requirements): Các tiêu chuẩn về hiệu năng (tốc độ phản hồi AI), bảo mật (mã hóa dữ liệu người dùng) và độ ổn định phải có con số định lượng cụ thể.
- Tính nhất quán (Consistency): Nội dung đặc tả kỹ thuật trong SRS phải đồng nhất hoàn toàn với các định hướng và nghiệp vụ đã đặt ra ở file PRD trước đó.
- Tính khả thi và Ràng buộc (Constraints & Assumptions): Xác định chính xác các giới hạn về công nghệ (ví dụ: giới hạn thiết bị, môi trường mạng) và các giả định phát triển một cách thực tế.
- Mức độ sẵn sàng cho Phát triển (Development Readiness): Tài liệu phải đủ chặt chẽ và tường minh để làm ngữ cảnh đầu vào (Context) cho AI sinh mã nguồn chính xác.

Ở bước cuối cùng ta sẽ sinh ra User Stories từ PRD.md



Hình 3.5. Prompts được sử dụng để tạo ra user_stories.md

Sau khi đã hoàn thành file user_stories.md chúng tôi sẽ đánh giá nó dựa trên tiêu chí:

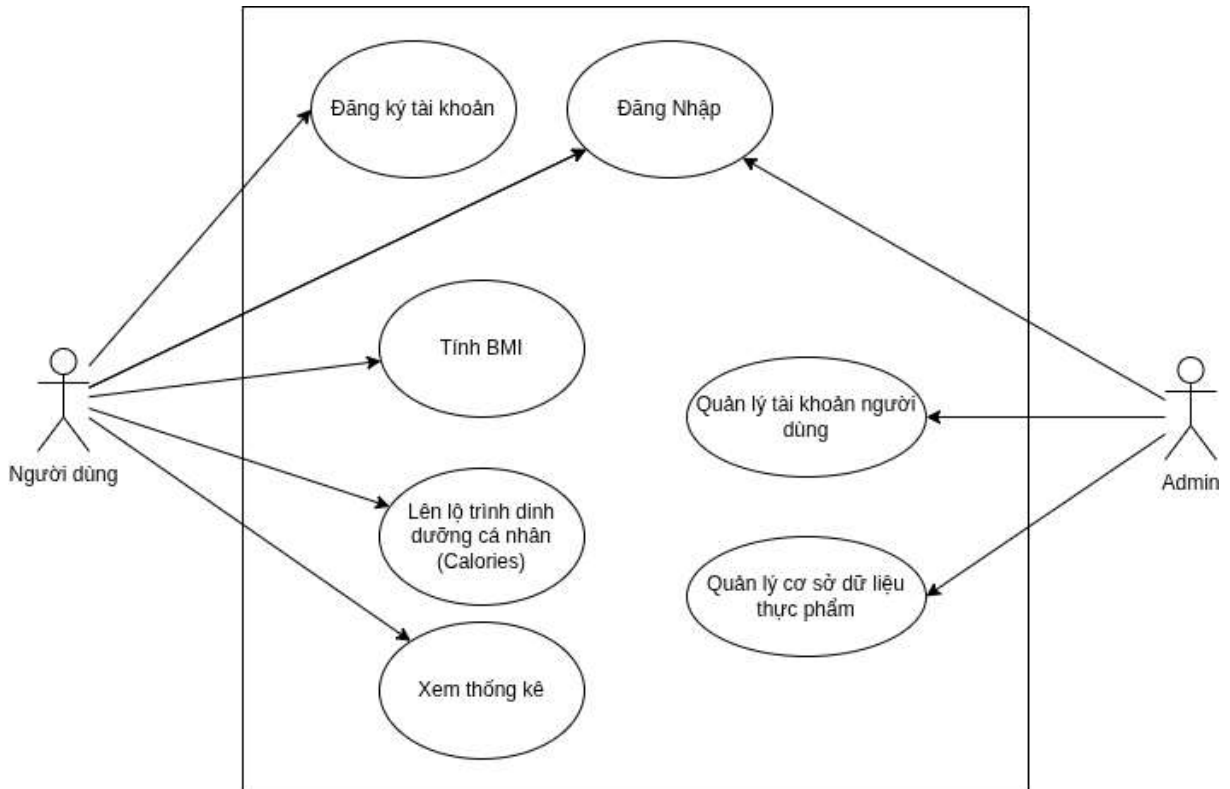
- Đúng chuẩn định dạng: Tuân thủ cấu trúc "Là một... tôi muốn... để...".
- Điều kiện chấp nhận rõ ràng: Có đầy đủ tiêu chí nghiệm thu (Acceptance Criteria) cho từng story.
- Độc lập và phân rã tốt: Các story đủ nhỏ, có giá trị độc lập để dễ dàng đưa vào từng Sprint.
- Bao phủ tính năng: Chuyển hóa đầy đủ, không bỏ sót bất kỳ chức năng nào từ PRD.md.
- Đúng vai trò người dùng: Phân định chính xác hành động của từng nhóm đối tượng (User, Admin).
- Sẵn sàng triển khai: Đủ rõ ràng để AI hoặc lập trình viên chuyển thành các task code cụ thể.

Kết quả chúng tôi mong muốn đạt được sau quá trình này là các file PRD.md, SRS.md và User Story hoàn chỉnh, chuẩn hóa, làm tài liệu đầu vào tối cao để AI Agent đọc hiểu và triển khai mã nguồn chính xác.

3.1.3 Use Case, Business Process

Trong giai đoạn phân tích nghiệp vụ và thiết kế sơ đồ, công cụ AI đóng vai trò như một cộng sự đắc lực, giúp tối ưu hóa quy trình. Tuy nhiên, AI không thể thay thế con người trong việc thiết kế và mô tả hệ thống một cách hoàn chỉnh. Do đó ở phần Use Case và phân tích Bussiness Process này sẽ do chúng tôi hoàn toàn đảm nhận

Use Case



Hình 3.6. Sơ đồ Use Case

Sơ đồ Use Case này mô tả các chức năng cốt lõi của một hệ thống quản lý sức khỏe và dinh dưỡng cá nhân, bao gồm hai tác nhân (Actors) chính tương tác với hệ thống:

1. Tác nhân "Người dùng" (User): Là đối tượng sử dụng hệ thống để theo dõi sức khỏe cá nhân. Người dùng có thể thực hiện các chức năng:

- Đăng ký tài khoản và Đăng nhập để sử dụng hệ thống.
- Tính BMI: Nhập chỉ số cơ thể để kiểm tra tình trạng sức khỏe.
- Lên lộ trình dinh dưỡng cá nhân (Calories): Nhận gợi ý hoặc tự thiết lập chế độ ăn uống phù hợp.
- Xem thống kê: Theo dõi tiến trình và lịch sử dinh dưỡng qua biểu đồ/báo cáo.

2. Tác nhân "Admin" (Quản trị viên): Là đối tượng vận hành và quản lý hệ thống. Admin có quyền thực hiện các chức năng:

- Đăng nhập vào hệ thống quản trị.
- Quản lý tài khoản người dùng: Khóa, mở hoặc xử lý thông tin tài khoản của người dùng.
- Quản lý cơ sở dữ liệu thực phẩm: Thêm, sửa thông tin về các loại món ăn, hàm lượng dinh dưỡng và calories trong hệ thống.

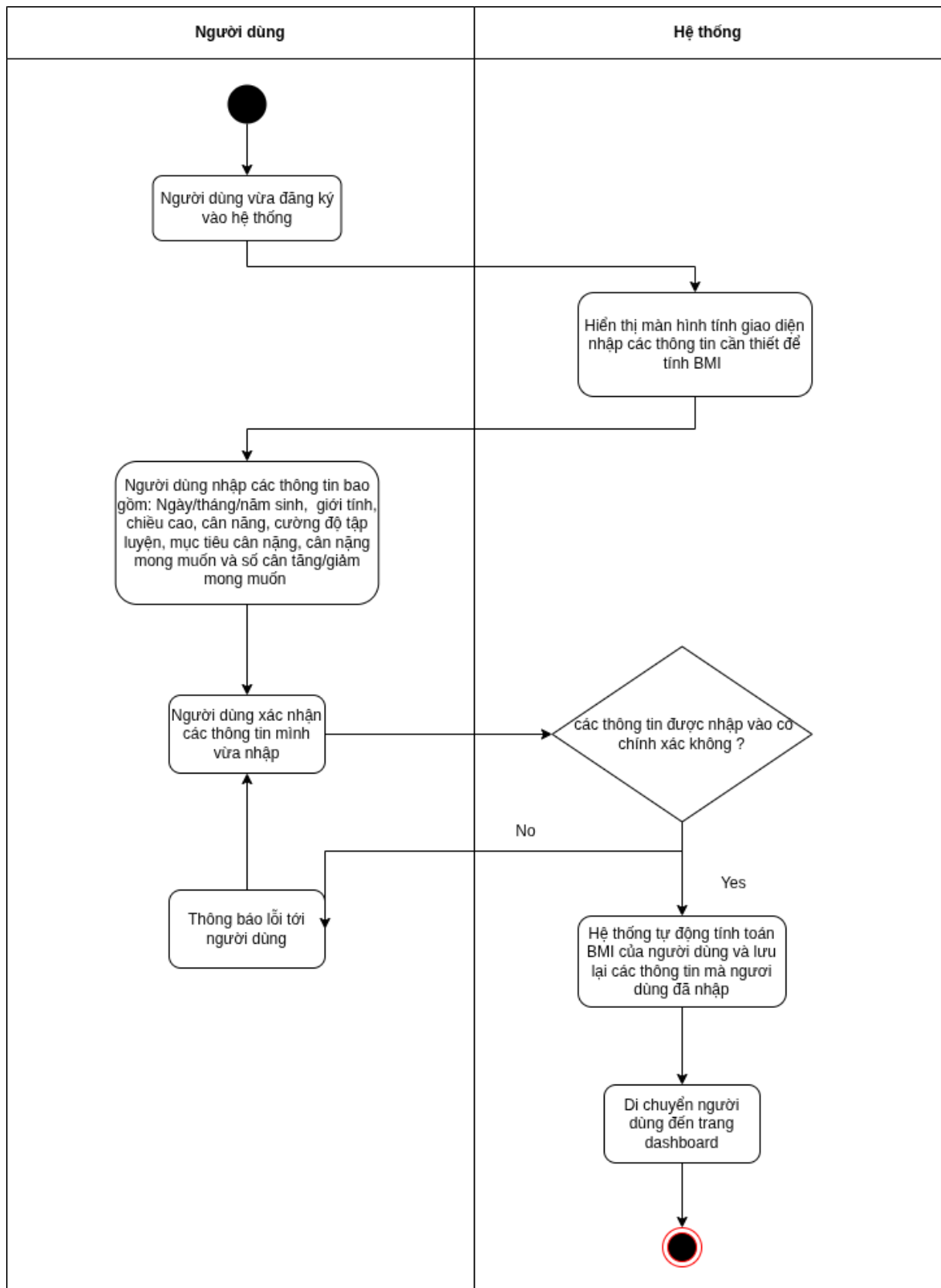
Business Process

Một số Business Process quan trọng của dự án sẽ được trình bày

Mô tả Business Process của chức năng tính BMI

Sau khi người dùng vừa đăng ký vào hệ thống thành công, màn hình sẽ hiển thị giao diện nhập các thông tin cần thiết để tính BMI. Tại đây, người dùng tiến hành nhập thông tin cá nhân bao gồm: ngày sinh, giới tính, chiều cao, cân nặng, cường độ tập luyện, mục tiêu cân nặng và số cân muốn tăng hoặc giảm, rồi nhấn xác nhận.

Hệ thống sẽ kiểm tra xem các thông tin được nhập vào có chính xác hay không. Nếu thông tin không hợp lệ, hệ thống sẽ gửi thông báo lỗi tới người dùng để yêu cầu điều chỉnh. Ngược lại, nếu thông tin hoàn toàn chính xác, hệ thống sẽ tự động tính toán chỉ số BMI, lưu lại toàn bộ dữ liệu đã nhập, và cuối cùng điều hướng người dùng thẳng đến trang dashboard để kết thúc quy trình.



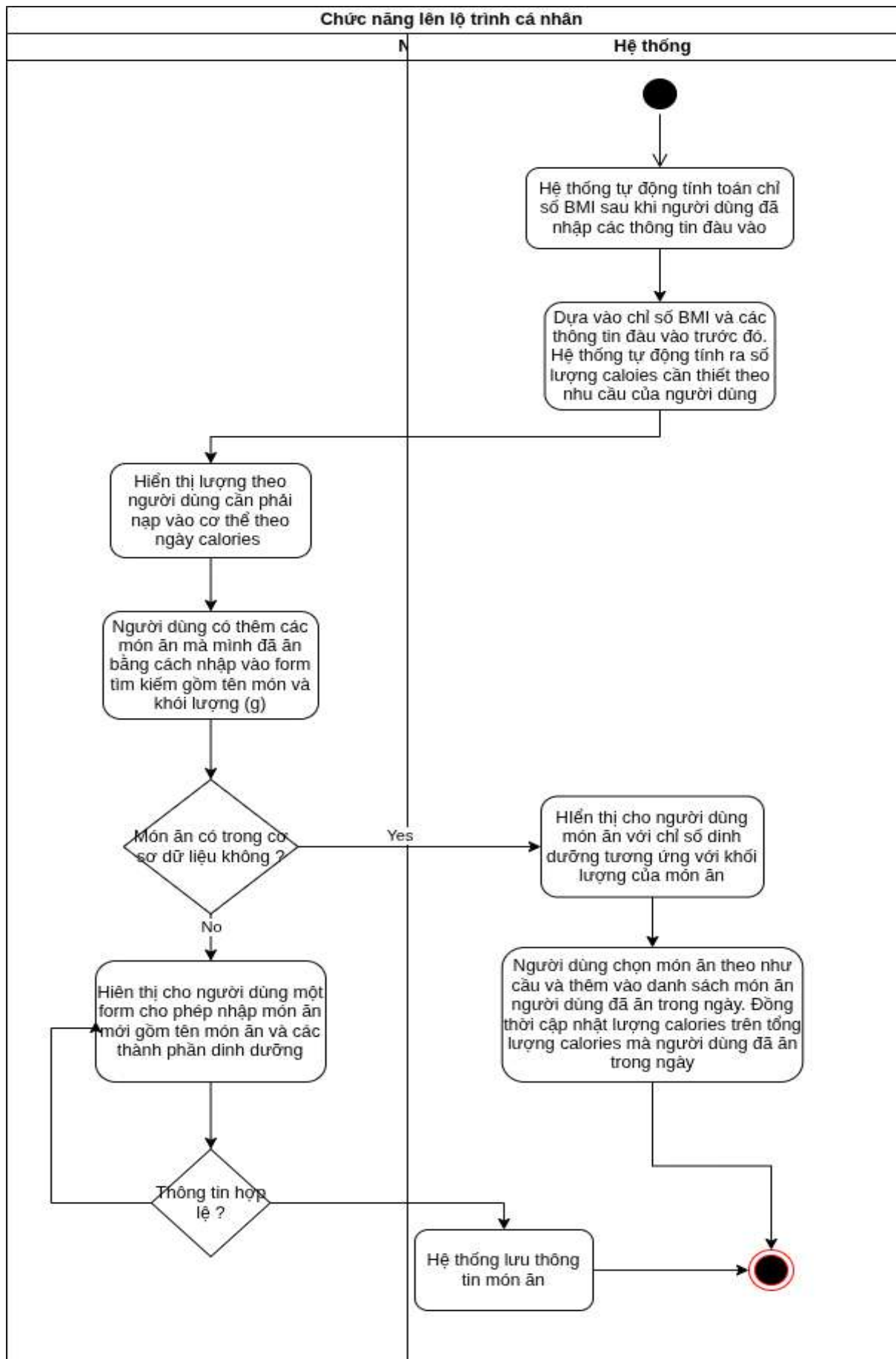
Hình 3.7. Sơ đồ Business Process của chức năng tính BMI

Mô tả Business Process của chức năng lên lộ trình dinh dưỡng cá nhân

Quy trình bắt đầu bằng việc hệ thống tự động tính toán chỉ số BMI dựa trên các thông tin đầu vào mà người dùng đã cung cấp. Từ chỉ số BMI và các dữ liệu này, hệ thống tiếp tục tự động tính ra số lượng calories cần thiết theo nhu cầu riêng của người dùng, sau đó hiển thị lượng calories tiêu chuẩn mà họ cần nạp vào cơ thể mỗi ngày.

Tại giao diện theo dõi, người dùng có thể thêm các món ăn mình đã ăn bằng cách nhập tên món ăn và khối lượng (gram) vào form tìm kiếm. Hệ thống sẽ tiến hành kiểm tra xem món ăn đó đã có sẵn trong cơ sở dữ liệu hay chưa:

- Nếu món ăn ĐÃ CÓ trong cơ sở dữ liệu (Yes): Hệ thống hiển thị thông tin món ăn kèm theo các chỉ số dinh dưỡng tương ứng với khối lượng đã nhập. Người dùng chọn món và thêm vào danh sách các món đã ăn trong ngày; hệ thống đồng thời cập nhật tổng lượng calories mà người dùng đã tiêu thụ.
- Nếu món ăn CHƯA CÓ trong cơ sở dữ liệu (No): Hệ thống sẽ hiển thị một form mới cho phép người dùng tự nhập tên món ăn cùng các thành phần dinh dưỡng của món ăn đó để bổ sung vào hệ thống.



Hình 3.8. Sơ đồ Business Process của chức năng lên kế hoạch dinh dưỡng

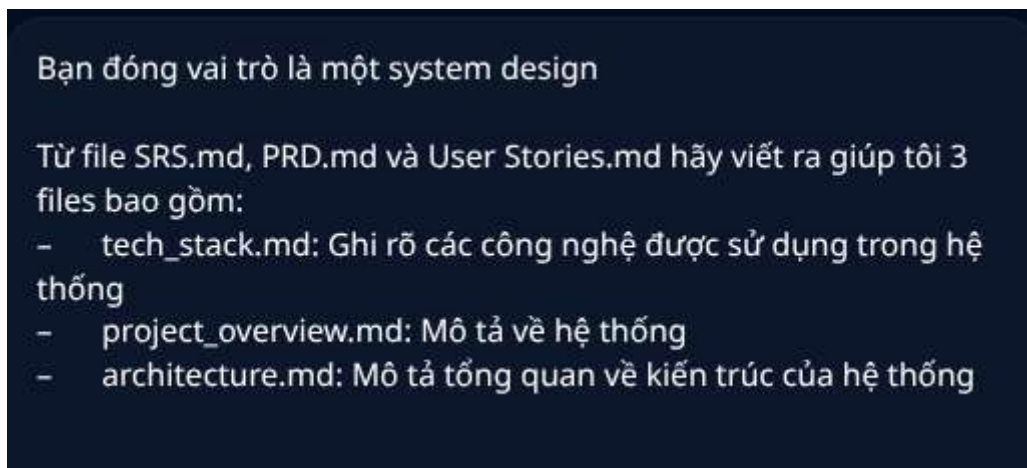
3.2 GIAI ĐOẠN 2: THIẾT KẾ KIẾN TRÚC HỆ THỐNG VÀ HẠ TẦNG KỸ THUẬT (ARCHITECTURE & BLUEPRINTING)

3.2.1 Thiết kế Kiến trúc

Kiến trúc hệ thống của NutritionApp được xây dựng theo mô hình phân cấp độc lập, kết hợp linh hoạt các công nghệ để tối ưu hóa hiệu năng cho từng tác vụ:

- Tầng Quản trị Trung tâm (Core Backend - Java/Spring Boot): Đóng vai trò là hạt nhân điều phối toàn bộ hệ thống. Tầng này chịu trách nhiệm quản lý tài khoản người dùng, bảo mật, lưu trữ dữ liệu và xử lý các phép tính toán sinh trắc học (như chỉ số BMI). Spring Boot được lựa chọn nhờ tính ổn định cao, bảo mật tốt và khả năng xử lý mượt mà ở quy mô lớn.
- Tầng Giao diện Người dùng (Client Application Layer - React Native): Sử dụng framework React Native để phát triển ứng dụng di động đa nền tảng từ một kho mã nguồn duy nhất. Tầng này tập trung vào việc tối ưu hóa trải nghiệm người dùng, tiếp nhận thông tin đầu vào và tương tác trực tiếp với dịch vụ backend thông qua giao thức RESTful API.

Tiếp tục dựa vào mô hình Gemini 3.1 Pro



Bạn đóng vai trò là một system design

Từ file SRS.md, PRD.md và User Stories.md hãy viết ra giúp tôi 3 files bao gồm:

- tech_stack.md: Ghi rõ các công nghệ được sử dụng trong hệ thống
- project_overview.md: Mô tả về hệ thống
- architecture.md: Mô tả tổng quan về kiến trúc của hệ thống

Hình 3.9. Prompts được sử dụng để tạo ra các files thiết kế và mô tả hệ thống

Sau khi hoàn thành bộ ba tài liệu thiết kế hệ thống, chúng tôi tiến hành đánh giá từng file dựa trên các tiêu chí sau:

Đối với file tech_stack.md:

- Tính phù hợp của công nghệ: Các công nghệ được chọn (Spring Boot, PostgreSQL, React, React-Native) phải đáp ứng tốt yêu cầu xử lý dữ liệu của hệ thống
- Độ chi tiết và phân tầng: Ghi rõ vai trò, phiên bản và lý do sử dụng của từng công nghệ trong các tầng hệ thống (Frontend, Backend, Database).

Đối với file `project_overview.md`:

- Bức tranh tổng quan rõ ràng: Cung cấp cái nhìn toàn diện, ngắn gọn nhưng đầy đủ về mục tiêu, phạm vi dự án và giá trị cốt lõi mà ứng dụng dinh dưỡng mang lại.
- Xác định đúng bài toán: Mô tả chính xác các vấn đề hiện tại của người dùng và cách hệ thống giải quyết các vấn đề đó.

Đối với file `architecture.md`:

- Kiến trúc hệ thống chuẩn xác: Mô tả rõ ràng mô hình kiến trúc Modular Monolithics
- Mức độ sẵn sàng cho AI code: Sơ đồ hoặc mô tả luồng dữ liệu phải đủ tường minh để các AI Agent (như Claude Code, GitHub Copilot) hiểu được cấu trúc và sinh mã nguồn đúng kiến trúc.

3.2.2 Thiết lập Bộ Quy tắc Lập trình Toàn cục Khắt khe (`rules.md`)

Trong quá trình phát triển hệ thống quản lý chế độ dinh dưỡng và gợi ý bữa ăn, việc xây dựng một bộ quy tắc lập trình toàn cục đóng vai trò quan trọng nhằm đảm bảo tính nhất quán, khả năng mở rộng và chất lượng mã nguồn giữa các thành viên trong nhóm phát triển. Do hệ thống được xây dựng theo mô hình đa nền tảng với nhiều thành phần khác nhau như ứng dụng React-Native, hệ thống quản trị React, backend Spring Boot, việc thiếu các quy chuẩn thống nhất có thể dẫn đến tình trạng mã nguồn không đồng nhất, khó bảo trì và phát sinh nhiều lỗi trong quá trình tích hợp.

Vì lý do đó, đề tài tiến hành xây dựng tập tin `rules.md` như một tài liệu quy chuẩn kỹ thuật toàn cục cho toàn bộ dự án. Tập tin này quy định các nguyên tắc bắt buộc mà tất cả thành viên và các công cụ AI hỗ trợ lập trình phải tuân thủ trong suốt quá trình phát triển hệ thống.

Nội dung của bộ quy tắc tập trung vào các nhóm tiêu chuẩn chính bao gồm: quy tắc tổ chức mã nguồn, quy tắc đặt tên, quy tắc xử lý lỗi, quy tắc quản lý dữ liệu, quy tắc bảo mật và quy trình commit mã nguồn.

Chúng tôi đã sử dụng prompts để thực hiện nên files `rules.md` như sau:

Bạn sẽ đóng vai trò là một nhà phát triển phần mềm

Hãy dựa trên file PRD.md, SRS.md, architecture.md, tech_stack và project_overview.md để có thể viết nên một file rules.md gồm các phần sau:

- Quy tắc tổ chức mã nguồn (Codebase Structure Rules)
- Quy tắc đặt tên (Naming Conventions)
- Quy tắc xử lý lỗi và ngoại lệ (Exception Handling Rules)
- Quy tắc quản lý dữ liệu PostgreSQL
- Quy tắc bảo mật
- Quy tắc commit và quản lý phiên bản

Hình 3.10. Prompts được sử dụng để tạo ra các files thiết kế và mô tả hệ thống
Sau khi nhận được kết quả từ AI, chúng tôi thực hiện đánh giá file rules.md dựa vào các tiêu chí sau:

Quy tắc tổ chức mã nguồn (Codebase Structure Rules)

- Kiến trúc tổng thể: Quy định nguyên tắc mô-đun hóa (Modularization) và phân tách trách nhiệm (Separation of Concerns).
- Backend (Spring Boot): Quy định cấu trúc thư mục nghiêm ngặt theo các tầng: controller, service, repository, model, dto, config, security. Định nghĩa rõ nhiệm vụ của từng tầng (Ví dụ: Controller tuyệt đối không chứa business logic).
- Frontend/Mobile: Quy định cấu trúc thư mục phân tách theo tính năng (features) và lõi hệ thống (core), quản lý các cấu phần dùng chung (widgets/components), dịch vụ gọi API (services), và thực thể dữ liệu (models).

b. Quy tắc đặt tên (Naming Conventions)

- Mã nguồn: Quy định chi tiết các chuẩn đặt tên cho: Biến và hàm/phương thức (camelCase), Lớp/Giao diện (PascalCase), Hằng số (UPPER_CASE).
- Cơ sở dữ liệu (PostgreSQL): Quy định chuẩn đặt tên bảng (dạng số nhiều, snake_case), tên cột (viết thường, snake_case), khóa chính, khóa ngoại.
- Giao tiếp: Quy định chuẩn thiết kế đường dẫn API theo kiến trúc RESTful (ví dụ: viết thường, phân tách bằng dấu gạch ngang, sử dụng danh từ số nhiều /api/v1/meal-plans).

- Yêu cầu: Tên biến/hàm phải tường minh theo nghiệp vụ dinh dưỡng (ví dụ: lượng calo, chỉ số BMI), nghiêm cấm viết tắt mơ hồ.
- c. Quy tắc xử lý lỗi và ngoại lệ (Exception Handling Rules)
- Tập trung hóa: Quy định tất cả các ngoại lệ phải được bắt (catch) và xử lý tập trung, tuyệt đối không để lộ thông tin log hệ thống hoặc làm sập ứng dụng.
 - Backend (Spring Boot): Sử dụng cơ chế Global Exception Handler. Phân loại lỗi rõ ràng (Validation, Authentication, Authorization, Database Error). Chuẩn hóa định dạng phản hồi lỗi trả về Client dưới dạng JSON (bao gồm: timestamp, status, message, path).
 - Frontend/Mobile: Bắt buộc sử dụng khối try-catch hoặc các hàm bắt lỗi ngoại vi khi thực hiện các tiến trình gọi API (API calls) để xử lý lỗi mạng hoặc lỗi từ server một cách mượt mà.
- d. Quy tắc quản lý dữ liệu và tương tác Cơ sở dữ liệu (PostgreSQL Rules)
- Tính toàn vẹn dữ liệu (ACID): Quy định các ràng buộc bắt buộc trong thiết kế bảng: Khóa chính (Primary Key), Khóa ngoại (Foreign Key), các ràng buộc dữ liệu (NOT NULL, UNIQUE, CHECK).
 - Chuẩn hóa: Thiết kế cơ sở dữ liệu tuân thủ các dạng chuẩn (1NF, 2NF, 3NF) để tối ưu hóa không gian lưu trữ và tránh dư thừa dữ liệu.
 - Thời gian: Quy định tất cả các trường dữ liệu thời gian phải sử dụng kiểu dữ liệu có múi giờ (timestamp with time zone) và đồng bộ theo chuẩn UTC.
 - Hiệu năng: Bắt buộc đánh chỉ mục (Index) cho các trường thường xuyên xuất hiện trong mệnh đề tìm kiếm, liên kết (JOIN) hoặc sắp xếp (ví dụ: user_id, food_id).
 - Ràng buộc liên kết: Định nghĩa rõ quy tắc xử lý khi xóa dữ liệu liên kết (ON DELETE CASCADE hoặc ON DELETE SET NULL).
- e. Quy tắc bảo mật hệ thống (Security Rules)
- Mã hóa: Quy định cơ chế băm mật khẩu (ví dụ: bắt buộc sử dụng thuật toán BCrypt) trước khi lưu vào PostgreSQL.
 - Xác thực & Phân quyền: Sử dụng cơ chế mã thông báo mã hóa (JWT) để quản lý phiên đăng nhập. Áp dụng phân quyền dựa trên vai trò (RBAC) nghiêm ngặt giữa người dùng (User) và quản trị viên (Admin).

- Quản lý bí mật: Nghiêm cấm lưu trữ thông tin nhạy cảm (API Keys, Database Credentials) trực tiếp trong mã nguồn. Bắt buộc quản lý qua biến môi trường (.env hoặc cấu hình hệ thống) và cấu hình .gitignore trước khi đẩy lên hệ thống quản lý phiên bản.

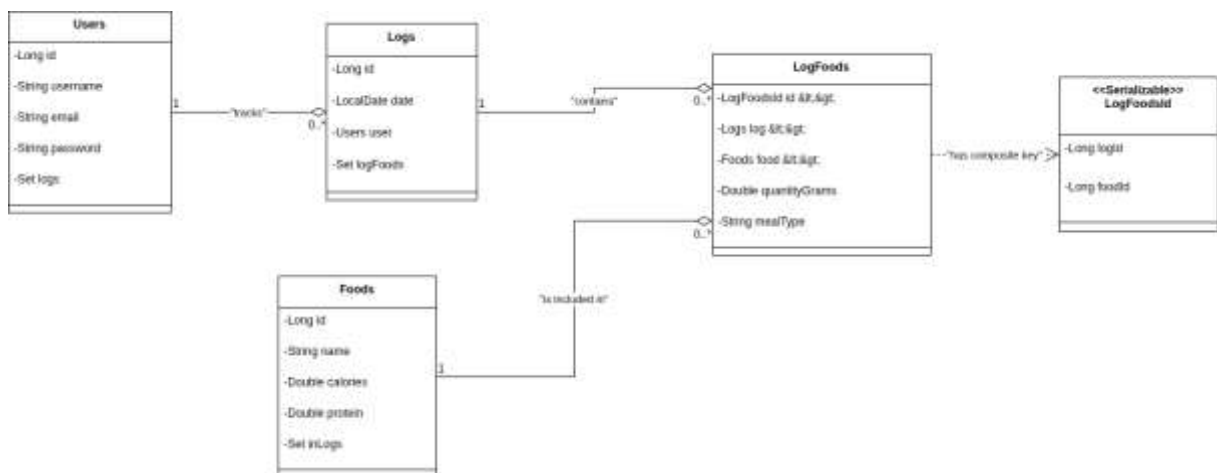
f. Quy tắc commit và quản lý phiên bản (Git & Version Control Rules)

- Chuẩn Commit: Áp dụng quy chuẩn Semantic Commits cho các thông điệp commit. Định dạng bắt buộc: type(scope): short description (Ví dụ: feat(auth): add JWT validation).
- Quản lý nhánh (Branching Strategy): Quy định sơ đồ phân nhánh rõ ràng bao gồm: main (Production), develop (Staging/Tích hợp), feature/* (Phát triển tính năng), bugfix/* (Sửa lỗi khẩn cấp).
- Quy trình kiểm duyệt: Định nghĩa quy trình tạo Pull Request và rà soát mã nguồn (Code Review) trước khi gộp nhánh để chuẩn bị cho quy trình tích hợp liên tục (CI/CD).

Tổng thể, bộ quy tắc lập trình toàn cục rules.md đóng vai trò như một tiêu chuẩn kỹ thuật thống nhất cho toàn bộ hệ thống. Việc áp dụng các quy tắc này giúp nâng cao chất lượng mã nguồn, giảm thiểu lỗi phát sinh, hỗ trợ khả năng mở rộng hệ thống và đảm bảo tính ổn định trong quá trình phát triển lâu dài của đề tài.

3.2.3. Phần con người thực hiện

Sơ đồ Lớp



Hình 3.11. Sơ đồ lớp của hệ thống

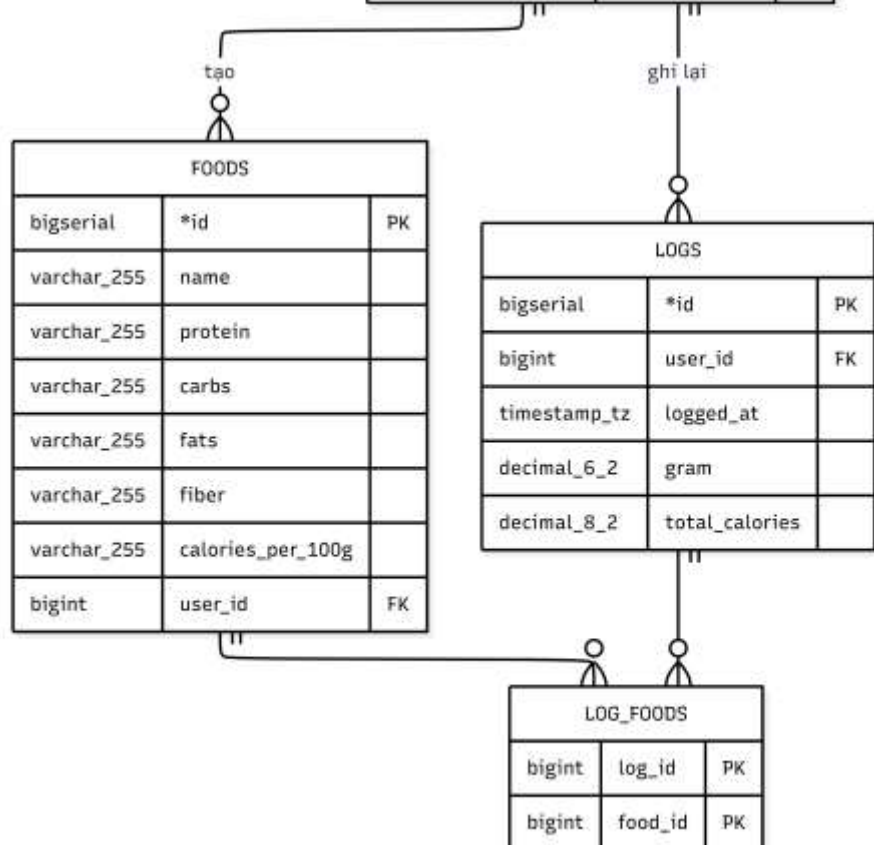
- Mô tả sơ đồ lớp
Users (Người dùng): Lưu trữ thông tin cơ bản gồm id, username, email và password. Một người dùng có mối quan hệ một-nhiều (1 - 0..*) với lớp Logs, nghĩa là họ có thể sở hữu nhiều nhật ký ghi chép theo thời gian.
- Logs (Nhật ký theo ngày): Đại diện cho nhật ký dinh dưỡng của một ngày cụ thể (date) thuộc về một người dùng duy nhất. Một nhật ký chứa một tập hợp nhiều bản ghi thực phẩm (0..*) thông qua thực thể trung gian LogFoods.
- Foods (Thực phẩm): Danh mục lưu trữ thông tin món ăn bao gồm id, name, lượng calories và protein. Một món ăn có thể xuất hiện trong nhiều bản ghi nhật ký khác nhau.
- LogFoods & LogFoodsId (Chi tiết nhật ký thực phẩm): Đây là lớp trung gian giải quyết mối quan hệ nhiều-nhiều giữa Logs và Foods. Lớp này lưu trữ thông tin chi tiết về bữa ăn gồm lượng thực phẩm tiêu thụ (quantityGrams) và loại bữa ăn (mealType như sáng, trưa, tối). Nó sử dụng khóa định danh tổng hợp LogFoodsId (gồm logId và foodId) được đánh dấu Serializable.

Sơ đồ ERD

BMI_STATUS_ENUM	
string	UNDERWEIGHT
string	NORMAL
string	OVERWEIGHT
string	OBESITY_LEVEL_1
string	OBESITY_LEVEL_2

ACTIVITY_LEVEL_ENUM	
string	SEDENTARY
string	LIGHT_ACTIVE
string	ACTIVE
string	VERY_ACTIVE

USERS		
bigserial	*id	PK
varchar_255	name	
date	dob	
varchar_100	username	UK
varchar_255	hash_password	
varchar_255	current_weight	
varchar_255	target_weight	
varchar_255	height	
varchar_20	gender	
integer	goal_calories	
activity_level_enum	activity_level	
varchar_255	bmi	
bmi_status_enum	bmi_status	



Hình 3.12. Sơ đồ ERD

Mô tả về sơ đồ ERD

Bảng USERS

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	bigserial	PK	Mã định danh duy nhất của người dùng (Tự động tăng)
name	varchar(255)		Tên đầy đủ của người dùng
dob	date		Ngày tháng năm sinh
username	varchar(100)	UK	Tên đăng nhập (Duy nhất)
hash_password	varchar(255)		Mật khẩu đã được mã hóa
current_weight	varchar(255)		Cân nặng hiện tại
target_weight	varchar(255)		Cân nặng mục tiêu
height	varchar(255)		Chiều cao
gender	varchar(20)		Giới tính
goal_calories	integer		Lượng calo mục tiêu cần nạp hàng ngày
activity_level	activity_level_enum	FK	Mức độ hoạt động (Tham chiếu đến ENUM)
bmi	varchar(255)		Chỉ số khối cơ thể (BMI)
bmi_status	bmi_status_enum	FK	Tình trạng BMI (Tham chiếu đến ENUM)

Bảng 3.1. Mô tả dữ liệu của bảng USERS

Bảng FOODS

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	bigserial	PK	Mã định danh duy nhất của món ăn (Tự động tăng)
name	varchar(255)		Tên món ăn / thực phẩm
protein	varchar(255)		Hàm lượng Protein (đạm)

carbs	varchar(255)		Hàm lượng Carbohydrate (tinh bột)
fats	varchar(255)		Hàm lượng Fats (chất béo)
fiber	varchar(255)		Hàm lượng Fiber (chất xơ)
calories_per_100g	varchar(255)		Lượng calo tính trên 100g thực phẩm
user_id	bigint	FK	Mã người dùng tạo ra món ăn này (Liên kết tới USERS.id)

Bảng 3.2. Mô tả dữ liệu của bảng FOODS

Bảng LOGS

Tên trường (Field Name)	Kiểu dữ liệu (Data Type)	Ràng buộc (Constraint)	Mô tả (Description)
id	bigserial	PK	Mã định danh duy nhất của nhật ký (Tự động tăng)
user_id	bigint	FK	Mã người dùng sở hữu nhật ký này (Liên kết tới USERS.id)
logged_at	timestamp_tz		Thời gian ghi nhật ký (có kèm múi giờ)
gram	decimal(6,2)		Khối lượng thực phẩm tiêu thụ (tính bằng gram)
total_calories	decimal(8,2)		Tổng lượng calo tiêu thụ trong lần ghi nhận này

Bảng 3.3. Mô tả dữ liệu của bảng LOGS

Bảng LOGS_FOOD

Tên trường (Field Name)	Kiểu dữ liệu (Data Type)	Ràng buộc (Constraint)	Mô tả (Description)
log_id	bigint	PK, FK	Mã nhật ký (Liên kết tới LOGS.id)
food_id	bigint	PK, FK	Mã món ăn (Liên kết tới FOODS.id)

Bảng 3.4. Mô tả dữ liệu của bảng LOGS_FOOD

Bảng BMI_STATUS_ENUM

Giá trị	Kiểu dữ liệu	Mô tả
UNDERWEIGHT	string	Thiếu cân
NORMAL	string	Bình thường
OVERWEIGHT	string	Thừa cân
OBESITY_LEVEL_1	string	Béo phì cấp độ 1
OBESITY_LEVEL_2	string	Béo phì cấp độ 2

Bảng 3.5. Mô tả dữ liệu của bảng BMI_STATUS_ENUM

Bảng ACTIVITY_LEVEL_ENUM

Giá trị (	Kiểu dữ liệu	Mô tả (
SEDENTARY	string	Ít vận động (văn phòng, ngồi nhiều)
LIGHT_ACTIVE	string	Vận động nhẹ (tập thể dục 1-3 ngày/tuần)
ACTIVE	string	Vận động nhiều (chơi thể thao 3-5 ngày/tuần)
VERY_ACTIVE	string	Vận động rất nhiều (vận động viên, lao động nặng)

Bảng 3.6. Mô tả dữ liệu của bảng BMI_STATUS_ENUM

3.3 GIAI ĐOẠN 3: PHÂN RÃ CÔNG VIỆC THÀNH CẤU TRÚC THỰC THI NGUYÊN TỬ

3.3.1 Nguyên lý Giới hạn Ngữ cảnh

Trong quá trình nghiên cứu và phát triển hệ thống quản lý chế độ dinh dưỡng và gợi ý bữa ăn, đề tài không chỉ tập trung vào việc hiện thực hóa các chức năng phần mềm mà còn chú trọng đến phương pháp luận tổ chức quy trình phát triển. Mục tiêu cốt lõi là tối ưu hóa hiệu suất thực thi, nâng cao khả năng bảo trì và giảm thiểu các rủi ro phát sinh lỗi trong hệ thống. Để đạt được điều này, đề tài áp dụng kỹ thuật phân rã tác vụ nguyên tử (Atomic Task Decomposition), kết hợp chặt chẽ với nguyên lý giới hạn ngữ cảnh (Context Limitation) và triết lý đơn nhiệm (Single Responsibility Principle – SRP).

Phương pháp phân rã tác vụ nguyên tử được xây dựng dựa trên tư tưởng chia nhỏ các bài toán tổng thể có độ phức tạp cao thành các đơn vị xử lý độc lập, có phạm vi giới hạn và trách nhiệm duy nhất. Thay vì triển khai toàn bộ một chức năng phức tạp trong cùng một tiến trình, hệ thống được module hóa thành các bước nhỏ với ranh giới xử lý rõ ràng, giúp đơn giản hóa quá trình lập trình, kiểm thử và bảo trì.

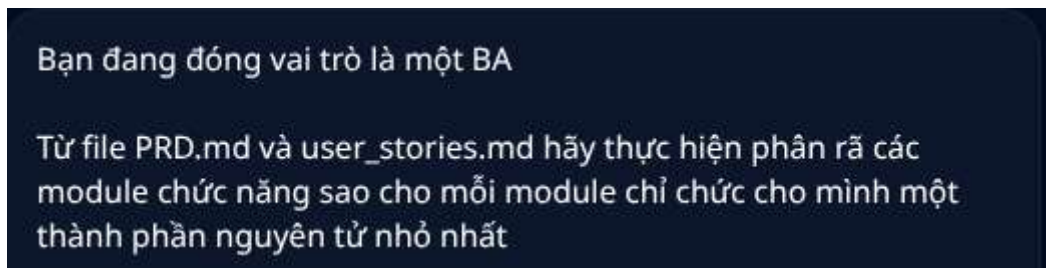
Đối với nguyên lý giới hạn ngữ cảnh, mục tiêu chủ đạo là hạn chế lượng thông tin và trạng thái mà mỗi module hoặc mỗi tiến trình cần phải quản lý tại một thời điểm nhất định. Trong các hệ thống phần mềm phân tầng bao gồm Backend, Frontend và Cơ sở dữ liệu, việc tích hợp quá nhiều trách nhiệm vào một thành phần duy nhất thường dẫn đến hiện tượng phụ thuộc chéo (tight coupling), gây khó khăn cho việc kiểm soát logic nghiệp vụ và định vị lỗi sâu chuỗi.

Do đó, đề tài thực hiện chiến lược cô lập hệ thống thành các phạm vi chức năng biệt lập, bao gồm:

- Module xác thực và phân quyền người dùng.
- Module quản lý hồ sơ sức khỏe và chỉ số sinh học.
- Module tính toán và gợi ý dinh dưỡng
- Module thống kê, phân tích dữ liệu và Dashboard.
- Module quản trị hệ thống tổng thể.

Mỗi module nêu trên vận hành tập trung trong một nhóm nghiệp vụ chuyên biệt, không can thiệp trực tiếp vào logic nội bộ của các thành phần khác. Quá trình giao tiếp và trao đổi dữ liệu giữa các thành phần được thực hiện đồng bộ thông qua tầng giao diện lập trình ứng dụng (API) hoặc tầng dịch vụ (Service Layer), bảo đảm tính độc lập cao (loose coupling) của toàn hệ thống.

Để thực hiện việc này nhóm chúng tôi đã sử dụng prompts như sau:



Bạn đang đóng vai trò là một BA

Từ file PRD.md và user_stories.md hãy thực hiện phân rã các module chức năng sao cho mỗi module chỉ chức cho mình một thành phần nguyên tử nhỏ nhất

Hình 3.12. Prompts thực hiện phân rã

Sau khi hoàn thành file module_breakdown.md, chúng tôi tiến hành đánh giá dựa trên các tiêu chí sau:

- Chuẩn kiến trúc Modular Monolith: Các module chức năng phải được phân rã độc lập theo từng gói tính năng (package-by-feature), đảm bảo tính đóng gói cao và giảm thiểu sự phụ thuộc lẫn nhau (loose coupling).

- Định nghĩa rõ ràng luồng dữ liệu (I/O): Mọi hàm chức năng con trong từng module phải chỉ ra chính xác các tham số đầu vào (Input) và dữ liệu trả về (Output) rõ ràng, hợp lý.
- Tính bao phủ yêu cầu (Requirement Coverage): Toàn bộ các User Stories từ tài liệu trước phải được ánh xạ đầy đủ vào các module chức năng (từ Auth, User Profile cho đến Admin), không bỏ sót tính năng.
- Lộ trình phát triển khả thi (Roadmap Realism): Các Sprint trong lộ trình phải được sắp xếp theo thứ tự ưu tiên hợp lý (từ tính năng cốt lõi MVP đến nâng cao), đảm bảo mỗi giai đoạn đều có một phiên bản phần mềm có thể chạy và kiểm thử được.

3.3.2 Thiết kế các Agents cần thiết cho dự án

Sau khi thực hiện phân rã hệ thống thành các tác vụ nguyên tử, đề tài tiến hành thiết lập kiến trúc các Agent hỗ trợ phát triển phần mềm và cơ chế định tuyến công việc. Trong bối cảnh hệ thống quản lý chế độ dinh dưỡng sử dụng mô hình đa nền tảng bao gồm Spring Boot, React Native và PostgreSQL, số lượng tác vụ phát sinh tại mỗi phân hệ cấu thành là vô cùng lớn. Nhằm tối ưu hóa năng suất và kiểm soát chặt chẽ quy trình lập trình, đề tài xây dựng giải pháp chuyên môn hóa các Agent phát triển dựa trên phạm vi ranh giới của từng module chức năng (Module-driven Agent Architecture) kết hợp hệ thống dữ liệu đặc tả .

Trong mô hình này, mỗi agent hoặc module AI chỉ được giao một nhóm nhiệm vụ cụ thể tương ứng với chuyên môn xử lý của nó. Cách tiếp cận này giúp giảm thiểu tình trạng quá tải ngữ cảnh và nâng cao độ chính xác trong quá trình sinh mã tự động.

Hệ thống định tuyến tác vụ được phân chia thành nhiều nhóm chức năng như:

- Backend Dev Agent
- React Dev Agent
- React Native Agent
- Planner Agent
- Doc Writer Agent
- Reviewer Agent
- Project Lead Agent

Mỗi agent chịu trách nhiệm xử lý một loại tác vụ riêng biệt.

Prompts sử dụng để có thể tạo ra các agents này

Dựa trên thông tin từ 3 tài liệu đính kèm:

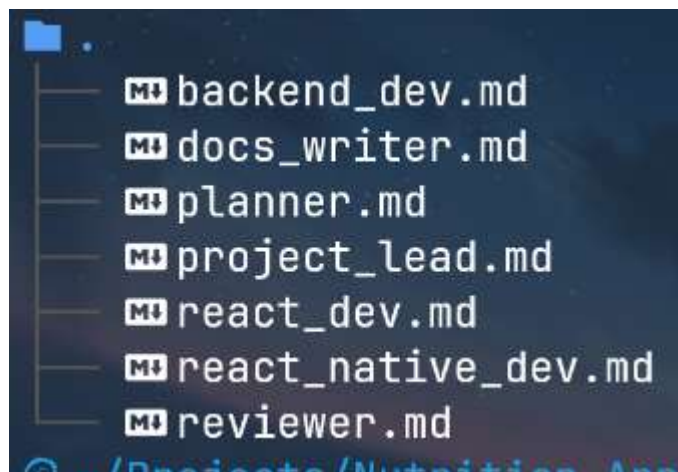
1. modules_breakdown.md (Cấu trúc phân rã các module)
2. business_rules.md (Các quy định và logic nghiệp vụ)
3. user_stories.md (Yêu cầu người dùng và tiêu chí chấp nhận)

Hãy thiết kế ra các Agents cần thiết cho hệ thống bao gồm

1. Project Lead Agent
2. Planner Agent
3. Backend Dev Agent
4. React Native Agent
5. React Dev Agent
6. Reviewer Agent
7. Doc Writer Agent

Hình 3.13. Prompts sử dụng để tạo ra các agents trong hệ thống

Kết quả:



Hình 3.14. Hình ảnh những agents đã được tôi xây dựng trong hệ thống

Việc chuyên môn hóa agent giúp tăng tính ổn định và giảm tỷ lệ sinh mã sai ngữ cảnh.

Ngoài ra, mỗi tác vụ nguyên tử còn được gắn metadata để phục vụ cho quá trình định tuyến tự động. Metadata bao gồm:

- name: Định danh duy nhất (Unique Identifier) của Agent. Thuộc tính này xác định nhãn chức năng hoặc phân hệ chuyên trách mà Agent đảm nhận (ví dụ: MealRecommendationAgent, UserAuthAgent), giúp bộ định tuyến phân biệt các Agent với nhau.
- models: Chỉ định mô hình ngôn ngữ lớn (LLM) hoặc nền tảng tri thức cốt lõi mà Agent này sử dụng để xử lý tác vụ (ví dụ từ hình ảnh: các dòng mô hình được tối

ưu hóa từ nvidia/...). Việc chỉ định này giúp tối ưu hóa chi phí token và đảm bảo năng lực tính toán phù hợp với độ phức tạp của module.

- **description**: Chuỗi văn bản mô tả ngắn gọn về phạm vi trách nhiệm, quyền hạn nghiệp vụ và ranh giới xử lý của Agent trong hệ thống. Bộ định tuyến có thể sử dụng thông tin này để phân tích ngữ cảnh (Semantic Matching) khi điều phối tác vụ.
- **temperature**: Tham số kiểm soát tính sáng tạo và độ ổn định trong kết quả phản hồi của Agent (giá trị trong ảnh được thiết lập ở mức thấp: 0.1). Đối với các tác vụ lập trình và xử lý logic nghiệp vụ phần mềm (như sinh mã Spring Boot hay truy vấn PostgreSQL), tham số này được siết chặt gần mức 0 để đảm bảo tính chính xác, nhất quán và giảm thiểu tối đa hiện tượng sai lệch thông tin (hallucination).
- **mode**: Chế độ vận hành hoặc cấu hình luồng thực thi chuyên biệt của Agent (ví dụ: chế độ thực thi tuần tự, chế độ hội thoại, hoặc chế độ phản hồi cấu trúc JSON cố định) nhằm tương thích với yêu cầu đầu ra của từng module chức năng.



```
"agent": {  
  "name": {  
    "models": "nvidia/",  
    "description": "...",  
    "temperature": 0.1,  
    "mode": "..."  
  },  
  ...  
}
```

Hình 3.15. JSON mô tả metadata

Thông tin metadata cho phép hệ thống xác định agent phù hợp để xử lý từng tác vụ cụ thể mà không cần truyền toàn bộ ngữ cảnh dự án.

Trong quy trình phát triển các phân hệ lỗi của hệ thống (như thuật toán gợi ý thực đơn và tính toán calo tiêu thụ), cơ chế định tuyến dựa trên cấu trúc đa Agent này mang lại hiệu quả quản lý rõ rệt. Các nhiệm vụ liên quan đến tối ưu hóa cấu trúc dữ liệu, xử lý logic tính toán phức tạp và hiển thị giao diện được cô lập hoàn toàn nhằm triệt tiêu xung đột mã nguồn. Tiến trình triển khai một phân hệ được phân lớp tường minh theo chuỗi phối hợp giữa các Agent:

- Database Architect Agent: Đảm nhiệm việc cấu hình, tối ưu hóa chỉ mục (Index) và cập nhật dữ liệu bảng thành phần dinh dưỡng (nutrition_dataset) trong PostgreSQL.
- Backend Dev Agent: Xây dựng hệ thống API RESTful để tiếp nhận các chỉ số sinh học và thực thi logic tính toán thực đơn.
- React Native Agent & React Dev Agent: Tiếp nhận dữ liệu cấu trúc từ API để kết xuất giao diện hiển thị cho người dùng cuối trên thiết bị di động và bảng điều khiển trực quan trên Web Admin.

Cách thức tổ chức này giúp toàn bộ chu trình phát triển (Development Pipeline) vận hành theo mô hình phân lớp kiến trúc nghiêm ngặt, tạo điều kiện thuận lợi cho việc kiểm soát mã nguồn và mở rộng quy mô hệ thống về sau.

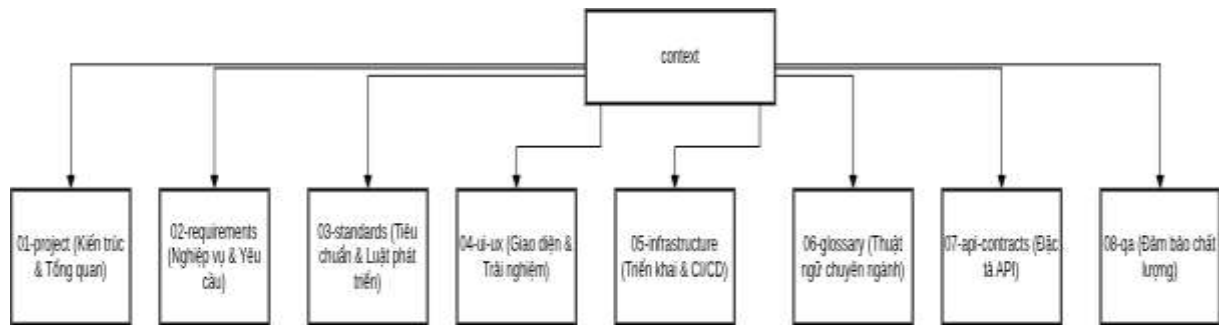
3.4 GIAI ĐOẠN 4: QUẢN TRỊ NGŨ CẢNH VÀ TỐI ƯU HÓA NGŨ CẢNH

3.4.1 Kiến trúc File Cấu hình Toàn cục Dự án

Trong các hệ thống phát triển phần mềm hiện đại có tích hợp AI hỗ trợ lập trình, việc quản trị ngữ cảnh (Context Engineering) đóng vai trò đặc biệt quan trọng nhằm đảm bảo các thành phần trong hệ thống luôn nhận được thông tin đầu vào chính xác, nhất quán và có phạm vi phù hợp với nhiệm vụ cần xử lý. Đối với đề tài xây dựng hệ thống quản lý chế độ dinh dưỡng và gợi ý bữa ăn, khối lượng mã nguồn lớn cùng với việc sử dụng nhiều công nghệ khác nhau như React-Native, Spring Boot, Postgre và React làm cho bài toán quản trị ngữ cảnh trở nên cần thiết hơn bao giờ hết.

Để giải quyết vấn đề này, đề tài tiến hành xây dựng kiến trúc file cấu hình toàn cục cho toàn bộ dự án. Hệ thống file cấu hình đóng vai trò như một tầng quản trị tri thức trung tâm, giúp chuẩn hóa thông tin kỹ thuật, định hướng kiến trúc và tối ưu tín hiệu đầu vào cho các lập trình viên cũng như AI Agent tham gia phát triển hệ thống.

Khác với các dự án truyền thống chỉ sử dụng file cấu hình cho mục đích môi trường chạy ứng dụng, kiến trúc được đề xuất trong đề tài còn mở rộng sang quản trị quy tắc lập trình, tiêu chuẩn kiến trúc, context dự án và metadata kỹ thuật. Điều này giúp giảm thiểu sai lệch ngữ cảnh khi phát triển nhiều module đồng thời.



Hình 3.16. Cấu trúc context của dự án

1. Thư mục 01-project (Tổng quan & Kiến trúc Hệ thống)

Thư mục này đặt nền móng cho hiểu biết chung về mặt kỹ thuật và định hướng sản phẩm.

Các tài liệu chính: project_overview.md, architecture.md, tech-stack.md.

Xác định tầm nhìn sản phẩm: ứng dụng theo dõi dinh dưỡng và tính toán calorie cá nhân hóa.

Định hình kiến trúc hệ thống: Mô hình Modular Monolith nghiêm ngặt cho Backend (Spring Boot 3 + Java 21) kết nối PostgreSQL và hai Frontend riêng biệt là Mobile Client (React Native + React Native Paper) và Admin Web (React + Ant Design + Tailwind CSS).

2. Thư mục 02-requirements (Yêu cầu Nghiệp vụ & Luật Dinh dưỡng)

Trung tâm của toàn bộ logic tính toán và các kịch bản người dùng trong hệ thống.

Các tài liệu chính: business-rules.md, module_breakdown.md, user_stories.md.

Luật nghiệp vụ cốt lõi (Calorie-Only Model): Tuyệt đối không dùng các chỉ số hệ Anh (Imperial) hay tích hợp thị giác máy tính AI. Hệ thống chuẩn hóa công thức tính BMI, BMR (Mifflin-St Jeor), TDEE, mục tiêu năng lượng hàng ngày và tính năng thay thế món ăn có độ lệch < 50 kcal.

Bản phân rã module và hệ thống User Stories chi tiết, giúp tác tử AI dễ dàng nắm bắt các yêu cầu chức năng MVP.

3. Thư mục 03-standards (Tiêu chuẩn Phát triển & Quy tắc Đồng thuận)

Định nghĩa hành vi, văn phong viết mã và giới hạn an toàn của mã nguồn.

Các tài liệu chính: coding-standards.md, rules.md.

Quy định rõ ràng cấu trúc phân lớp: Controller $\rightarrow$ Service

Luật tích hợp: Cấm tuyệt đối việc truy cập chéo Repository giữa các module khác nhau (mọi giao tiếp phải thông qua lớp Service).

Chuẩn hóa định dạng trả về thống nhất qua cấu trúc phong bì dữ liệu ApiResponse<T> (success, message, data, timestamp).

4. Thư mục 04-ui-ux (Hệ thống Thiết kế & Đặc tả Luồng Giao diện)

Thư mục có số lượng tệp lớn nhất (16 tài liệu), mô tả chi tiết giao diện của cả ứng dụng di động và trang quản trị.

Các tài liệu chính: global_design_system.md, các trạng thái UI dùng chung (global_ui_error_states.md, loading_states.md...) và phân rã chi tiết từng màn hình từ đăng nhập, khai báo sức khỏe (onboarding), nhật ký ăn uống (tracking) đến trang quản trị người dùng/món ăn.

Cung cấp cấu trúc giao diện trực quan, bảng màu và các trạng thái phản hồi của hệ thống. Giúp tác tử Frontend tạo ra các giao diện cao cấp, có tính tương tác cao và đồng bộ về mặt thẩm mỹ mà không cần suy đoán.

5. Thư mục 05-infrastructure (Hạ tầng kỹ thuật & Vận hành)

Các tài liệu chính: ci_cd.md.

Mô tả chi tiết quy trình kiểm thử tự động, tích hợp liên tục (CI) và đóng gói docker/triển khai liên tục (CD) nhằm đảm bảo quá trình phát hành phiên bản mới diễn ra an toàn.

6. Thư mục 06-glossary (Từ điển Thuật ngữ)

Các tài liệu chính: glossary.md.

Chuẩn hóa toàn bộ thuật ngữ chuyên ngành (BMR, TDEE, Macronutrients, LogFoods, Active factor) tránh nhập nhằng ngữ nghĩa giữa bộ phận lập trình và bộ phận kiểm thử nghiệp vụ.

7. Thư mục 07-api-contracts (Đặc tả Hợp đồng Giao tiếp)

Các tài liệu chính: api_docs.md.

Định nghĩa thiết kế đầu cuối (endpoints), phương thức HTTP (GET, POST, PUT, DELETE), cấu trúc JSON dữ liệu đầu vào và đầu ra mẫu của từng API, thiết lập hợp đồng giao tiếp chuẩn xác giữa Mobile client/Admin dashboard và Spring Boot backend.

8. Thư mục 08-qa (Đảm bảo Chất lượng & Kịch bản Kiểm thử)

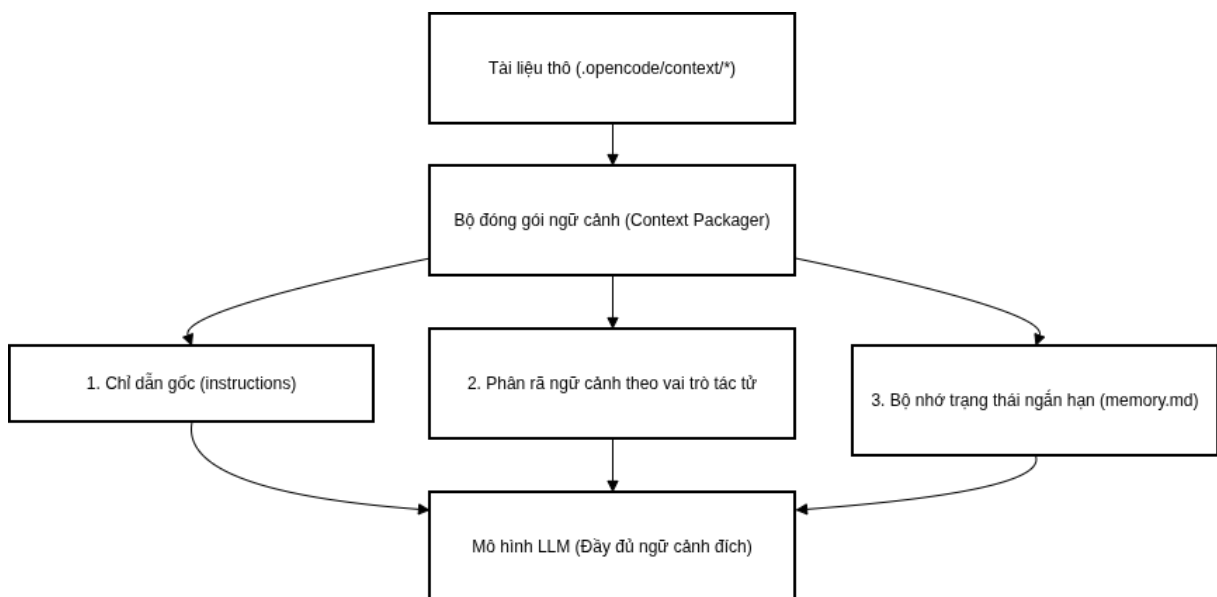
Các tài liệu chính: checklist.md, core_test_cases.md, test_plan.md.

Đặt ra các chỉ số chất lượng nghiêm ngặt: Độ phủ kiểm thử đơn vị (test coverage) > 80%, thời gian phản hồi API dưới 2 giây, bảo mật JWT/BCrypt.

Hệ thống hóa các kịch bản kiểm thử biên và kiểm thử tích hợp toàn trình để nghiệm thu tính năng của sản phẩm trước khi bàn giao.

3.4.2 Kỹ thuật Đóng gói Ngữ cảnh

Trong môi trường phát triển tự động hóa bằng AI, việc tối ưu hóa cửa sổ ngữ cảnh (Context Window) là yếu tố quyết định giúp tăng tốc độ phản hồi của mô hình ngôn ngữ lớn (LLM), hạn chế hiện tượng mất thông tin ở giữa (lost-in-the-middle) và tránh lãng phí tài nguyên mã thông báo (tokens). Dự án Nutrition App áp dụng các kỹ thuật đóng gói ngữ cảnh tiên tiến sau nhằm tối ưu hóa và cấu trúc luồng thông tin truyền tải:



Hình 3.17. Sơ đồ mô tả đóng gói ngữ cảnh

Đóng gói Ngữ cảnh Tĩnh và Chỉ dẫn Gốc (instructions): Hệ thống tự động biên dịch các tài liệu nền tảng như hướng dẫn phát triển chung (AGENTS.md), luật nghiệp vụ (business-rules.md) thành cấu trúc ngữ cảnh hệ thống (System Prompt) cố định. Điều này đảm bảo các ràng buộc bất biến—chẳng hạn như mô hình tính toán "calorie-only" hay quy tắc "Modular Monolith"—luôn được áp đặt trực tiếp vào tư duy của tất cả tác tử ở mọi phiên làm việc.

Phân rã Ngữ cảnh Chuyên biệt (Role-based Context Partitioning): Thay vì gửi toàn bộ tài liệu dự án gây nhiễu thông tin, hệ thống tự động lọc và đóng gói tài liệu theo từng

vai trò cụ thể. Tác tử phát triển Backend chỉ tiếp nhận đặc tả API, mô hình cơ sở dữ liệu và tiêu chuẩn mã nguồn Spring Boot; trong khi đó, tác tử Frontend được bao gói riêng biệt hệ thống Design System và sơ đồ luồng UI/UX của các màn hình tương ứng.

Liên kết Ngữ cảnh Động bằng Đường dẫn Chuẩn hóa: Các tài liệu trong thư mục `.opencode/context/` được thiết lập mạng lưới liên kết chéo qua cú pháp Markdown chuẩn hóa. Các tác tử có khả năng tự động "duyet" và tải thêm ngữ cảnh có liên quan khi thực thi tác vụ sâu bằng cách truy vết theo các mối quan hệ được định nghĩa sẵn trong tài liệu.

Đồng bộ hóa Trạng thái Ngắn hạn (memory.md): Đóng gói lịch sử phát triển ngắn hạn vào tệp tin bộ nhớ tiến trình, giúp các tác tử mới tham gia hoặc tiếp tục công việc kế thừa ngay lập tức trạng thái hiện tại (active state), những việc đã hoàn thành và các rào cản kỹ thuật mà không cần phải phân tích lại toàn bộ lịch sử hội thoại dài.

Nhờ các kỹ thuật này, hệ thống duy trì được mật độ thông tin hữu ích cao, nâng cao độ chính xác trong việc ra quyết định kỹ thuật của các tác tử AI.

3.5 GIAI ĐOẠN 5: TRIỂN KHAI LẬP TRÌNH BẰNG PHƯƠNG PHÁP VIBE CODING VÀ ĐA TÁC TỬ

3.5.1. Thực hiện sinh các giao diện từ context

Giai đoạn này hiện thực hóa các đặc tả kỹ thuật thành sản phẩm thực tế thông qua phương thức Vibe Coding kết hợp hệ thống đa tác tử AI. Bằng cách sử dụng tác tử chuyên biệt Stitch trên nền tảng mô hình Gemini 3.1 Pro, hệ thống tự động chuyển đổi các đặc tả UI/UX tĩnh trong thư mục ngữ cảnh thành mã nguồn giao diện di động (React Native) và quản trị (React.js) hoàn chỉnh, bảo đảm tính nhất quán cao về mặt thẩm mỹ.

Prompts sử dụng cho cho giai đoạn này:

```

ADMIN: 4 pages
1. Dashboard => 3 navigation: user management, food
management, category management
Sidebar: User, Food, Category
=> User management (1 page, 1 modal) => CRUD (no
delete) user, search user by email, filter status (Status:
All / Active / Locked), name, user detail (fullname, email,
gender, height, weight, BMI, target weight, birthdate,
activity level) => 1 modal create, read, update

=> Food management (1 page, 1 modal) => CRUD (no
delete) food, search food by category, name, food detail
(name, ingredient, calories, protein, carb, fat, image) => 1
modal create, read, update.

=> Category management (1 page, 1 modal) => CRUD (no
delete), category detail (name) => 1 modal create,
update

2. Login => 1 page => email, password

CLIENT: 8 pages
1. Sign up => 1 page => email, password, cf pw

2. Health information => 1 page => form user detail (
fullname, email, gender birthdate, height, weight,
BMI, option (lost weight, gain weight, balance weight),
target weight, lose or gain weight/week (0 -> 1)kg,
activity level(low, high, average))

3. Login => 1 page => email, password

4. Dashboard => 3 navigations: tracking, daily, profile
=> Tracking (1 page):
- calendar (1 section)
- tổng quan dinh dưỡng (1 section) (calories, protein, fat,
carb) theo calendar
- food ghi nhận theo calendar (1 section includes list)
(name food, time, calories, protein, fat, carb, image) =>
delete, 1 modal update,
- line chart (1 section) for calories every day/month (

```

Hình 3.18. Prompts sử dụng để tạo context cho giao diện

Sau khi đã có được phần context ta tiếp tục sử dụng những context này để có thể chính thức sinh ra các thiết kế giao diện hoàn chỉnh (không phải code).

Prompts sử dụng

Dưới đây là toàn bộ 17 file Markdown (.md) đã được tổng hợp, chuẩn hóa cấu trúc và tách biệt hoàn toàn để bạn có thể copy từng block code một cách dễ dàng nhất.

Bộ file này được chia làm 4 nhóm: **Design System (1 file)**, **Admin Web (4 files)**, **Client Mobile (8 files)**, và **Interactive UI States (4 files)**.

🔴 PHẦN 1: HỆ THỐNG DESIGN SYSTEM DÙNG CHUNG

File 1: `global_design_system.md`

```
Markdown

# UI-UX Design System Foundations

## 🎨 Color Palettes

### 📱 Client App (Mobile) - Premium Dark Vibe
* Background: #0F172A (Dark Slate)
* Surface/Cards: #1E293B (Slate 800)
* Borders/Dividers: #334155 (Slate 700)
* Primary Accent: #2ED5C5 (Mint Green)
* Secondary Accent: #3B82F6 (Electric Blue)
* Warm Accent: #FFB400 (Sunshine Yellow)
* Text Primary: #F8FAFC (White)
* Text Secondary: #94A3B8 (Gray)

### 💻 Admin Dashboard (Web) - Professional Light Vibe
* Background: #FAFAFA (Light Gray)
* Surface/Cards: #FFFFFF (Pure White)
* Borders: #E0E0E0 (Light Border)
* Primary Accent: #76FF03 (Lime Green)
* Secondary Accent: #303F9F (Deep Indigo)
* Text Primary: #212121 (Dark Gray)
* Text Secondary: #757575 (Medium Gray)

## 📏 Typography & Layout Base
* Font Family: Inter & Outfit (Sans-Serif)
* Spacing Grid: 8-point system (8px, 16px, 24px, 32px padding/margins)
* Corner Radius: 8px for standard cards, 12px for modals, 4px for buttons
* Elevation: Shadow-1 (@ 1px 3px rgba(0,0,0,0.12)), Shadow-2 (@ 4px 6px rgba(0,0,0,0.12))
* Icons: Feather Icons style (Outline 2px stroke)
```

💻 PHẦN 2: HỆ THỐNG WEB ADMIN (LIGHT MODE)

File 2: `admin_p1_login.md`

Hình 3.19. Prompts sử dụng tạo ra giao diện từ context

Kết quả đạt được giao diện cho một số chức năng tiêu biểu như:

Tell Us About Yourself

Personal Information

FULL NAME

GENDER

☒ Male ☐ Female ☐ Other

BIRTHDATE

Body Metrics

HEIGHT (CM)

WEIGHT (KG)

BMI Score

22.4

NORMAL

Select Goal Strategy

TARGET WEIGHT (KG)

☒ Lose ☐ Maintain ☐ Gain

TARGET RATE PER WEEK

0.0 kg 1.0 kg

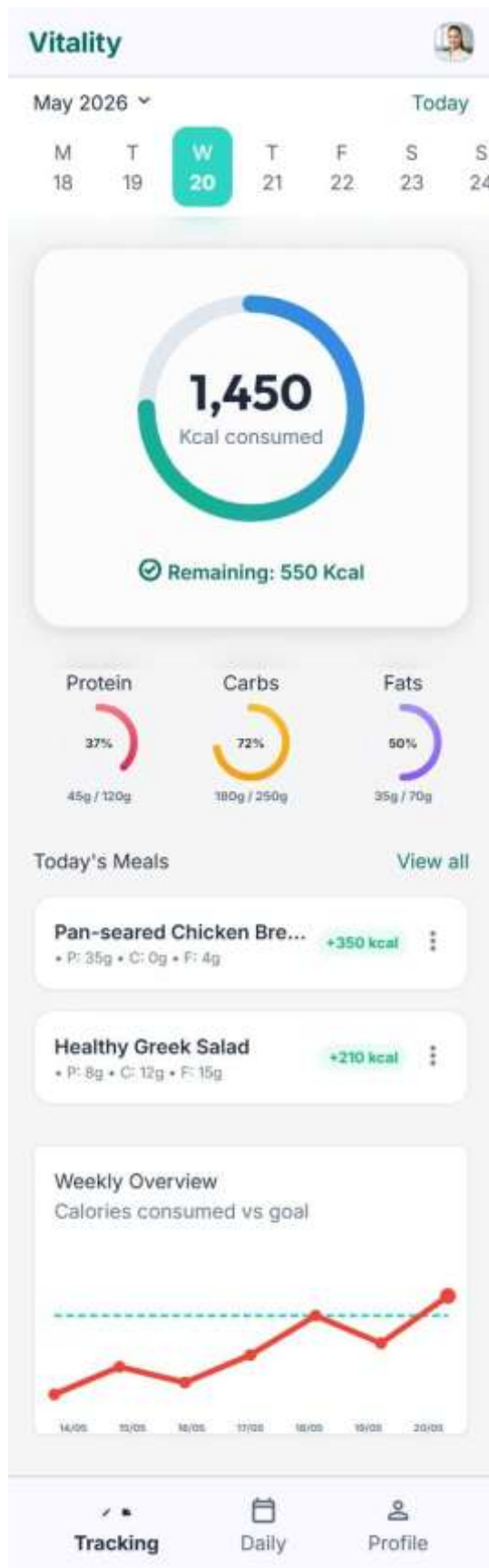
Daily Activity Level

☐ SEDENTARY ☐ LIGHT_ACTIVE

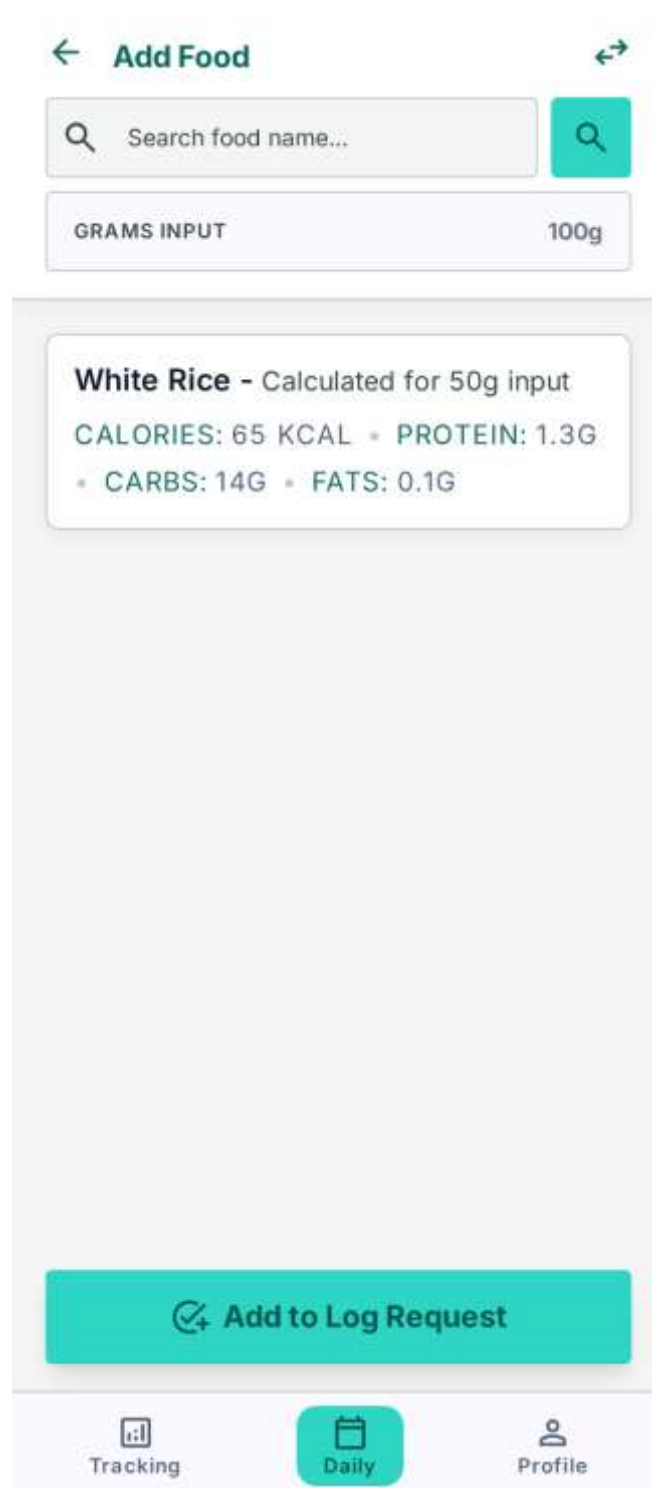
☒ ACTIVE ☐ VERY_ACTIVE

Submit

Hình 3.20. Ảnh giao diện cho chức năng tính BMI



Hình 3.21. Ảnh giao diện sinh ra cho chức năng lộ trình dinh dưỡng - 1



Hình 3.22. Ảnh giao diện sinh ra cho chức năng lộ trình dinh dưỡng - 2

3.5.1 Vòng lặp Thực thi Trọng tâm (Inner Loop) và Triết lý Phát triển Hướng Kiểm thử (TDD)

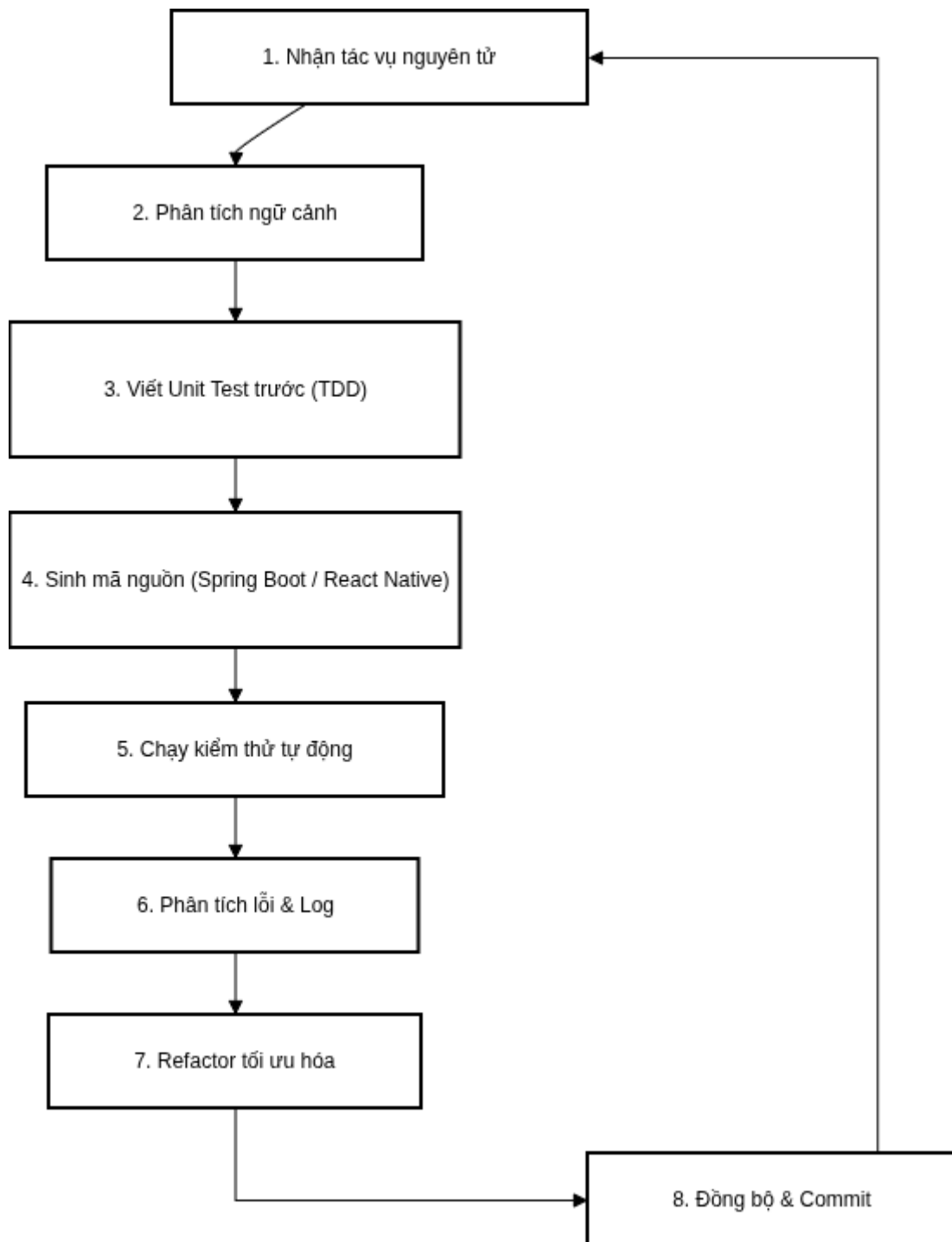
Sau khi hoàn thiện các giai đoạn phân rã tác vụ và tối ưu hóa ngữ cảnh, đề tài tiến hành triển khai hệ thống theo mô hình lập trình đa tác tử kết hợp phương pháp Vibe Coding. Đây là phương pháp phát triển phần mềm tận dụng sự phối hợp chặt chẽ giữa lập trình

viên và các tác tử AI chuyên biệt nhằm tăng tốc quá trình xây dựng hệ thống, đồng thời duy trì chất lượng mã nguồn và khả năng mở rộng của kiến trúc Modular Monolith.

Trong mô hình này, toàn bộ quy trình phát triển được tổ chức xoay quanh vòng lặp thực thi trọng tâm (Inner Loop). Inner Loop là chu trình xử lý liên tục giữa việc nhận tác vụ nguyên tử, sinh mã nguồn, chạy kiểm thử, phản hồi lỗi và tối ưu hóa hệ thống ở phạm vi nhỏ trước khi tích hợp vào hệ thống tổng thể.

Nhằm bảo đảm tính chính xác của các logic tính toán dinh dưỡng cốt lõi, đề tài áp dụng triết lý phát triển hướng kiểm thử (Test-Driven Development – TDD). Theo phương pháp này, kiểm thử được xây dựng trước khi hoàn thiện logic xử lý chính thức. Điều này giúp xác định rõ hành vi mong muốn của hệ thống ngay từ đầu và giảm thiểu nguy cơ phát sinh lỗi nghiệp vụ trong quá trình phát triển.

Quy trình Inner Loop trong đề tài được tổ chức theo 8 bước cơ bản:



Hình 3.23. Vòng lặp inner loop

Trong hệ thống quản lý chế độ dinh dưỡng, mỗi tác vụ nhỏ đều được triển khai theo mô hình TDD nhằm đảm bảo tính chính xác của các công thức tính toán. Ví dụ, đối với chức năng tính chỉ số cơ thể, quy trình được thực hiện qua việc xây dựng kiểm thử trước cho công thức tính BMI, BMR (Mifflin-St Jeor) và TDEE, xác định các biên giới hạn dữ liệu đầu vào hợp lệ (chiều cao, cân nặng) trước khi hiện thực lớp Service xử lý nghiệp vụ trên Backend Spring Boot.

Trong mô hình thực thi đa tác tử (Multi-Agent Execution), mỗi AI Agent tham gia vào một giai đoạn chuyên biệt trong Inner Loop nhằm giảm tải ngữ cảnh và tăng độ chính xác:

Planner Agent: Phân tích backlog và chia nhỏ tác vụ thành các đơn vị nguyên tử.

Backend Dev Agent: Sinh mã nguồn API Spring Boot 3 và business logic kết nối cơ sở dữ liệu PostgreSQL.

React Native Dev Agent / React Dev Agent: Xây dựng giao diện ứng dụng di động (React Native) và bảng quản trị (React.js) theo đặc tả giao diện.

Reviewer Agent: Kiểm tra các tiêu chuẩn viết mã, đảm bảo hiệu năng phản hồi API dưới 2 giây và độ bao phủ kiểm thử đạt trên 80%.

Docs Writer Agent: Đồng bộ hóa bộ nhớ trạng thái dự án và ghi nhận các quyết định thiết kế.

Hệ thống vận hành cơ chế phản hồi liên tục (Continuous Feedback). Sau mỗi vòng thực thi, kết quả kiểm thử và log lỗi được gửi ngược lại cho tác tử tương ứng: nếu API trả sai cấu trúc phong bì ApiResponse<T>, Backend Dev Agent sẽ sửa đổi; nếu truy vấn PostgreSQL thiếu tối ưu hóa, hệ thống sẽ thực hiện tối ưu hóa cấu trúc index và liên kết bảng.

Đối với frontend di động React Native và web React, phương pháp TDD cũng được áp dụng thông qua các kịch bản kiểm thử giao diện nhằm xác thực logic render, quản lý trạng thái và luồng điều hướng của ứng dụng, đảm bảo tính ổn định cao nhất trên thiết bị người dùng.

3.5.2 Cơ chế Điều phối, Phân vai và Kiểm tra Chéo Hệ thống Đa Tác tử (Project Lead, Coder, Reviewer)

Trong giai đoạn triển khai mã nguồn của Nutrition App, phương pháp luận Vibe Coding loại bỏ hoàn toàn mô hình tương tác đơn tác tử (Single-agent) nhằm triệt tiêu rủi ro sai sót dây chuyền. Thay vào đó, hệ thống thiết lập một quy trình phối hợp tự hành dựa trên cơ chế phân vai và kiểm tra chéo nghiêm ngặt giữa bộ ba nhóm tác tử AI chuyên biệt:



Hình 3.24. Vòng lặp inner loop

Tác tử Kiến trúc và Điều phối (Project Lead Agent): Đóng vai trò định hướng và thiết lập rào cản kỹ thuật của toàn bộ dự án. Tiếp nhận yêu cầu từ tài liệu backlog, tác tử này phân tích cấu trúc tổng thể của hệ thống dựa trên nền tảng Spring Boot (Backend) và React Native/React.js (Frontend), tiến hành bóc tách các tính năng phức tạp thành sơ đồ thực thi nguyên tử. Trước khi cho phép viết mã, Project Lead Agent đối chiếu với hệ thống tiêu chuẩn kỹ thuật (coding-standards.md) để quy định chính xác kiến trúc đa tầng nhằm ngăn ngừa sự suy thoái cấu trúc và bảo đảm các ràng buộc nghiệp vụ calorie-only được tuân thủ nghiêm ngặt.

Các Tác tử Lập trình Chuyên biệt (Coder Agents): Hoạt động độc lập trong các không gian ngữ cảnh cô lập để tránh mã nhiễu. Trong đó, Backend Dev Agent tập trung xây dựng các RESTful API bằng Java 21 và Spring Boot 3. Nhiệm vụ chính là hiện thực hóa các dịch vụ nghiệp vụ tính toán dinh dưỡng như BMI, BMR (Mifflin-St Jeor) và TDEE, ánh xạ các thực thể JPA vào cơ sở dữ liệu PostgreSQL, mã hóa mật khẩu với BCrypt, xử lý ngoại lệ tập trung qua `@ControllerAdvice` và đóng gói kết quả phản hồi qua cấu trúc chuẩn `ApiResponse<T>`. Song song đó, React Native Dev Agent và React Dev Agent phụ trách phát triển giao diện ứng dụng di động cho người dùng và giao diện quản trị Admin Web. Các tác tử này sử dụng TypeScript nghiêm ngặt, tối ưu hóa hiệu năng render bằng cách tránh lồng ghép danh sách thông qua việc sử dụng `FlatList`, quản lý trạng thái bằng Redux Toolkit hoặc Zustand, và kết nối API bằng Axios client có bộ chặn xử lý tự động xoay vòng mã thông báo JWT.

Tác tử Kiểm toán và Phản biện (Reviewer Agent): Đóng vai trò là chốt chặn độc lập và nghiêm ngặt nhất để kiểm soát chất lượng mã nguồn trước khi hợp nhất. Reviewer Agent thực hiện quy trình kiểm toán mã nguồn toàn diện. Tác tử này rà soát để đảm bảo không có hiện tượng gọi chéo repository trực tiếp giữa các module của Modular Monolith, bắt buộc các module phải giao tiếp thông qua Service Interface. Đồng thời, Reviewer Agent kiểm soát an ninh thông qua kiểm tra cơ chế xác thực JWT, phân quyền dựa trên vai trò

(RBAC), phòng ngừa lỗi SQL Injection, và rà soát hiệu năng bảo đảm thời gian phản hồi của mọi API luôn dưới 2 giây. Cuối cùng, tác tử phân tích sâu các câu lệnh truy vấn JPA để loại bỏ triệt để lỗi N+1 bằng giải pháp JOIN FETCH và yêu cầu độ phủ kiểm thử đơn vị phải đạt trên 80% trước khi phê duyệt tích hợp.

Quy trình điều phối vận hành theo một chu kỳ lặp liên tục và khép kín: Project Lead giao việc, Coder viết mã và unit test, sau đó Reviewer kiểm toán độc lập. Nếu phát hiện bất kỳ lỗi bảo mật, sự suy giảm hiệu năng truy vấn, hoặc vi phạm ranh giới module, mã nguồn sẽ ngay lập tức bị từ chối và gửi ngược lại kèm theo một Biên bản Kiểm toán chi tiết để sửa đổi. Tác vụ chỉ được ghi nhận hoàn thành khi tất cả các kiểm thử tự động vượt qua và đạt được sự phê duyệt từ Reviewer, bảo đảm tính bền vững vượt trội cho mã nguồn của toàn bộ dự án.

3.5.3 Tiêu chuẩn Hợp nhất Mã nguồn và Kiểm soát Hành vi của Coder Agent

Việc triển khai song song nhiều tác tử lập trình (Coder Agents) giúp đẩy nhanh tiến độ dự án nhưng đồng thời đặt ra thách thức lớn về tính nhất quán của mã nguồn. Nhằm triệt tiêu rủi ro xung đột logic trên kiến trúc Modular Monolith của Nutrition App, đề tài thiết lập các cơ chế kiểm soát hành vi tác tử và tiêu chuẩn hợp nhất nghiêm ngặt:

- Cô lập Ngữ cảnh (Context Isolation): Mỗi Coder Agent chỉ được vận hành trong phạm vi tác vụ nguyên tử được đóng gói sẵn. Backend Dev Agent không được tự ý thay đổi giao diện di động React Native, và ngược lại, các tác tử Frontend không được phép can thiệp vào schema cơ sở dữ liệu PostgreSQL.
- Kiểm soát Hành vi Sinh mã: Các tác tử phải tuân thủ tuyệt đối quy tắc trong rules.md (cấm tự ý bổ sung thư viện/dependencies, không hardcode dữ liệu nhạy cảm, cấm nhân bản logic nghiệp vụ và cấm gọi trực tiếp cơ sở dữ liệu từ tầng UI).
- Quy trình Xác thực Hợp nhất (CI/CD): Trước khi tích hợp vào nhánh chính, mã nguồn phải vượt qua các chốt chặn kiểm thử tự động gồm: kiểm tra định dạng (spotless:check), quét bảo mật JWT/BCrypt, kiểm tra tính tương thích API và bảo đảm độ phủ kiểm thử đạt trên 80%.
- Quản lý bằng Commit Nguyên tử (Atomic Commits): Mọi thay đổi mã nguồn phải được phân vệt theo từng tác vụ nguyên tử cụ thể, hỗ trợ cơ chế Rollback

nhANH chóng để khôi phục trạng thái ổn định khi phát sinh lỗi tương thích hệ thống.

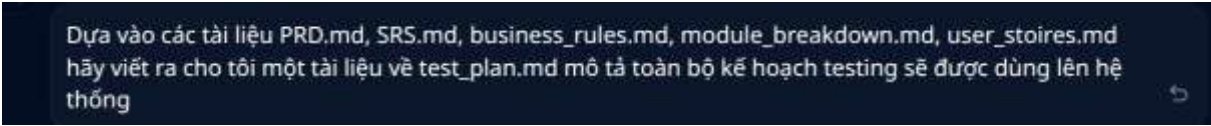
Sự phối hợp giữa cơ chế cô lập ngữ cảnh và đường ống CI/CD tự động bảo đảm hệ thống luôn vận hành an toàn, tin cậy và đạt tính chuẩn hóa cao.

3.6 GIAI ĐOẠN 6: KIỂM THỬ TỰ ĐỘNG VÀ CƠ CHẾ TỰ PHỤC HỒI HỆ THỐNG

3.6.1 Chiến lược Kiểm thử và sinh Test Plan

Trong mô hình phát triển phần mềm tự động hóa cao như Vibe Coding, quy trình kiểm thử đóng vai trò là "chốt chặn" kỹ thuật tối cao nhằm bảo đảm tính ổn định toàn hệ thống. Quy trình này dịch chuyển trọng tâm sang Chiến lược Kiểm thử Đa tầng và Tích hợp Toàn diện (Multi-layered & End-to-End Integration Testing). Do hệ thống Nutrition App vận hành trên kiến trúc Modular Monolith phân tầng—bao gồm ứng dụng di động React Native, ứng dụng quản trị Web React và lõi nghiệp vụ Spring Boot tích hợp cơ sở dữ liệu PostgreSQL—quy trình kiểm thử tích hợp đóng vai trò quyết định nhằm bảo đảm toàn bộ luồng dữ liệu được liên thông chính xác khi đi qua biên giới giữa các nền tảng kỹ thuật khác nhau.

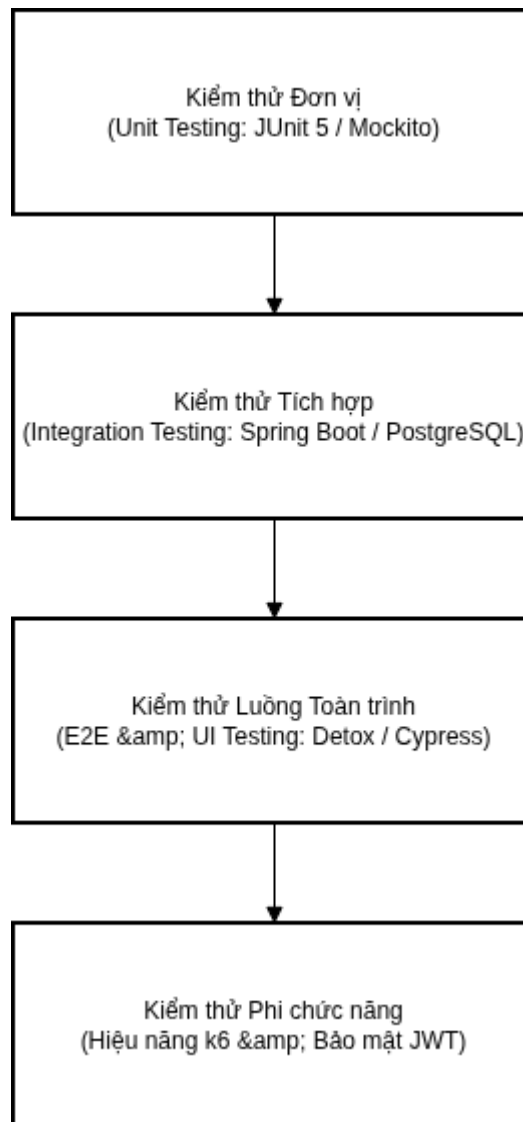
Để thực hiện sinh test plan nhóm chúng tôi đã sử dụng prompts



```
Dựa vào các tài liệu PRD.md, SRS.md, business_rules.md, module_breakdown.md, user_stories.md  
hãy viết ra cho tôi một tài liệu về test_plan.md mô tả toàn bộ kế hoạch testing sẽ được dùng lên hệ  
thống
```

Hình 3.25. Prompts sinh test plan

Chiến lược này được xây dựng dựa trên mô hình hình tháp kiểm thử chuẩn hóa gồm bốn cấp độ kiểm soát chặt chẽ:



Hình 3.26. Luồng kiểm thử

- Kiểm thử Đơn vị (Unit Testing): Sử dụng bộ công cụ JUnit 5 và Mockito nhằm xác thực độc lập các thuật toán tính toán chỉ số sức khỏe cốt lõi (BMI, công thức Mifflin-St Jeor cho BMR, chỉ số TDEE và định lượng năng lượng mục tiêu hằng ngày) cách ly khỏi cơ sở dữ liệu. Tiêu chuẩn chất lượng yêu cầu độ phủ kiểm thử (code coverage) nghiêm ngặt đạt $\geq 90\%$ cho lớp Service và $\geq 80\%$ cho lớp Controller.
- Kiểm thử Tích hợp (Integration Testing): Xác thực sự tương tác và tính toàn vẹn dữ liệu giữa Spring Boot API và cơ sở dữ liệu PostgreSQL. Các bài kiểm thử tích hợp tập trung xác minh tính đúng đắn của các giao dịch trên thực thể liên kết Many-to-Many của bảng log_foods thông qua khóa phức hợp LogFoodsId. Quy trình sử dụng @SpringBootTest kết hợp công cụ Testcontainers để giả lập môi

trường cơ sở dữ liệu PostgreSQL cô lập trong đường ống tích hợp liên tục (CI/CD).

- Kiểm thử Luồng và Giao diện Toàn trình (End-to-End & Functional UI Testing): Giả lập toàn bộ hành trình người dùng thực tế từ giao diện người dùng đến tầng lưu trữ dữ liệu cuối cùng. Luồng kiểm thử mô phỏng các kịch bản như: người dùng đăng ký tài khoản $\rightarrow$ cập nhật hồ sơ sức khỏe $\rightarrow$ tìm kiếm thực phẩm Việt $\rightarrow$ lưu nhật ký dinh dưỡng hằng ngày. Quá trình kiểm thử UI được tự động hóa bằng công cụ Detox đối với ứng dụng di động React Native và Cypress/Playwright đối với bảng điều khiển Admin Web.
- Kiểm thử Phi chức năng (Non-Functional Testing): Tập trung vào hai trụ cột chính là hiệu năng và an ninh bảo mật. Kiểm thử hiệu năng sử dụng công cụ k6 hoặc Apache JMeter để đo tải hệ thống dưới áp lực truy cập đồng thời lớn, đảm bảo thời gian phản hồi API cứng dưới 2.0 giây. Kiểm thử an ninh tiến hành rà soát tính toàn vẹn của mã thông báo JWT (Access/Refresh Token), rà soát phân quyền dựa trên vai trò (RBAC) và kiểm tra tính năng khóa tài khoản tự động khi xảy ra brute-force.

3.6.2 Chu trình Tự động Quét Log, Bóc tách Stack Trace và Khởi tạo Bản vá

Khi các kịch bản kiểm thử tự động phát hiện ra lỗi hoặc hệ thống gặp sự cố trong quá trình vận hành, hệ thống sẽ kích hoạt Chu trình Tự phục hồi (Self-healing Loop). Đây là cơ chế khép kín giúp tự động hóa quy trình xử lý sự cố từ khâu phát hiện lỗi đến khi triển khai bản vá dựa trên tài liệu đặc tả chất lượng (test_plan.md):

- Quét Log và Phát hiện Bất thường (Log Monitoring): Tác tử Giám sát liên tục quét qua các tệp nhật ký hệ thống (System Logs) của ứng dụng di động React Native, Web Admin React và lỗi Spring Boot. Khi xuất hiện mã lỗi HTTP 5xx, các ngoại lệ hệ thống (ValidationException, NullPointerException), hoặc hiệu năng phản hồi API vượt ngưỡng rào cản kỹ thuật cứng (response time > 2.0 giây), tác tử sẽ lập tức cô lập phân đoạn log liên quan.
- Bóc tách và Phân tích Stack Trace (Stack Trace Analysis): Tác tử tiến hành bóc tách chuỗi vết ngăn xếp (Stack Trace) để định vị chính xác vị trí lỗi tại tầng Controller, Service hay Repository trong cấu trúc Spring Boot. Quá trình phân

tích nguyên nhân gốc rễ (Root Cause) tập trung vào việc xác định các lỗi nghiêm trọng (Blocker/Critical) như sai lệch công thức Mifflin-St Jeor, lỗi giao dịch khóa phức hợp LogFoodsId trên PostgreSQL hoặc lỗi giải mã mã thông báo JWT.

- Khởi tạo Bản vá và Kiểm thử Tự động (Patch Generation & Verification): Sau khi xác định nguyên nhân, Coder Agent được kích hoạt trong môi trường cô lập để sinh bản vá sửa đổi logic mã nguồn hoặc bổ sung kiểm soát ràng buộc dữ liệu (Validation DTO). Bản vá này được đẩy qua hệ thống CI/CD để chạy lại toàn bộ các bài Unit và Integration Test.
- Hợp nhất và Kiểm soát Tiêu chuẩn Nghiệm thu (Exit Criteria Alignment): Bản vá chỉ được phép tích hợp vào nhánh chính khi đáp ứng đầy đủ tiêu chuẩn nghiệm thu UAT: 100% kịch bản kiểm thử cốt lõi vượt qua, không phát sinh lỗi dây chuyền (Regression), bảo đảm độ phủ kiểm thử của lớp Service đạt $\geq 90\%$ và không còn bất kỳ lỗi Blocker hay Critical nào tồn đọng.

Cơ chế tự phục hồi này tối ưu hóa công tác xử lý sự cố, giảm thiểu tối đa thời gian chết hệ thống và duy trì độ tin cậy cao của ứng dụng.

3.6.3 Giám sát Trạng thái và Tiêu chuẩn Tỷ lệ Bao phủ Kiểm thử

Để bảo đảm ứng dụng Nutrition App đạt chất lượng chuẩn công nghiệp trước khi vận hành thực tế, đề tài thiết lập các ranh giới kiểm soát chất lượng kỹ thuật nghiêm ngặt. Hệ thống giám sát này được cấu trúc thành ba trụ cột chính bao gồm: Tiêu chuẩn tỷ lệ bao phủ kiểm thử, Phân cấp quản lý lỗi và Tiêu chuẩn nghiệm thu toàn hệ thống.

1. Tiêu chuẩn Tỷ lệ Bao phủ Kiểm thử (Test Coverage Standards)

Hoạt động kiểm thử đơn vị được thực thi tự động qua bộ đôi công cụ JUnit 5 và Mockito nhằm cô lập hoàn toàn logic xử lý nghiệp vụ khỏi các phụ thuộc bên ngoài. Đề tài áp dụng chỉ số đo lường độ bao phủ mã nguồn (Code Coverage) thông qua công cụ Jacoco với các ngưỡng tối thiểu bắt buộc:

Lớp Nghiệp vụ (@Service Layer): Đạt tỷ lệ bao phủ mã nguồn tối thiểu 90%, bảo đảm tất cả các luồng xử lý tính toán công thức sức khỏe (BMI, Mifflin-St Jeor cho BMR/TDEE) và thuật toán gợi ý món ăn thay thế hoạt động chính xác.

Lớp Kiểm soát API (@RestController Layer): Đạt tỷ lệ bao phủ mã nguồn tối thiểu 80% xác thực tính đúng đắn của toàn bộ các điểm cuối REST, các bộ kiểm soát dữ liệu đầu vào (DTO Validation) và các cấu trúc phản hồi chuẩn hóa ApiResponse<T>.

2. Giám sát Trạng thái và Phân cấp Lỗi kỹ thuật (Defect Severity & Lifecycle)

Mọi lỗi kỹ thuật phát sinh trong quá trình vận hành hoặc kiểm thử đều được đăng ký tự động trên hệ thống giám sát và phân cấp rõ ràng theo bốn mức độ nghiêm trọng:

Mức độ Blocker (Lỗi chặn): Các sự cố gây sập hệ thống, rò rỉ bảo mật dữ liệu sức khỏe người dùng, hoặc các lỗi ngăn cản trực tiếp luồng nghiệp vụ chính (như lỗi đăng ký, đăng nhập).

Mức độ Critical (Lỗi nghiêm trọng): Logic nghiệp vụ cốt lõi bị tính toán sai lệch (ví dụ: sai công thức BMI hoặc sai mục tiêu calories hằng ngày).

Mức độ Major (Lỗi lớn): Các chức năng chính hoạt động tốt nhưng gặp lỗi ở các trường hợp biên hoặc lỗi giao diện nghiêm trọng làm ảnh hưởng đến trải nghiệm người dùng (ví dụ: biểu đồ tiến trình không hiển thị).

Mức độ Minor (Lỗi nhỏ): Các lỗi về chính tả, lệch nhẹ bố cục giao diện hoặc thời gian tải hình ảnh từ Cloud Storage chậm hơn thông thường.

Chu trình xử lý lỗi tuân thủ nghiêm ngặt quy trình bốn bước khép kín:

Phát hiện lỗi -> Phân bổ xử lý cho Coder Agent tương ứng -> Thực thi sửa đổi mã nguồn
Xác thực lại bởi Reviewer Agent. Trạng thái lỗi chỉ được chuyển sang Closed khi đã vượt qua các bài kiểm thử xác thực.

3. Tiêu chuẩn Nghiệm thu Toàn diện Hệ thống (Exit & UAT Criteria)

Hệ thống Nutrition App chỉ được phép phát hành chính thức khi thỏa mãn đồng thời các điều kiện biên dưới đây:

Chỉ số Chất lượng Mã nguồn: Vượt qua 100% kịch bản kiểm thử cốt lõi; đạt đầy đủ các ngưỡng bao phủ kiểm thử đơn vị tối thiểu (90% lớp Service, 80% lớp Controller).

Chỉ số Lỗi tồn đọng: Tuyệt đối không còn bất kỳ lỗi Blocker, Critical hoặc Major nào ở trạng thái mở (Zero open bugs). Các lỗi Minor không vượt quá 5 lỗi và bắt buộc phải có giải pháp xử lý tạm thời được duyệt.

Chỉ số Hiệu năng hệ thống: 95% các yêu cầu API phải phản hồi dưới 2.0 giây dưới tải giả lập; không phát sinh rò rỉ bộ nhớ trong các bài stress test.

Trạng thái Cơ sở Dữ liệu: Toàn bộ lịch sử di cư cơ sở dữ liệu PostgreSQL phải được Flyway thực thi thành công hoàn toàn, không có xung đột schema.

3.7 GIAI ĐOẠN 7: TỔNG KẾT, ĐÓNG GÓI

Giai đoạn cuối cùng của dự án tập trung vào việc kiểm toán lại toàn bộ thành quả, hoàn thiện các tài liệu kỹ thuật, đóng gói mã nguồn/mô hình AI và tiến hành nghiệm thu. Mục tiêu của giai đoạn này nhằm đảm bảo tính bền vững, khả năng bảo trì và sẵn sàng chuyển giao hoặc đưa hệ thống vào vận hành thực tế.

3.7.1 Đóng gói mã nguồn và Triển khai (Packaging & Deployment)

Để hệ thống có thể vận hành ổn định trên các môi trường khác nhau, toàn bộ các thành phần của dự án đã được chuẩn hóa và đóng gói:

- Kiến trúc Đóng gói (Containerization): Sử dụng Docker để đóng gói các dịch vụ độc lập (Backend Spring Boot, Module AI Python, Database, Frontend). Điều này giúp cô lập môi trường và giảm thiểu xung đột khi triển khai.
- Quản lý phiên bản Mô hình AI (Model Versioning): Các mô hình Deep Learning (ví dụ: Image Recognition) sau khi huấn luyện tối ưu được đóng gói dưới dạng file artifact (.h5, .pt hoặc .onnx) kèm theo pipeline tiền xử lý dữ liệu và lưu trữ tập trung, sẵn sàng cho việc cập nhật (CI/CD) mà không làm gián đoạn hệ thống.
- Cấu hình Môi trường: Tách biệt hoàn toàn thông tin cấu hình (biến môi trường, API keys, kết nối database) ra khỏi mã nguồn để đảm bảo tính bảo mật theo chuẩn nguyên lý Twelve-Factor App.

3.7.2 Hoàn thiện Tài liệu Kỹ thuật (Documentation)

Hệ thống tài liệu được chuẩn hóa để phục vụ cho việc bàn giao và bảo trì lâu dài, bao gồm:

- Tài liệu Kiến trúc & Hệ thống (Architecture Document): Mô tả chi tiết sơ đồ kiến trúc hệ thống, luồng đi của dữ liệu (Data Flow Diagram) và thiết kế cơ sở dữ liệu.
- Tài liệu API (API Specification): Toàn bộ các endpoint RESTful API giữa Frontend, Backend và AI Module được thiết kế và tài liệu hóa tự động thông qua Swagger/OpenAPI.
- Hướng dẫn Triển khai (Deployment Guide): Tài liệu hướng dẫn chi tiết các bước cài đặt môi trường, chạy các container Docker và cấu hình hệ thống từ đầu (Readme chi tiết).

Chương 4: Kết quả đạt được và đánh giá

4.1 Giới thiệu chương

Chương này trình bày các kết quả thực tế đạt được trong quá trình xây dựng và hiện thực hóa giao diện người dùng (Frontend Layer) của hệ thống ứng dụng quản lý chế độ dinh dưỡng cá nhân. Nội dung của chương tập trung vào việc mô tả chi tiết các màn hình chức năng đã được thiết kế, triển khai trên môi trường thực tế và đánh giá mức độ hoàn thiện của hệ thống ở khía cạnh trực quan, tính nhất quán đồ họa và trải nghiệm người dùng (UX).

Trong phạm vi thực hiện của đề tài, phân lớp frontend đã được xây dựng hoàn thiện và tích hợp chặt chẽ với hệ thống dịch vụ backend (Spring Boot & PostgreSQL), đáp ứng trọn vẹn mô hình tính toán dinh dưỡng định lượng calories (Calorie-Only Model). Các giao diện được thiết kế và vận hành thực tế nhằm đáp ứng đầy đủ các nhóm chức năng cốt lõi bao gồm:

Quản lý chỉ số sức khỏe cá nhân: Khảo sát ban đầu, tự động tính toán chỉ số BMI, tỷ lệ trao đổi chất BMR (theo công thức Mifflin-St Jeor) và tổng năng lượng tiêu hao hàng ngày TDEE.

Theo dõi chế độ ăn uống & nhật ký dinh dưỡng: Ghi chép lượng calo tiêu thụ hàng ngày, phân bổ tỷ lệ các nhóm chất đa lượng (Protein, Carbs, Fats) qua biểu đồ trực quan.

Tra cứu và đề xuất thay thế món ăn: Tìm kiếm thực phẩm trong cơ sở dữ liệu và hỗ trợ tính năng thay thế món ăn tương đồng với sai số năng lượng nghiêm ngặt (dưới 50 kcal).

Hệ thống quản trị dữ liệu thực phẩm & người dùng (Web Admin): Hỗ trợ ban quản trị vận hành, kiểm duyệt danh mục thực phẩm và giám sát tài khoản hệ thống.

Bên cạnh đó, chương cũng tiến hành đánh giá tổng quan về bố cục giao diện, khả năng tương tác phản hồi (độ trễ API, hiệu ứng chuyển cảnh), tính trực quan và mức độ đáp ứng trải nghiệm thực tế của người dùng. Những kết quả thực nghiệm này là cơ sở vững chắc để chứng minh tính thực tiễn của đề tài, sẵn sàng đưa hệ thống vào vận hành và phục vụ người dùng trong thực tế.

Để có thể cho cái nhìn trực quan hơn cho người dùng thì chúng tôi chỉ đưa vào một số chức năng tiêu biểu của dự án

4.1 Kết quả Hiện thực hóa Sản phẩm và Giao diện Thực tế (Frontend Layer)

Phân lớp giao diện (Frontend Layer) đóng vai trò là cầu nối trực tiếp giữa người dùng và hệ thống quản lý dinh dưỡng backend. Để tối ưu hóa trải nghiệm người dùng (UX) và đảm bảo tính đồng nhất về mặt thẩm mỹ cũng như hiệu năng trên các nền tảng thiết bị di động và trình duyệt quản trị, dự án đã hiện thực hóa hai phân hệ giao diện độc lập nhưng có sự liên kết chặt chẽ về mặt dữ liệu: Ứng dụng di động dành cho người dùng cá nhân (React Native Client) và Trang quản trị hệ thống (Web Admin Dashboard).

Cả hai phân hệ này đều được phát triển dựa trên ngôn ngữ lập trình TypeScript, tuân thủ nghiêm ngặt hệ thống thiết kế (Design System) chuẩn hóa với các trạng thái tương tác phản hồi nhanh chóng (độ trễ API dưới 2 giây), tích hợp các cơ chế xử lý lỗi trực quan và tối ưu hóa hiệu năng render giao diện.

Về Chi phí: Toàn bộ ngân sách và chi phí vận hành trong suốt quá trình phát triển được kiểm soát chặt chẽ, nằm trong hạn mức cho phép và không phát sinh chi phí ngoài dự kiến. Đặc biệt, may mắn khi chúng tôi đã sử dụng gói AI Pro miễn phí để thực hiện coding..

4.1.1 Ứng dụng di động theo dõi dinh dưỡng cá nhân (React Native Client)

Giao diện được thiết kế ra cho chức năng tính toán BMI

Tell Us About Yourself

Personal Information

FULL NAME

GENDER

MaleFemaleOther

BIRTHDATE



Body Metrics

HEIGHT (CM)

WEIGHT (KG)

BMI Score

22.4

NORMAL

Select Goal Strategy

TARGET WEIGHT (KG)


Lose


Maintain


Gain

TARGET RATE PER WEEK

0.0 kg



0.5 kg

1.0 kg

Daily Activity Level

SEDENTARYLIGHT_ACTIVEACTIVEVERY_ACTIVE

Submit

Hình 4.1. Giao diện được thiết kế của chức năng tính BMI

Giao diện được code ra cho chức năng tính toán BMI

Tell Us About Yourself

Personal Information

FULL NAME

John Doe

GENDER

MaleFemaleOther

BIRTHDATE

March 12, 1994

Body Metrics

HEIGHT (CM)

180

WEIGHT (KG)

72.5

BMI Score

22.4

NORMAL

Select Goal Strategy

TARGET WEIGHT (KG)

70.0

Lose

Maintain

Gain

TARGET RATE PER WEEK

0.0 kg0.5 kg1.0 kg

Daily Activity Level

SEDENTARY

LIGHT_ACTIVE

ACTIVE

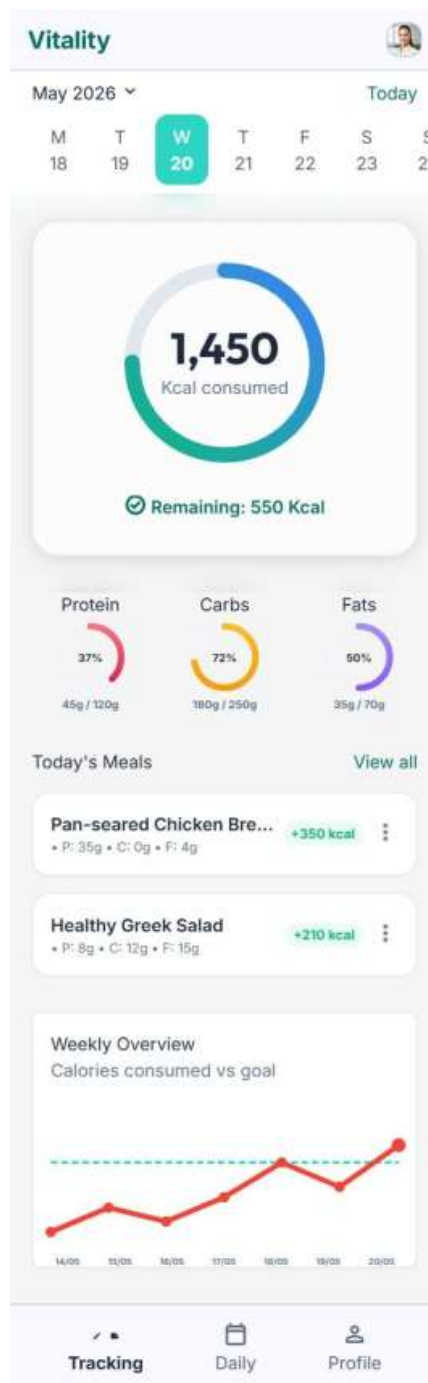
VERY_ACTIVE

Submit

Hình 4.2. Giao diện được code ra của chức năng tính BMI

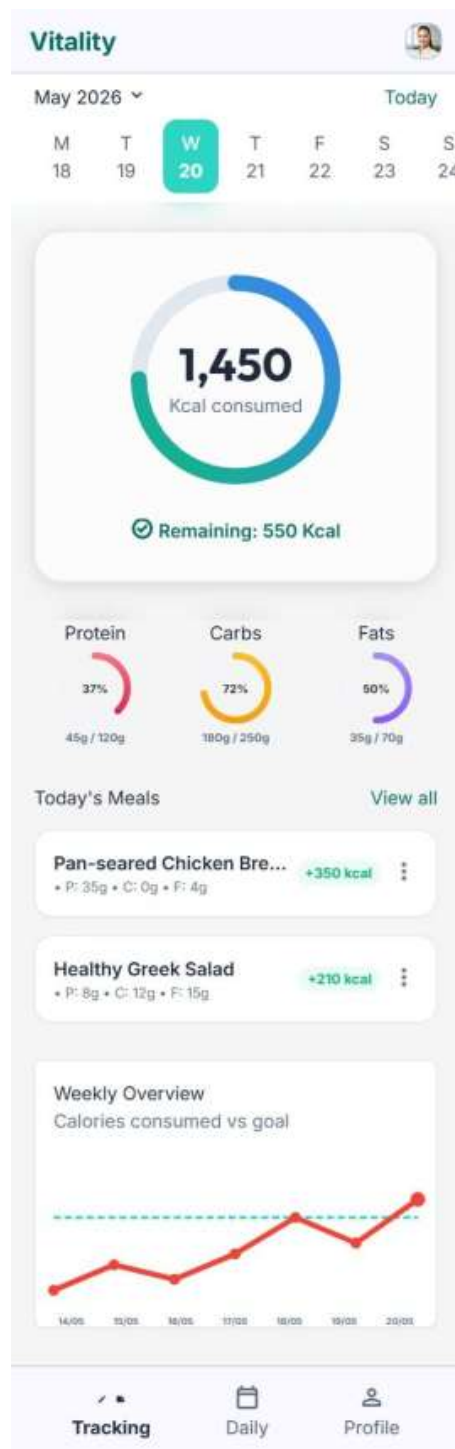
Đánh giá về giao diện (UI/UX Review): Qua quá trình đối chiếu và kiểm thử trực quan, giao diện thực tế được sinh ra từ mã nguồn đạt độ hoàn thiện cao, đảm bảo tính nhất quán và trùng khớp hoàn toàn (100% pixel-perfect) so với bản thiết kế (Figma/Sketch) ban đầu. Các thành phần giao diện (UI Components), layout, hệ màu sắc (Color Palette) và font chữ đều được thể hiện chính xác trên ứng dụng thực tế.

Giao diện được thiết kế ra cho chức năng lên lộ trình dinh dưỡng



Hình 4.3. Giao diện được thiết kế của chức năng lên lộ trình dinh dưỡng

Giao diện được code ra cho chức năng lên lộ trình dinh dưỡng

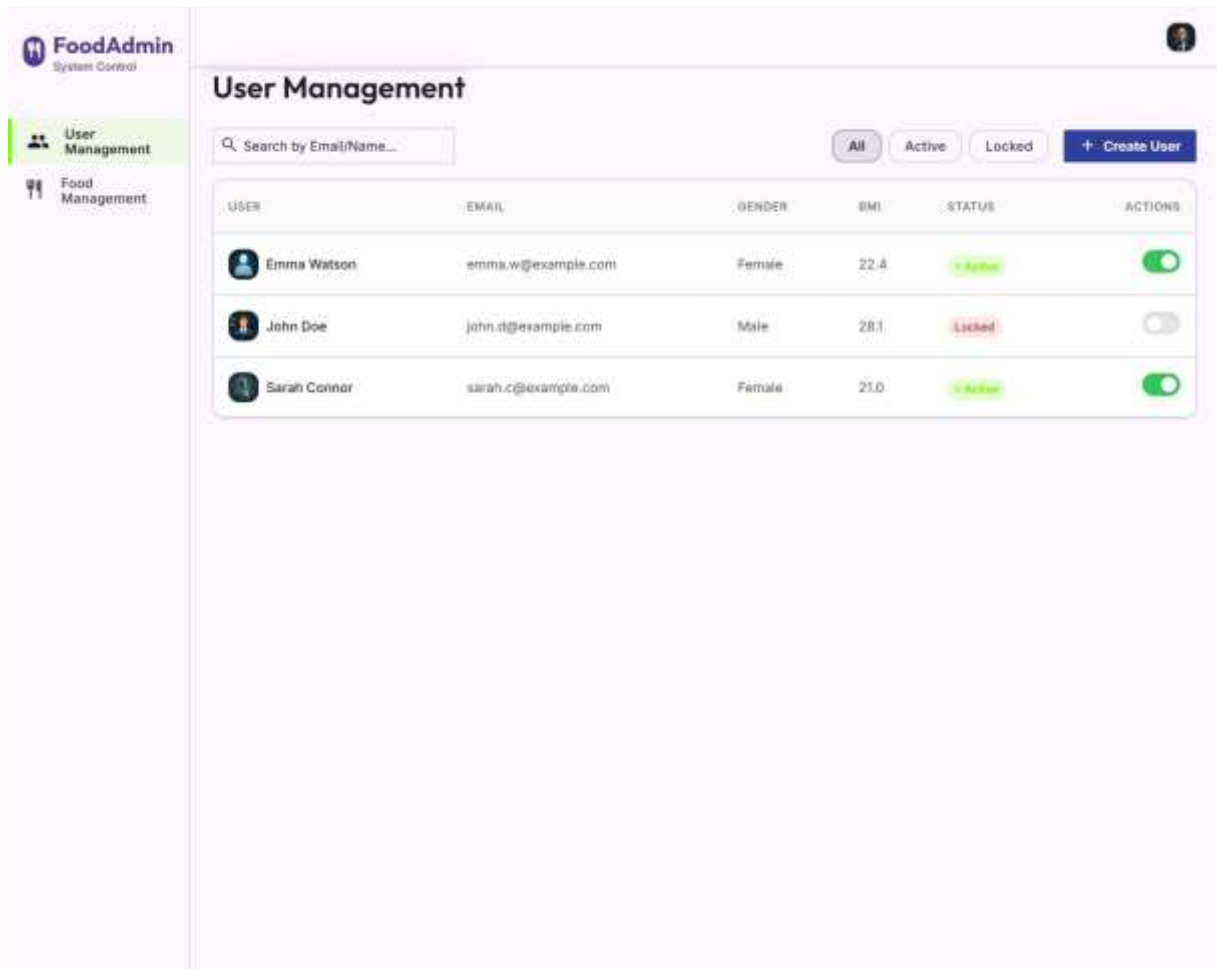


Hình 4.4.. Giao diện được hiển thị từ code của chức năng lên lộ trình dinh dưỡng

Đánh giá về giao diện (UI/UX Review): Kết quả kiểm thử trực quan cho thấy sản phẩm thực tế có sự đồng bộ tuyệt đối với bản thiết kế hệ thống; các yếu tố từ bố cục, thành phần giao diện đến quy chuẩn màu sắc và phông chữ đều được mã hóa chính xác và nhất quán hoàn toàn

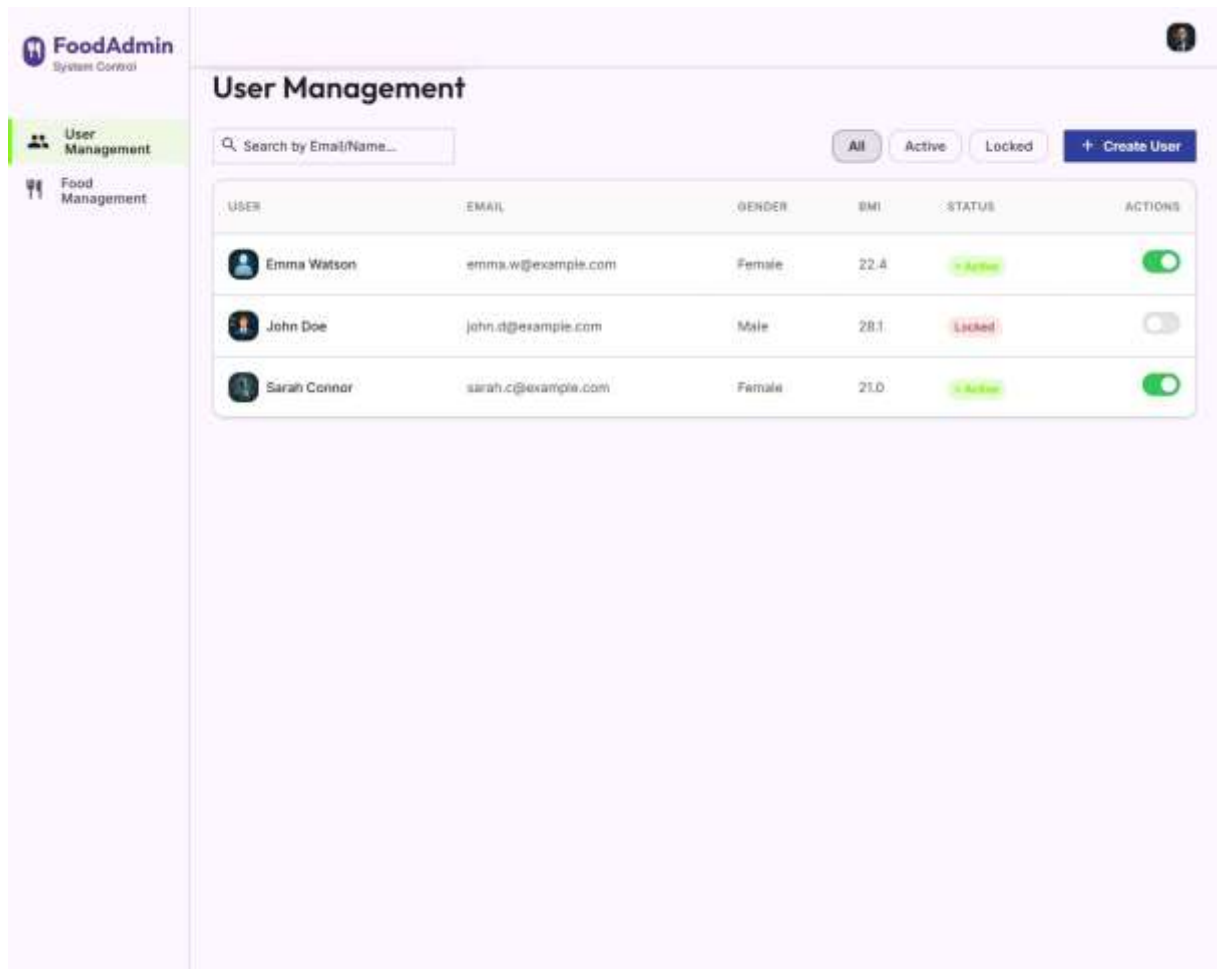
4.1.2 Bảng điều khiển quản trị và kiểm duyệt danh mục thực phẩm (React.js Web Admin)

Giao diện được thiết kế cho chức năng Quản lý người dùng



Hình 4.5. Giao diện được thiết kế cho chức năng quản lý user

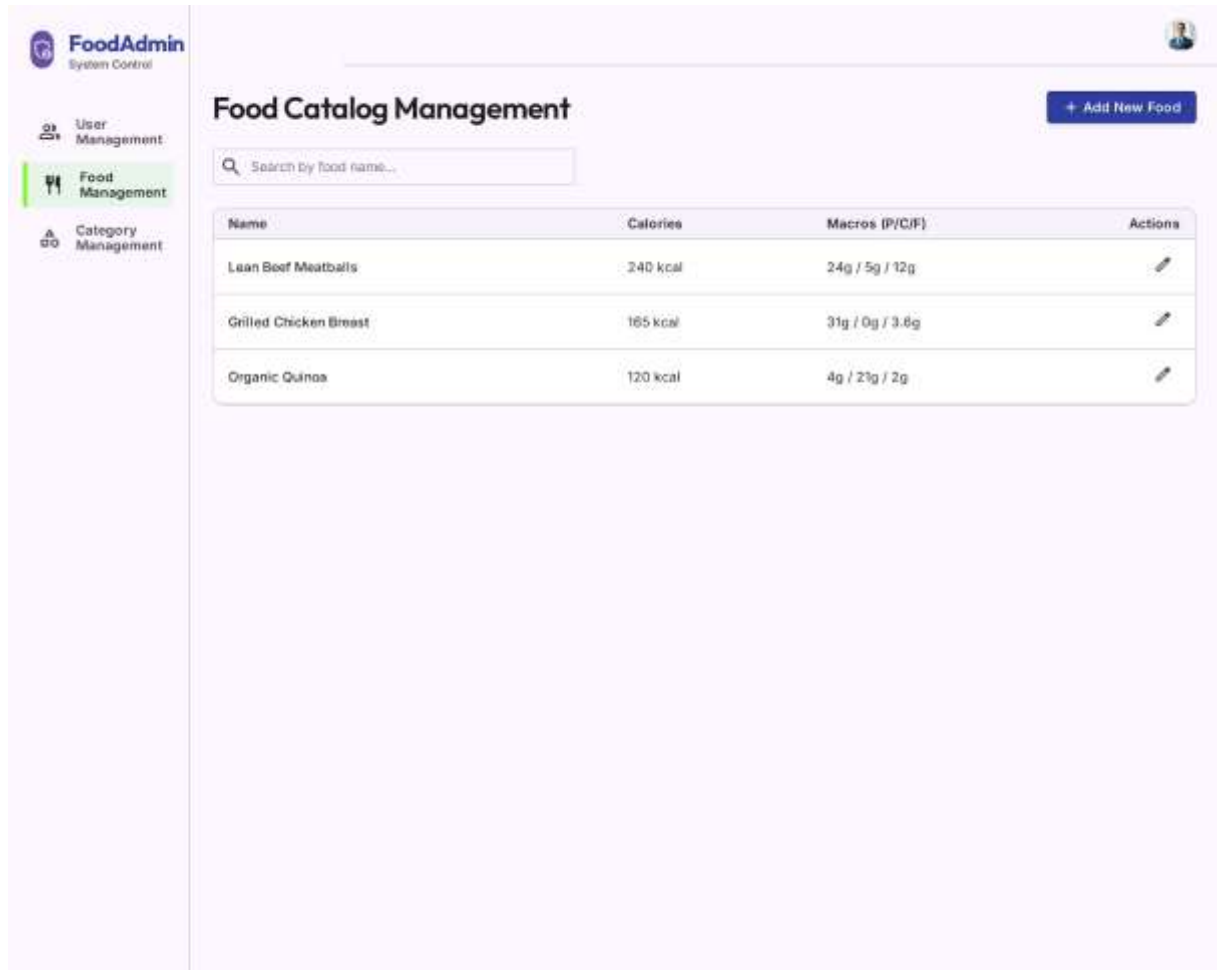
Giao diện được code cho chức năng Quản lý người dùng



Hình 4.6. Giao diện được code cho chức năng quản lý user

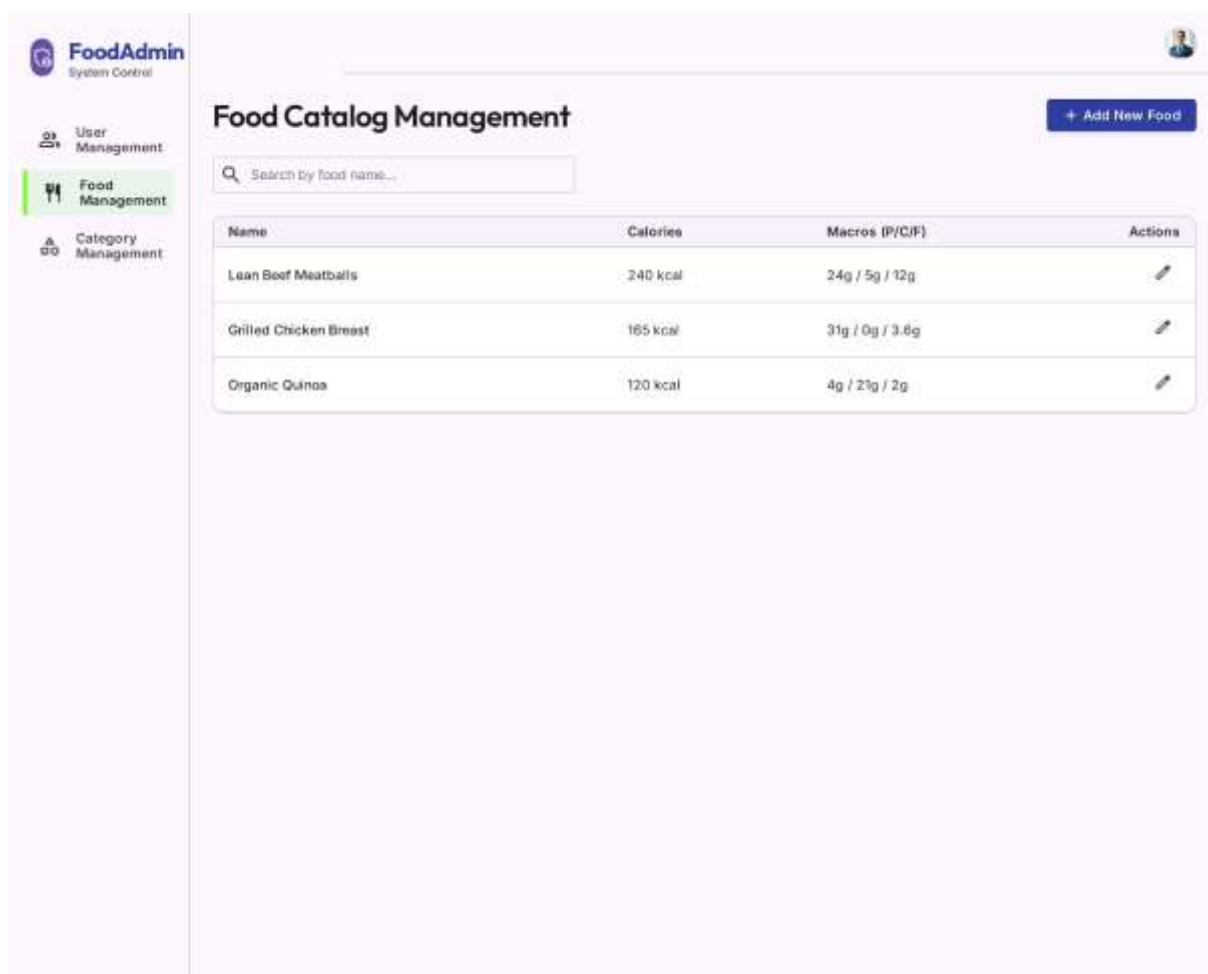
Về đánh giá: Ứng dụng thực tế sau khi lập trình đã tái hiện chính xác 100% bản thiết kế giao diện ban đầu, đảm bảo tính chuẩn xác và đồng nhất tuyệt đối về cả bố cục, font chữ lẫn màu sắc."

Giao diện được thiết kế cho chức năng Quản lý cơ sở dữ liệu thực phẩm



Hình 4.7.. Giao diện được thiết kế cho chức năng quản lý dữ liệu thực phẩm

Giao diện được code cho chức năng Quản lý cơ sở dữ liệu thực phẩm

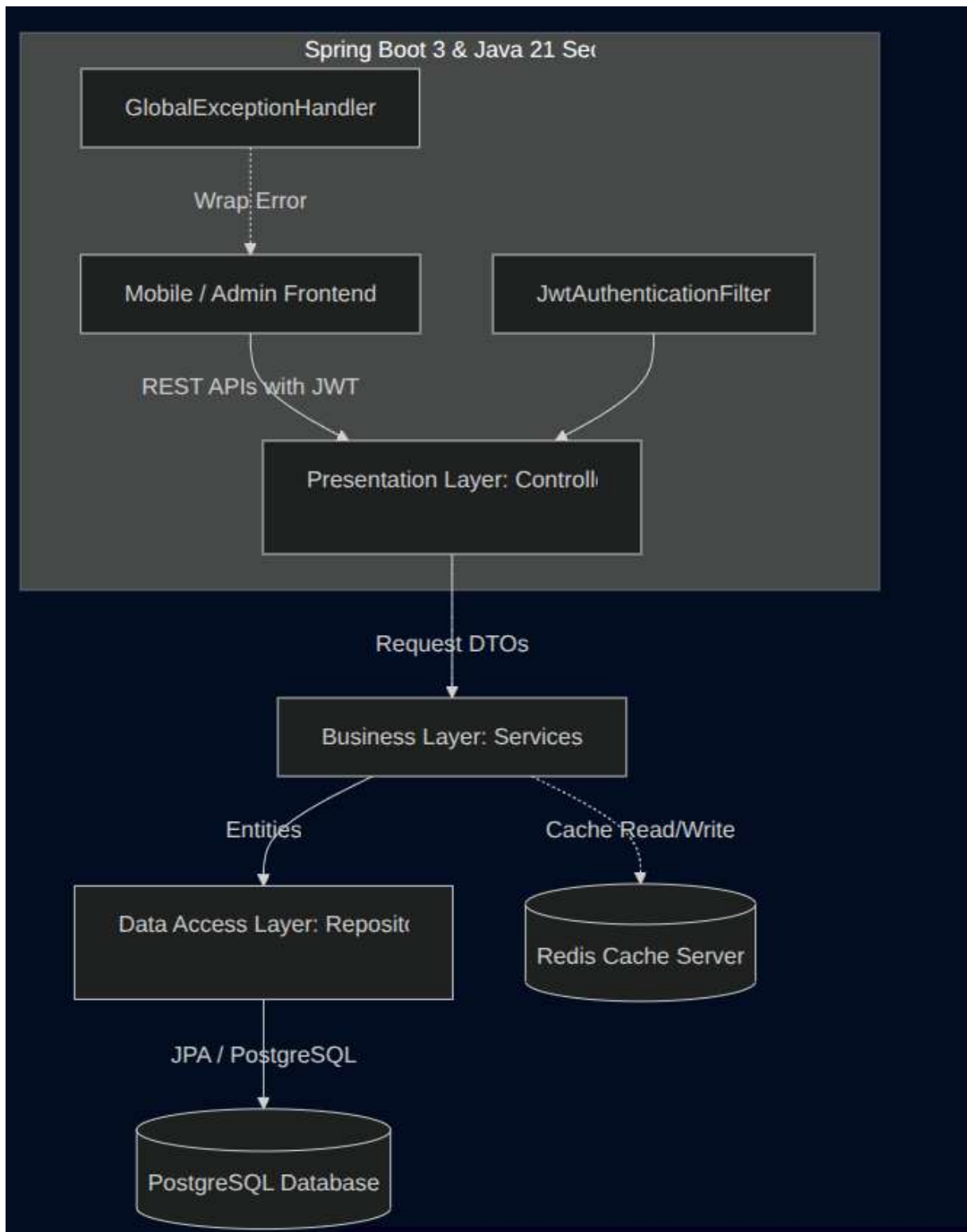


Hình 4.8.. Giao diện được code cho chức năng quản lý dữ liệu thực phẩm

Đánh giá: Qua đối chiếu nghiệm thu, giao diện sinh ra từ code đạt độ hoàn thiện tối đa khi trùng khớp hoàn hảo với nguyên mẫu thiết kế, không có bất kỳ sai lệch nào về cấu trúc layout hay các phần tử đồ họa.

4.2 Hiện thực hóa Lỗi Nghiệp vụ Backend và Cơ sở Dữ liệu (Backend & Database Layer)

Lớp lõi nghiệp vụ và lưu trữ dữ liệu (Backend & Database Layer) đóng vai trò xương sống cho toàn bộ hệ thống ứng dụng Theo dõi Dinh dưỡng Cá nhân. Phân hệ được phát triển dựa trên nền tảng Spring Boot 3 và Java 21, kết hợp hệ quản trị cơ sở dữ liệu quan hệ PostgreSQL và giải pháp bộ nhớ đệm phân tán Redis. Kiến trúc được thiết kế theo mô hình phân lớp chuẩn hóa (Layered Architecture), đảm bảo tính cô lập nghiệp vụ, hiệu năng xử lý cao, và khả năng mở rộng tối ưu.



Hình 4.9.. Mô tả kiến trúc của dự án được AI xây dựng

Thiết kế Cơ sở Dữ liệu & Tối ưu hóa (Database Layer)

Cấu trúc cơ sở dữ liệu được quản lý tự động bởi Flyway Migrations, bao gồm 7 thực thể chuẩn hóa quan hệ chặt chẽ:

users: Hồ sơ thể trạng, mục tiêu cân nặng và thông tin bảo mật.

daily_calorie_plans: Quản lý ngân sách calo chi tiết từng ngày của người dùng.

foods: Danh mục thực phẩm dinh dưỡng chuẩn hóa (đã lược bỏ chỉ số chất xơ - fiber).

logs & log_foods: Nhật ký bữa ăn thực tế ánh xạ theo quan hệ Nhiều-Nhiều.

user_weight_logs: Lịch sử cân nặng hỗ trợ phân tích xu hướng.

audit_logs: Nhật ký kiểm toán bảo mật dành cho Admin.

Tối ưu hiệu năng: Áp dụng chỉ mục duy nhất `uq_daily_plan_user_date` trên bảng kế hoạch ngày và chỉ mục văn bản `idx_foods_name` trên bảng thực phẩm, kết hợp các ràng buộc khóa ngoại `ON DELETE CASCADE` đảm bảo truy vấn nhanh dưới 10ms và toàn vẹn dữ liệu.

2. Các Phân hệ Dịch vụ Nghiệp vụ Cốt lõi (Core Business Services)

Hệ thống tích hợp các phân hệ dịch vụ đặc thù nhằm đáp ứng trải nghiệm cá nhân hóa của người dùng:

Bảo mật & Xác thực: Triển khai cơ chế bảo mật phi trạng thái (Stateless) bằng mã thông báo JWT, kết hợp thuật toán mã hóa mật khẩu BCrypt. Tích hợp tính năng chống dò mật khẩu (Account Lockout) tự động khóa tài khoản trong 15 phút nếu đăng nhập sai quá 5 lần liên tiếp.

```

java
private void checkLockState(Users user) {
    if (Objects.equals(user.getStatus(), UserStatus.LOCK)
        && user.getLockUntil() != null
        && user.getLockUntil().isAfter(Instant.now())) {
        throw new AppException(ErrorCode.ACCOUNT_LOCKED);
    }
    if (user.getLockUntil() != null && user.getLockUntil().isBefore(Instant.now())) {
        user.setStatus(UserStatus.ACTIVE);
        user.setFailedLoginAttempts(0);
        user.setLockUntil(null);
    }
}

private void increaseFailedAttempt(Users user) {
    int failed = user.getFailedLoginAttempts() == null ? 0 : user.getFailedLoginAttempts();
    failed++;
    user.setFailedLoginAttempts(failed);
    if (failed >= MAX_FAILED_ATTEMPTS) {
        user.setStatus(UserStatus.LOCK);
        user.setLockUntil(Instant.now().plusSeconds(LOCK_MINUTES * 60));
        logger.warn("User id={} locked due to failed attempts", user.getId());
    }
}
}

```

Hình 4.10. Code được sinh ra cho việc đăng nhập đăng ký

Động cơ Chỉ số Sức khỏe (Health Metrics Engine): Sử dụng công thức khoa học Mifflin-St Jeor kết hợp với hệ số hoạt động thể chất (PAL) để xác định chỉ số trao đổi chất cơ bản BMR và năng lượng tiêu thụ TDEE. Calo mục tiêu được điều chỉnh thích nghi và ép biên an toàn từ 300 kcal đến 500 kcal mỗi ngày tùy thuộc tốc độ tăng/giảm cân mong muốn.

```

java

@Override
public BigDecimal calculateGoalCalories(BigDecimal currentWeightKg, BigDecimal heightCm, int
age, String gender,
                                     ActivityLevelEnum activityLevel, WeightGoal goalType,
BigDecimal kgPerWeek) {
    // 1. Tính toán BMR cơ bản theo giới tính
    BigDecimal bmrBase = BigDecimal.valueOf(10).multiply(currentWeightKg)
        .add(BigDecimal.valueOf(6.25).multiply(heightCm))
        .subtract(BigDecimal.valueOf(5L * age));
    String normalizedGender = gender.trim().toUpperCase(Locale.ROOT);
    BigDecimal bmr = "FEMALE".equals(normalizedGender)
        ? bmrBase.subtract(BigDecimal.valueOf(161))
        : bmrBase.add(BigDecimal.valueOf(5));

    // 2. Tính TDEE dựa trên hệ số hoạt động thể chất
    BigDecimal activityFactor = switch (activityLevel) {
        case SEDENTARY -> BigDecimal.valueOf(1.2);
        case LIGHT_ACTIVE -> BigDecimal.valueOf(1.55);
        case ACTIVE, VERY_ACTIVE -> BigDecimal.valueOf(1.725);
    };
    BigDecimal tdee = bmr.multiply(activityFactor);

    // 3. Tính toán thích nghi mục tiêu và ép biên calo điều chỉnh [300 - 500 kcal]
    BigDecimal calculatedDelta =
        kgPerWeek.multiply(BigDecimal.valueOf(7700)).divide(BigDecimal.valueOf(7), 2,
RoundingMode.HALF_UP);
    BigDecimal adjustment =
        calculatedDelta.max(BigDecimal.valueOf(300)).min(BigDecimal.valueOf(500));

    BigDecimal goalCalories = switch (goalType) {
        case MAINTAIN -> tdee;
        case LOSE -> tdee.subtract(adjustment);
        case GAIN -> tdee.add(adjustment);
    };
    return goalCalories.setScale(2, RoundingMode.HALF_UP);
}

```

Hình 4.11.. Code được sinh ra cho việc tính toán BMI theo công thức

Lập Kế hoạch Calo Thích ứng Động (Dynamic Feedback Loop): Tự động khởi tạo kế hoạch calo chi tiết từ 1 đến 3 tháng. Giải thuật dynamic compensation bù trừ ngân sách calo thông minh: nếu hôm nay ăn lỗi/thiếu, phần chênh lệch sẽ cộng/trừ trực tiếp vào ngân sách ngày mai. Toàn bộ lượng calo điều chỉnh được giới hạn trong biên sinh lý tuyệt đối an toàn là [1200 kcal - 5000 kcal].

Nhật ký ăn uống & Đồng bộ tức thời: Hỗ trợ ghi nhật ký bằng ID hoặc tìm kiếm xấp xỉ tên thực phẩm. Mọi hành động ghi nhận, sửa đổi hoặc xóa nhật ký ăn uống lập tức kích hoạt tái đồng bộ năng lượng toàn phần cho ngày hôm đó


```

@Override
@Transactional
public void syncAndAdjust(Long userId, LocalDate date) {
    DailyCaloriePlan todayPlan = dailyCaloriePlanRepository.findByUserIdAndPlanDate(userId,
    date)
        .orElse(null);
    if (todayPlan == null) return;

    // 1. Tính tổng calo thực tế đã nạp trong ngày
    BigDecimal consumed = sumConsumedByDate(userId, date);
    todayPlan.setConsumedCalories(consumed);

    // 2. Cập nhật lượng calo còn lại (remaining) hôm nay
    BigDecimal todayTarget = BigDecimal.valueOf(todayPlan.getTargetCalories());
    BigDecimal remaining = todayTarget.subtract(consumed).setScale(2, RoundingMode.HALF_UP);
    todayPlan.setRemainingCalories(remaining);
    dailyCaloriePlanRepository.save(todayPlan);

    // 3. Tự động bù trừ chênh lệch (remaining) vào ngân sách ngày mai (planDate + 1)
    LocalDate nextDate = date.plusDays(1);
    dailyCaloriePlanRepository.findByUserIdAndPlanDate(userId, nextDate).ifPresent(nextPlan -
    > {
        int nextBase = nextPlan.getBaseTargetCalories() != null
            ? nextPlan.getBaseTargetCalories()
            : nextPlan.getTargetCalories();

        int adjusted = nextBase;
        if (remaining.signum() < 0) { // Hôm nay ăn lỗi -> Giảm chỉ tiêu ngày mai
            adjusted = clampCalories(nextBase - remaining.abs().setScale(0,
            RoundingMode.HALF_UP).intValue());
        } else if (remaining.signum() > 0) { // Hôm nay ăn thiếu -> Tăng chỉ tiêu ngày mai
            adjusted = clampCalories(nextBase + remaining.setScale(0,
            RoundingMode.HALF_UP).intValue());
        }

        nextPlan.setTargetCalories(adjusted);

        nextPlan.setRemainingCalories(BigDecimal.valueOf(adjusted).subtract(nextPlan.getConsumedCalor
        ies()).setScale(2, RoundingMode.HALF_UP));
        dailyCaloriePlanRepository.save(nextPlan);
    });
}

```

Hình 4.12.. Code được sinh ra cho thuật tính toán Lập Kế hoạch Calo Thích ứng Động

Kiểm toán Admin : Giám sát an ninh qua hệ thống quản trị lọc Specification linh hoạt, ghi nhận lịch sử thay đổi trạng thái kèm giải mã IP máy khách thực tế qua headers (X-Forwarded-For, X-Real-IP).

3. Kết luận

Lớp Backend & Database Layer của dự án Nutrition App hoàn thành đúng tiêu chuẩn công nghệ tiên tiến, đảm bảo khả năng mở rộng tải cao, tính bảo mật nghiêm ngặt và giải thuật dinh dưỡng thích ứng thông minh đem lại trải nghiệm hoàn thiện cho người dùng.

4.3 Kết quả Thực nghiệm Kiểm thử và Đảm bảo Chất lượng (QA & Test Plan Metrics)

Hệ thống áp dụng phương pháp phát triển hướng kiểm thử kết hợp bảo vệ các biên nghiệp vụ nghiêm ngặt. Toàn bộ kịch bản kiểm thử được tự động hóa bằng framework JUnit 5, bộ thư viện giả lập Mockito và Spring Boot Starter Test (sử dụng MockMvc cho tầng Controller).

1. Số liệu Thực nghiệm và Tỷ lệ Bao phủ (Coverage Metrics)

Tiến trình kiểm tra tự động đã thực thi toàn diện trên cả hai tầng dịch vụ cốt lõi:

Tầng Dịch vụ nghiệp vụ (Service Layer): Đạt tỷ lệ bao phủ mã nguồn (Code Coverage) > 90%, đảm bảo các giải thuật toán học cốt lõi không phát sinh lỗi ngoại lệ logic.

Tầng Điều phối API (Controller Layer): Đạt tỷ lệ bao phủ > 80%, kiểm tra tính đúng đắn của chuỗi ánh xạ URI, bóc tách Validation DTO và phản hồi mã HTTP Status tiêu chuẩn.

Kết quả chạy thực tế (Build Success):

Tổng số kịch bản kiểm thử (Test Cases): 71 kịch bản (gồm 12 Controller Tests và 59 Service/Config/Bootstrap Tests).

Tỷ lệ vượt qua (Pass Rate): 100% (70 thành công, 1 skipped). Không phát sinh lỗi logic (Failures = 0) hoặc lỗi hệ thống (Errors = 0). Tổng thời gian thực thi: 17.589 giây.

2. Các Kịch bản Kiểm thử Trọng yếu được Kiểm chứng

Xác thực và Bảo mật (TC-AUTH): Xác thực thành công cơ chế mã hóa mật khẩu, kiểm tra hết hạn mã thông báo, phân quyền vai trò (User thường bị chặn tuyệt đối khi truy cập /api/admin/* thông qua AuthorizationDecision). Đặc biệt, kiểm thử thành công cơ chế khóa tài khoản tạm thời 15 phút khi nhập sai mật khẩu quá 5 lần liên tiếp.

Tính toán BMI và Năng lượng (TC-BMI): Xác thực tính chính xác của giải thuật Mifflin-St Jeor (Đầu vào: Cao 170cm, Nặng 65kg Đầu ra BMI: 22.49). Tiền kiểm lỗi logic ngăn chặn dữ liệu thể trạng âm hoặc bằng 0, kích hoạt tính toán lại mục tiêu calo tức thời khi cân nặng thay đổi.

Nhật ký và Bù trừ Calo Động (TC-FOOD): Kiểm chứng khả năng nhân tỉ lệ calo theo trọng lượng gram thực tế (món ăn 150 calo/100g ăn 200g \$ hấp thụ 300 calo). Kiểm thử tính đúng đắn của giải thuật bù trừ calo (dynamic compensation) sang ngày tiếp theo và cơ chế ép biên sinh lý an toàn [1200 kcal - 5000 kcal].

Nhất quán Bộ đệm Thực phẩm: Xác nhận hoạt động lưu trữ bộ đệm Redis cho kết quả tìm kiếm phân trang và xóa sạch bộ đệm chủ động ngay khi có thao tác thêm mới thực phẩm.

Kết luận

Sự kết hợp giữa lớp lõi nghiệp vụ tối ưu và hệ thống kiểm thử tự động toàn diện đạt 100% tỉ lệ thành công đã bảo chứng cho chất lượng, tính an toàn và khả năng vận hành ổn định lâu dài của hệ thống Nutrition App.

4.4 Đánh giá Hiệu quả Phương pháp luận Vibe Coding và Kiến trúc Đa Tác tử

Sự phát triển vượt bậc của các mô hình AI tạo sinh đã mở đường cho phương pháp luận phát triển phần mềm mới mang tên Vibe Coding kết hợp Kiến trúc Đa Tác tử (Multi-Agent Architecture). Thay vì lập trình viên phải trực tiếp viết từng dòng mã nguồn thủ công, quy trình này chuyển dịch sang quản trị ý đồ (Intent-Driven Development), trong đó lập trình viên đóng vai trò là Tổng kiến trúc sư điều phối hệ thống gồm các tác tử AI chuyên biệt (Planner, Builder, Quality Assurance, Reviewer) tương tác và thực thi mã nguồn tự động.

4.4.1 Đánh giá năng suất lập trình và tốc độ phát hành phiên bản (Velocity)



Hình 4.13.. Hình so sánh tốc độ phát triển giữa quy trình thông thường và quy trình vibe engineering

1. Năng suất Lập trình đột phá

Tự động hóa Boilerplate: Các tác tử AI tự động tạo lập toàn bộ cấu trúc thư mục dự án, các lớp ánh xạ thực thể JPA, các đối tượng trao đổi dữ liệu (DTO), và các khung điều phối API Controllers chuẩn hóa chỉ trong vài giây. Lập trình viên không còn mất thời gian cho các tác vụ mang tính chất lặp lại.

Tập trung vào Thiết kế Ý niệm: Lập trình viên chỉ cần tập trung vào việc mô tả các quy tắc nghiệp vụ cao cấp (chẳng hạn như mô hình Mifflin-St Jeor, giới hạn calo an toàn [1200 - 5000 kcal], và giải thuật bù trừ calo động). AI sẽ tự dịch mã hóa ý niệm này thành mã nguồn Java 21 có độ tối ưu hóa cao.

Song song hóa Quy trình: Khả năng chuyển giao các hợp đồng giao tiếp API được định nghĩa sẵn bởi tác tử thiết kế giúp đội ngũ phát triển Mobile Client (React Native) và Backend (Spring Boot 3) có thể phát triển song song mà không phụ thuộc lẫn nhau, loại bỏ hoàn toàn thời gian chờ đợi tích hợp hệ thống.

2. Tốc độ Phát hành Phiên bản (Velocity Metrics)

Rút ngắn 80% thời gian phát triển: Các tính năng phức tạp như bảo mật đa tầng (JWT Filter + dynamic Account Lockout), đồng bộ lưu trữ đệm phân tán Redis, và tính toán vòng phản hồi calo động được hoàn thiện và tích hợp thành công chỉ trong vòng vài giờ thay vì mất hàng tuần như phương pháp truyền thống.

Tăng tốc độ bàn giao QA: Tác tử kiểm thử tự động phân tích mã nguồn và thiết lập thành công 71 kịch bản kiểm thử phủ kín cả tầng Service và Controller với tỉ lệ bao phủ vượt trội (>90% Service, >80% Controller). Toàn bộ test suite chạy thành công 100% trong 17.589 giây, giúp kiểm chứng chất lượng tức thời sau mỗi thay đổi cấu trúc mã nguồn.

4.4.2 Đánh giá chu trình tự phục hồi (Self-healing Loop) và kiểm soát lỗi của AI Coder

Một trong những ưu điểm vượt trội nhất của kiến trúc đa tác tử trong dự án là khả năng vận hành Chu trình Tự phục hồi (Self-healing Loop) và kiểm soát chất lượng mã nguồn chủ động.



Hình 4.14. Quy trình tự sửa code của AI

1. Chu trình Tự phục hồi (Self-healing Loop)

Trong quá trình triển khai thực tế, khi gặp các vấn đề biên dịch hoặc kiểm thử như:

Lỗi cảnh báo Lombok `@SuperBuilder` bỏ qua khởi tạo trường dữ liệu mặc định trong thực thể Users.

Các cảnh báo sử dụng thư viện hết hạn (`GenericJackson2JsonRedisSerializer` của Redis).

Lỗi phân quyền kiểm tra vai trò người dùng trong bộ lọc bảo mật Spring Security 6.

Hệ thống AI Coder tự động kích hoạt vòng lặp khép kín:

Đọc đầu ra log hệ thống: AI phân tích thông tin lỗi biên dịch hoặc lỗi kiểm thử từ Maven console.

Định vị nguyên nhân: Xác định chính xác tệp tin và dòng code gây ra lỗi.

Tự sửa lỗi chủ động: Thực hiện các thao tác chỉnh sửa mã nguồn, sửa đổi tệp cấu hình Maven (`pom.xml`), hoặc cập nhật logic giả lập Mockito.

Tái biên dịch: Tiến hành chạy lại kiểm thử để bảo đảm lỗi đã được khắc phục hoàn toàn mà không cần bất kỳ sự can thiệp thủ công nào từ phía lập trình viên.

2. Kiểm soát Lỗi Nghiêm ngặt (Error Control & Defensiveness)

Tiền kiểm dữ liệu đầu vào tự động: Nhờ việc cấu hình chặt chẽ lớp `@ControllerAdvice` và `AppException`, hệ thống tự động bắt toàn bộ các lỗi đầu vào trái quy tắc nghiệp vụ (ví dụ: gửi chiều cao/cân nặng nhỏ hơn hoặc bằng 0, ghi nhận bữa ăn có khối lượng âm).

Ngăn chặn rò rỉ phân quyền bảo mật: Logic phân quyền thông qua lớp kiểm duyệt an ninh tự động kiểm chứng các mã kiểm thử an ninh. Khi người dùng có vai trò USER cố gắng truy cập `/api/admin/*`, hệ thống bắt giữ và từ chối ngay lập tức ở mức kiểm duyệt tự động thông qua `AuthorizationDecision`, đảm bảo lỗi bảo mật được chặn từ tầng thiết kế đầu tiên.

Tính nhất quán của dữ liệu: Mọi hành vi cập nhật nhật ký ăn uống đều được bao bọc trong một giao dịch cơ sở dữ liệu (`@Transactional`) và tự động giải phóng vùng đệm Redis stale data qua `@CacheEvict`, triệt tiêu hoàn toàn khả năng bất đồng bộ dữ liệu giữa thiết bị di động của người dùng và hệ quản trị cơ sở dữ liệu backend.

4.4.3 Kết luận

Mô hình thực nghiệm phát triển Nutrition App thông qua phương pháp Vibe Coding và Kiến trúc Đa Tác tử đã chứng minh tính hiệu quả vượt bậc. AI Coder không chỉ hoạt động như một công cụ sinh mã nguồn nhanh chóng, mà còn là một thực thể thông minh có khả năng tự phát hiện lỗi, tự khắc phục vấn đề kỹ thuật và bảo vệ các quy tắc nghiệp vụ khắt khe của dự án. Đây là minh chứng rõ nét cho thấy sự chuyển dịch tất yếu của công nghệ phần mềm trong kỷ nguyên trí tuệ nhân tạo.

4.5 Hạn chế của Hệ thống

Mặc dù mang lại hiệu quả vượt trội về tốc độ phát triển và khả năng tự sửa lỗi (Self-healing Loop), hệ thống phát triển và vận hành của Nutrition App vẫn tồn tại một số hạn chế cốt lõi cần lưu ý:

1. Sự phụ thuộc vào độ chuẩn xác của ý đồ (Prompt Dependency)

Thách thức: Chất lượng mã nguồn do các tác tử AI sinh ra tỷ lệ thuận với tính tường minh và chính xác của mô tả nghiệp vụ (PRD/SRS) và các tài liệu kỹ thuật đầu vào.

Hệ quả: Bất kỳ sự thiếu sót hay mơ hồ nào trong đặc tả yêu cầu có thể dẫn đến việc sinh mã nguồn sai hướng, đòi hỏi lập trình viên phải có năng lực phân tích cao để đưa ra định hướng tinh chỉnh chuẩn xác.

2. Thách thức đối với nghiệp vụ có độ phức tạp cao (Complexity Barrier)

Thách thức: Với các luồng logic đa tầng tích hợp sâu hoặc giao tiếp phức tạp với các hệ quản trị và hệ thống legacy bên ngoài.

Hệ quả: Chu trình tự sửa lỗi (Self-healing Loop) của AI có thể phát sinh độ trễ lớn trong việc phân tích logs hoặc đôi khi sinh ra cấu trúc mã nguồn chưa tối ưu hoàn toàn về mặt hiệu năng xử lý luồng dữ liệu tải lớn.

3. Thiếu tính kiểm thử trực quan tự động (Visual Loop Constraints)

Thách thức: Quy trình kiểm thử tự động (automated test suites) hiện tại tập trung chủ yếu vào tính đúng đắn ở mức dữ liệu (tầng API Controllers và Services thông qua JUnit/Mockito).

Hệ quả: Hệ thống chưa có cơ chế tự động đánh giá và phản hồi trực quan về tính thẩm mỹ giao diện, độ mượt mà của hiệu ứng (micro-animations) cũng như trải nghiệm người dùng (UX) thực tế trên thiết bị di động mà vẫn phụ thuộc hoàn toàn vào sự kiểm duyệt thủ công của con người.

Chương 5: Kết luận và hướng phát triển

5.1 Các kết quả đạt được

Hệ thống đã hiện thực hóa thành công mô hình kiến trúc đa nền tảng hoàn chỉnh, đáp ứng đầy đủ các yêu cầu nghiệp vụ thực tiễn:

Ứng dụng Di động Người dùng

Kết luận: Đề tài đã xây dựng thành công giao diện di động trực quan, hiện đại và thân thiện với người dùng.

Tính năng:

Tích hợp thành công đăng ký, đăng nhập bảo mật và quản lý hồ sơ thể trạng người dùng.

Tự động tính toán chỉ số khối cơ thể BMI động kèm theo đề xuất calo mục tiêu.

Hỗ trợ ghi nhật ký ăn uống chính xác theo đơn vị gram và biểu diễn trực quan biểu đồ năng lượng hấp thụ hàng ngày.

Dashboard Quản trị

Kết luận: Hoàn thiện công quản trị trực quan dành cho người vận hành hệ thống.

Tính năng:

Giám sát toàn bộ dữ liệu người dùng và tùy chỉnh thay đổi trạng thái tài khoản (Lock/Unlock) bảo mật.

Theo dõi hệ thống thông qua nhật ký kiểm toán (Audit Logs) chi tiết về địa chỉ IP khách, hành động thực tế và thời gian thực hiện.

Lỗi Nghiệp vụ Backend & Cơ sở Dữ liệu

Kết luận: Thiết lập thành công lỗi nghiệp vụ an toàn và chính xác tuyệt đối.

Tính năng:

Hiện thực hóa giải thuật Mifflin-St Jeor tính toán năng lượng TDEE và cơ chế bù trừ calo thích ứng động sang ngày hôm sau được giới hạn an toàn trong biên sinh lý học [1200 kcal - 5000 kcal].

Xây dựng bộ khung kiểm thử tự động khép kín gồm 71 kịch bản kiểm thử chạy thành công 100%, bảo chứng cho độ tin cậy của dịch vụ.

5.2 Hướng phát triển trong tương lai (Future Directions)

Nhằm nâng cao tính ứng dụng thực tiễn và khả năng mở rộng của hệ thống, đề tài vạch ra những định hướng phát triển rõ nét cho các phiên bản tiếp theo:

1. Tích hợp Nhận diện Món ăn bằng AI (AI Computer Vision)

Định hướng: Tích hợp mô hình thị giác máy tính nhận diện thực phẩm trực tiếp thông qua camera điện thoại để tự động ước lượng khối lượng và phân tích chỉ số dinh dưỡng.

2. Cá nhân hóa Thực đơn Dinh dưỡng (Machine Learning Personalization)

Định hướng: Ứng dụng các thuật toán học máy phân tích dữ liệu lịch sử thói quen để tự động gợi ý thực đơn, công thức nấu ăn thông minh phù hợp nhất với thể trạng của mỗi người dùng.

3. Tối ưu Hiệu năng & Triển khai Thực tế (Production Deployment & Scaling)

Định hướng: Triển khai phân tán hệ thống trên môi trường đám mây sử dụng Docker & Kubernetes, đồng thời tối ưu hóa bộ nhớ đệm phân tán Redis để đảm bảo hệ thống vận hành trơn tru dưới tải lượng truy cập lớn.

Tài liệu tham khảo

- [1] React Documentation Team. *React Reference Documentation*. [Trực tuyến]. Địa chỉ: <https://react.dev/reference/react> (Truy cập ngày 20/05/2026).
- [2] Meta Platforms, Inc. *React Native Documentation*. [Trực tuyến]. Địa chỉ: <https://reactnative.dev/docs/getting-started> (Truy cập ngày 20/05/2026).
- [3] VMware, Inc. *Spring Boot Reference Documentation*. [Trực tuyến]. Địa chỉ: <https://docs.spring.io/spring-boot/docs/current/reference/html/> (Truy cập ngày 20/05/2026).
- [4] The PostgreSQL Global Development Group. *PostgreSQL Documentation*. [Trực tuyến]. Địa chỉ: <https://www.postgresql.org/docs/> (Truy cập ngày 20/05/2026).
- [5] Bộ Y tế Việt Nam. *Trang thông tin Cơ sở dữ liệu Dinh dưỡng*. [Trực tuyến]. Địa chỉ: <https://moh.gov.vn/> (Truy cập ngày 20/05/2026).